complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + collect() accumulate elements

filter() null checks + collect() accumulate elements

filter() comparisons + collect() accumulate elements

filter() method calls returning boolean + collect() accumulate elements

distinct() rely on equals() / hashcode() + collect() accumulate elements

distinct() immutable objects + collect() accumulate elements

map() transform objects + collect() accumulate elements

map() create new objects + collect() accumulate elements

map() call pure functions + collect() accumulate elements

flatmap() return stream + collect() accumulate elements

flatmap() split data + collect() accumulate elements

flatmap() expand collections + collect() accumulate elements

sorted() natural ordering + collect() accumulate elements

sorted() custom comparator + collect() accumulate elements

sorted() multi-field compare + collect() accumulate elements

peek() logging + collect() accumulate elements

peek() debugging + collect() accumulate elements

peek() metrics + collect() accumulate elements

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, self-contained Java class examples** for **every requested Stream combination**, each focusing on **one intermediate operation + collect()**.

```
import java.util.*;

public class FilterBooleanCheckExample {
   public static void main(String[] args) {

      List < Integer > numbers = Arrays.asList(10, 15, 20, 25, 30);
      List < Integer > evenNumbers = new ArrayList < Integer > ();

      for (Integer n: numbers) {
         if (n % 2 == 0) {
            evenNumbers.add(n);
         }
      }

      System.out.println(evenNumbers);
   }
}
```

1 filter()— boolean checks + collect()

```
import java.util.*;
import java.util.stream.*;

public class FilterBooleanCheckExample {
   public static void main(String[] args) {
      List < Integer > numbers = List.of(10, 15, 20, 25, 30);

      List < Integer > evenNumbers = numbers.stream()
         .filter(n - > n % 2 == 0)                    // boolean check
         .collect(Collectors.toList());

      System.out.println(evenNumbers);
   }
}
```

```
import java.util.*;

public class FilterNullCheckExample {
   public static void main(String[] args) {

      List < String > names = Arrays.asList("Anurag", null, "Bharti", null);
      List < String > nonNullNames = new ArrayList < String > ();

      for (String name: names) {
         if (name != null) {
            nonNullNames.add(name);
         }
      }

      System.out.println(nonNullNames);
   }
}
```

2 filter()— null checks + collect()

```
import java.util.*;
import java.util.stream.*;

public class FilterNullCheckExample {
   public static void main(String[] args) {
      List < String > names = Arrays.asList("Anurag", null, "Bharti", null);

      List < String > nonNullNames = names.stream()
         .filter(Objects::nonNull)
         .collect(Collectors.toList());

      System.out.println(nonNullNames);
   }
}
```

```
import java.util.*;

public class FilterComparisonExample {
   public static void main(String[] args) {

      List < Integer > salaries = Arrays.asList(20000, 35000, 50000);
      List < Integer > highSalaries = new ArrayList < Integer > ();

      for (Integer salary: salaries) {
         if (salary > 30000) {
            highSalaries.add(salary);
         }
      }

      System.out.println(highSalaries);
   }
}
```

3 filter()— comparisons + collect()

```
import java.util.*;
import java.util.stream.*;

public class FilterComparisonExample {
   public static void main(String[] args) {
      List < Integer > salaries = List.of(20000, 35000, 50000);

      List < Integer > highSalaries = salaries.stream()
         .filter(s - > s > 30000)
         .collect(Collectors.toList());

      System.out.println(highSalaries);
   }
}
```

```
import java.util.*;

public class FilterMethodCallExample {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {

        List < User > users = new ArrayList < User > ();
        users.add(new User("Anurag", true));
        users.add(new User("Ravi", false));

        List < User > activeUsers = new ArrayList < User > ();

        for (User user: users) {
            if (user.isActive()) {
                activeUsers.add(user);
            }
        }

        System.out.println(activeUsers.size());
    }
}
```

4 filter()— method calls returning boolean + collect()

```
import java.util.*;
import java.util.stream.*;

public class FilterMethodCallExample {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {
        List < User > users = List.of(
            new User("Anurag", true),
            new User("Ravi", false)
        );

        List < User > activeUsers = users.stream()
            .filter(User::isActive)
            .collect(Collectors.toList());

        System.out.println(activeUsers.size());
    }
}
```

```
import java.util.*;

public class DistinctWithEqualsExample {

   static class Person {
      String name;

      Person(String name) {
         this.name = name;
      }

      public boolean equals(Object o) {
         if (o instanceof Person) {
            Person p = (Person) o;
            return name.equals(p.name);
         }
         return false;
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List < Person > people = new ArrayList < Person > ();
      people.add(new Person("Anurag"));
      people.add(new Person("Anurag"));
      people.add(new Person("Ravi"));

      Set < Person > set = new LinkedHashSet < Person > (people);
      List < Person > unique = new ArrayList < Person > (set);

      System.out.println(unique.size());
   }
}
```

5 distinct()— rely on equals() / hashCode() + collect()

```
import java.util.*;
import java.util.stream.*;

public class DistinctWithEqualsExample {

    static class Person {
        String name;

        Person(String name) {
            this.name = name;
        }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List < Person > people = List.of(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
        );

        List < Person > unique = people.stream()
            .distinct()
            .collect(Collectors.toList());

        System.out.println(unique.size());
    }
}
```

```
import java.util.*;

public class DistinctImmutableExample {
   public static void main(String[] args) {

      List < String > values = Arrays.asList("A", "B", "A", "C");

      Set < String > set = new LinkedHashSet < String > (values);
      List < String > unique = new ArrayList < String > (set);

      System.out.println(unique);
   }
}
```

6 distinct()— immutable objects + collect()

```
import java.util.*;
import java.util.stream.*;

public class DistinctImmutableExample {
   public static void main(String[] args) {
      List < String > values = List.of("A", "B", "A", "C");

      List < String > unique = values.stream()
         .distinct()
         .collect(Collectors.toList());

      System.out.println(unique);
   }
}
```

```
import java.util.*;

public class MapTransformExample {
   public static void main(String[] args) {

      List < String > names = Arrays.asList("anurag", "bharti");
      List < String > upper = new ArrayList < String > ();

      for (String name: names) {
         upper.add(name.toUpperCase());
      }

      System.out.println(upper);
   }
}
```

7 map()— transform objects + collect()

```
import java.util.*;
import java.util.stream.*;

public class MapTransformExample {
   public static void main(String[] args) {
      List < String > names = List.of("anurag", "bharti");

      List < String > upper = names.stream()
         .map(String::toUpperCase)
         .collect(Collectors.toList());

      System.out.println(upper);
   }
}
```

```
import java.util.*;

public class MapCreateObjectExample {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {

      List < String > names = Arrays.asList("Anurag", "Ravi");
      List < UserDTO > dtos = new ArrayList < UserDTO > ();

      for (String name: names) {
         dtos.add(new UserDTO(name));
      }

      System.out.println(dtos.size());
   }
}
```

8 map()— create new objects + collect()

```
import java.util.*;
import java.util.stream.*;

public class MapCreateObjectExample {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {
      List < String > names = List.of("Anurag", "Ravi");

      List < UserDTO > dtos = names.stream()
         .map(UserDTO::new)
         .collect(Collectors.toList());

      System.out.println(dtos.size());
   }
}
```

```
import java.util.*;

public class MapPureFunctionExample {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List < Integer > numbers = Arrays.asList(2, 3, 4);
      List < Integer > squares = new ArrayList < Integer > ();

      for (Integer n: numbers) {
         squares.add(square(n));
      }

      System.out.println(squares);
   }
}
```

9 map()— call pure functions + collect()

```
import java.util.*;
import java.util.stream.*;

public class MapPureFunctionExample {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List < Integer > numbers = List.of(2, 3, 4);

      List < Integer > squares = numbers.stream()
         .map(MapPureFunctionExample::square)
         .collect(Collectors.toList());

      System.out.println(squares);
   }
}
```

```
import java.util.*;

public class FlatMapStreamExample {
   public static void main(String[] args) {

      List < List < Integer >> lists = new ArrayList < List < Integer >> ();
      lists.add(Arrays.asList(1, 2));
      lists.add(Arrays.asList(3, 4));

      List < Integer > result = new ArrayList < Integer > ();

      for (List < Integer > inner: lists) {
         result.addAll(inner);
      }

      System.out.println(result);
   }
}
```

10 flatMap() — return Stream + collect()

```
import java.util.*;
import java.util.stream.*;

public class FlatMapStreamExample {
   public static void main(String[] args) {
      List < List < Integer >> lists = List.of(
         List.of(1, 2),
         List.of(3, 4)
      );

      List < Integer > result = lists.stream()
         .flatMap(List::stream)
         .collect(Collectors.toList());

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class FlatMapSplitExample {
   public static void main(String[] args) {

      List < String > sentences = Arrays.asList("Java Stream", "Flat Map");
      List < String > words = new ArrayList < String > ();

      for (String s: sentences) {
         String[] split = s.split(" ");
         for (String word: split) {
            words.add(word);
         }
      }

      System.out.println(words);
   }
}
```

11 flatMap()— split data + collect()

```
import java.util.*;
import java.util.stream.*;

public class FlatMapSplitExample {
   public static void main(String[] args) {
      List < String > sentences = List.of("Java Stream", "Flat Map");

      List < String > words = sentences.stream()
         .flatMap(s - > Arrays.stream(s.split(" ")))
         .collect(Collectors.toList());

      System.out.println(words);
   }
}
```

```
import java.util.*;

public class FlatMapExpandExample {
   public static void main(String[] args) {

      Map < String, List < String >> data = new HashMap < String, List < String >> ();
      data.put("A", Arrays.asList("1", "2"));
      data.put("B", Arrays.asList("3"));

      List < String > values = new ArrayList < String > ();

      for (List < String > list: data.values()) {
         values.addAll(list);
      }

      System.out.println(values);
   }
}
```

12 flatMap()— expand collections + collect()

```
import java.util.*;
import java.util.stream.*;

public class FlatMapExpandExample {
   public static void main(String[] args) {
      Map < String, List < String >> data = Map.of(
         "A", List.of("1", "2"),
         "B", List.of("3")
      );

      List < String > values = data.values().stream()
         .flatMap(Collection::stream)
         .collect(Collectors.toList());
      System.out.println(values);
   }
}
```

```
import java.util.*;

public class SortedNaturalExample {
   public static void main(String[] args) {

      List < Integer > numbers = new ArrayList < Integer > (
         Arrays.asList(5, 1, 3));

      Collections.sort(numbers);

      System.out.println(numbers);
   }
}
```

13 sorted()— natural ordering + collect()

```
import java.util.*;
import java.util.stream.*;

public class SortedNaturalExample {
   public static void main(String[] args) {
      List < Integer > numbers = List.of(5, 1, 3);

      List < Integer > sorted = numbers.stream()
         .sorted()
         .collect(Collectors.toList());

      System.out.println(sorted);
   }
```

```
}
```

```
import java.util.*;

public class SortedCustomComparatorExample {
   public static void main(String[] args) {

      List < String > names = new ArrayList < String > (
         Arrays.asList("Anurag", "Ravi", "Alok"));

      Collections.sort(names, Collections.reverseOrder());

      System.out.println(names);
   }
}
```

14 sorted()— custom comparator + collect()

```
import java.util.*;
import java.util.stream.*;

public class SortedCustomComparatorExample {
   public static void main(String[] args) {
      List < String > names = List.of("Anurag", "Ravi", "Alok");

      List < String > sorted = names.stream()
         .sorted(Comparator.reverseOrder())
         .collect(Collectors.toList());

      System.out.println(sorted);
   }
}
```

```
import java.util.*;

public class SortedCustomComparatorExample {
   public static void main(String[] args) {

      List < String > names = new ArrayList < String > (
         Arrays.asList("Anurag", "Ravi", "Alok"));

      Collections.sort(names, Collections.reverseOrder());

      System.out.println(names);
   }
}
```

14 sorted()— custom comparator + collect()

```
import java.util.*;
import java.util.stream.*;

public class SortedCustomComparatorExample {
   public static void main(String[] args) {
      List < String > names = List.of("Anurag", "Ravi", "Alok");

      List < String > sorted = names.stream()
         .sorted(Comparator.reverseOrder())
         .collect(Collectors.toList());

      System.out.println(sorted);
   }
}
```

15 sorted()— multi - field compare + collect()

```
import java.util.*;
import java.util.stream.*;

public class SortedMultiFieldExample {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {
        List < Employee > employees = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
        );

        List < Employee > sorted = employees.stream()
            .sorted(Comparator
                .comparing((Employee e) - > e.name)
                .thenComparingInt(e - > e.age))
            .collect(Collectors.toList());

        System.out.println(sorted.size());
    }
}
```

```
import java.util.*;

public class PeekLoggingExample {
   public static void main(String[] args) {

      List < Integer > numbers = Arrays.asList(1, 2, 3);
      List < Integer > result = new ArrayList < Integer > ();

      for (Integer n: numbers) {
         System.out.println("Processing: " + n);
         result.add(n);
      }

      System.out.println(result);
   }
}
```

16 peek()— logging + collect()

```
import java.util.*;
import java.util.stream.*;

public class PeekLoggingExample {
   public static void main(String[] args) {
      List < Integer > result = List.of(1, 2, 3).stream()
         .peek(n - > System.out.println("Processing: " + n))
         .collect(Collectors.toList());

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class PeekDebugExample {
   public static void main(String[] args) {

      List < Integer > numbers = Arrays.asList(10, 20, 30);
      List < Integer > result = new ArrayList < Integer > ();

      for (Integer n: numbers) {
         if (n == 20) {
            System.out.println("Breakpoint value: " + n);
         }
         result.add(n);
      }

      System.out.println(result);
   }
}
```

17 peek()— debugging + collect()

```
import java.util.*;
import java.util.stream.*;

public class PeekDebugExample {
   public static void main(String[] args) {
      List < Integer > result = List.of(10, 20, 30).stream()
         .peek(n - > {
            if (n == 20) System.out.println("Breakpoint value: " + n);
         })
         .collect(Collectors.toList());

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class PeekMetricsExample {
   public static void main(String[] args) {

      List < Integer > numbers = Arrays.asList(5, 10, 15);
      List < Integer > result = new ArrayList < Integer > ();

      int counter = 0;

      for (Integer n: numbers) {
         counter++;
         result.add(n);
      }

      System.out.println("Processed count: " + counter);
   }
}
```

18 peek()— metrics + collect()

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.*;

public class PeekMetricsExample {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      List < Integer > result = List.of(5, 10, 15).stream()
         .peek(n - > counter.incrementAndGet())
         .collect(Collectors.toList());

      System.out.println("Processed count: " + counter.get());
   }
}
```

---

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + collect() transform & group

filter() null checks + collect() transform & group

filter() comparisons + collect() transform & group

filter() method calls returning boolean + collect() transform & group

distinct() rely on equals() / hashcode() + collect() transform & group

distinct() immutable objects + collect() transform & group

map() transform objects + collect() transform & group

map() create new objects + collect() transform & group

map() call pure functions + collect() transform & group

flatmap() return stream + collect() transform & group

flatmap() split data + collect() transform & group

flatmap() expand collections + collect() transform & group

sorted() natural ordering + collect() transform & group

sorted() custom comparator + collect() transform & group

sorted() multi-field compare + collect() transform & group

peek() logging + collect() transform & group

peek() debugging + collect() transform & group

peek() metrics + collect() transform & group

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Perfect — this time **every example explicitly does BOTH**:

1. **Transform** (via map, mapping, or downstream collectors)
2. **Group** (via Collectors.groupingBy(...))

Each item is a **complete, standalone Java class**, Java 8+, clean and interview-ready.

```
import java.util.*;

public class FilterBooleanTransformGroup_Java15 {
   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(10, 15, 20, 25);
      Map<String, List<Integer>> result = new HashMap<String, List<Integer>>();

      for (Integer n : numbers) {

         if (n % 2 == 0) {                                    // filter
            int transformed = n * 2;                          // map

            String key = transformed > 30 ? "HIGH" : "LOW";

            if (!result.containsKey(key)) {
               result.put(key, new ArrayList<Integer>());
            }
            result.get(key).add(transformed);
         }
      }

      System.out.println(result);
   }
}
```

1 filter() boolean checks + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FilterBooleanTransformGroup {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      Map<String, List<Integer>> result =
            numbers.stream()
                  .filter(n -> n % 2 == 0)                    // boolean check
                  .map(n -> n * 2)                            // transform
                  .collect(Collectors.groupingBy(
                        n -> n > 30 ? "HIGH" : "LOW"
                  ));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class FilterNullTransformGroup_Java15 {
   public static void main(String[] args) {

      List<String> names = Arrays.asList("Anurag", null, "Amit", null, "Bharti");
      Map<Integer, List<String>> result = new HashMap<Integer, List<String>>();

      for (String name : names) {

         if (name != null) {                                  // filter null
            String upper = name.toUpperCase();                // map
            Integer key = upper.length();

            if (!result.containsKey(key)) {
               result.put(key, new ArrayList<String>());
            }
            result.get(key).add(upper);
         }
      }

      System.out.println(result);
```

```
    }
}
```

2 filter() null checks + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FilterNullTransformGroup {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Anurag", null, "Amit", null, "Bharti");

        Map<Integer, List<String>> result =
                names.stream()
                        .filter(Objects::nonNull)                    // null check
                        .map(String::toUpperCase)                    // transform
                        .collect(Collectors.groupingBy(String::length));

        System.out.println(result);
    }
}
```

```
import java.util.*;

public class FilterComparisonTransformGroup_Java15 {
    public static void main(String[] args) {

        List<Integer> salaries = Arrays.asList(20000, 40000, 60000);
        Map<String, List<Integer>> result = new HashMap<String, List<Integer>>();

        for (Integer s : salaries) {

            if (s >= 30000) {
                int transformed = s + 5000;

                String key = transformed >= 50000 ? "SENIOR" : "MID";

                if (!result.containsKey(key)) {
                    result.put(key, new ArrayList<Integer>());
                }
                result.get(key).add(transformed);
            }
        }

        System.out.println(result);
    }
}
```

3 filter() comparisons + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FilterComparisonTransformGroup {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      Map<String, List<Integer>> result =
            salaries.stream()
                  .filter(s -> s >= 30000)                    // comparison
                  .map(s -> s + 5000)                         // transform
                  .collect(Collectors.groupingBy(
                        s -> s >= 50000 ? "SENIOR" : "MID"
                  ));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class FilterMethodTransformGroup_Java15 {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {

        List<User> users = Arrays.asList(
                new User("Anurag", true),
                new User("Ravi", false),
                new User("Amit", true)
        );

        Map<Boolean, List<String>> result = new HashMap<Boolean, List<String>>();

        for (User u : users) {

            if (u.isActive()) {                                  // filter
                String upper = u.name.toUpperCase();             // map
                Boolean key = upper.startsWith("A");

                if (!result.containsKey(key)) {
                    result.put(key, new ArrayList<String>());
                }
                result.get(key).add(upper);
            }
        }

        System.out.println(result);
    }
}
```

4 filter() method call boolean + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FilterMethodTransformGroup {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {
        List<User> users = List.of(
                new User("Anurag", true),
                new User("Ravi", false),
                new User("Amit", true)
        );

        Map<Boolean, List<String>> result =
                users.stream()
                        .filter(User::isActive)                    // method call
                        .map(u -> u.name.toUpperCase())            // transform
                        .collect(Collectors.groupingBy(
                                name -> name.startsWith("A")
                        ));

        System.out.println(result);
    }
}
```

```
import java.util.*;

public class DistinctEqualsTransformGroup_Java15 {

   static class Person {
      String name;

      Person(String name) { this.name = name; }

      public boolean equals(Object o) {
         if (this == o) return true;
         if (!(o instanceof Person)) return false;
         Person p = (Person) o;
         return name.equals(p.name);
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> people = Arrays.asList(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
      );

      Set<Person> unique = new HashSet<Person>(people);
      Map<Integer, List<String>> result = new HashMap<Integer, List<String>>();

      for (Person p : unique) {

         String name = p.name;
         Integer key = name.length();

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<String>());
         }
         result.get(key).add(name);
      }

      System.out.println(result);
   }
```

```
}
```

5 distinct() equals/hashCode + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class DistinctEqualsTransformGroup {

    static class Person {
        String name;

        Person(String name) { this.name = name; }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        Map<Integer, List<String>> result =
                people.stream()
                        .distinct()
                        .map(p -> p.name)                          // transform
                        .collect(Collectors.groupingBy(String::length));

        System.out.println(result);
    }
}
```

```
import java.util.*;

public class DistinctImmutableTransformGroup_Java15 {
   public static void main(String[] args) {

      List<String> values = Arrays.asList("Java", "Java", "Python", "Go");

      Set<String> unique = new HashSet<String>(values);
      Map<Character, List<String>> result = new HashMap<Character, List<String>>();

      for (String s : unique) {

         String upper = s.toUpperCase();
         Character key = upper.charAt(0);

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<String>());
         }
         result.get(key).add(upper);
      }

      System.out.println(result);
   }
}
```

6 distinct() immutable objects + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class DistinctImmutableTransformGroup {
   public static void main(String[] args) {
      List<String> values = List.of("Java", "Java", "Python", "Go");

      Map<Character, List<String>> result =
            values.stream()
                  .distinct()
                  .map(String::toUpperCase)                    // transform
                  .collect(Collectors.groupingBy(s -> s.charAt(0)));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class MapTransformGroup_Java15 {
   public static void main(String[] args) {

      List<String> names = Arrays.asList("anurag", "amit", "ravi");
      Map<Integer, List<String>> result = new HashMap<Integer, List<String>>();

      for (String name : names) {

         String upper = name.toUpperCase();
         Integer key = upper.length();

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<String>());
         }
         result.get(key).add(upper);
      }

      System.out.println(result);
   }
}
```

7 map() transform objects + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class MapTransformGroup {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit", "ravi");

      Map<Integer, List<String>> result =
            names.stream()
                  .map(String::toUpperCase)                    // transform
                  .collect(Collectors.groupingBy(String::length));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class MapNewObjectTransformGroup_Java15 {

    static class UserDTO {
        String name;

        UserDTO(String name) {
            this.name = name;
        }
    }

    public static void main(String[] args) {

        List<String> names = Arrays.asList("Anurag", "Amit", "Ravi");
        Map<Character, List<UserDTO>> result =
                new HashMap<Character, List<UserDTO>>();

        for (String name : names) {

            UserDTO dto = new UserDTO(name);                    // map → new object
            Character key = dto.name.charAt(0);

            if (!result.containsKey(key)) {
                result.put(key, new ArrayList<UserDTO>());
            }
            result.get(key).add(dto);
        }

        System.out.println(result);
    }
}
```

8 map() create new objects + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class MapNewObjectTransformGroup {

    static class UserDTO {
        String name;

        UserDTO(String name) {
            this.name = name;
        }
    }

    public static void main(String[] args) {
        List<String> names = List.of("Anurag", "Amit", "Ravi");

        Map<Character, List<UserDTO>> result =
                names.stream()
                        .map(UserDTO::new)                          // new object
                        .collect(Collectors.groupingBy(
                                dto -> dto.name.charAt(0)
                        ));

        System.out.println(result);
    }
}
```

```
import java.util.*;

public class MapPureFunctionTransformGroup_Java15 {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(2, 3, 4);
      Map<Boolean, List<Integer>> result =
            new HashMap<Boolean, List<Integer>>();

      for (Integer n : numbers) {

         int squared = square(n);                                    // map
         Boolean key = squared > 10;

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<Integer>());
         }
         result.get(key).add(squared);
      }

      System.out.println(result);
   }
}
```

9 map() pure functions + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class MapPureFunctionTransformGroup {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = List.of(2, 3, 4);

      Map<Boolean, List<Integer>> result =
            numbers.stream()
                  .map(MapPureFunctionTransformGroup::square)
                  .collect(Collectors.groupingBy(
                        n -> n > 10
                  ));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class FlatMapStreamTransformGroup_Java15 {
   public static void main(String[] args) {

      List<List<Integer>> lists = new ArrayList<List<Integer>>();
      lists.add(Arrays.asList(1, 2));
      lists.add(Arrays.asList(3, 4));

      Map<Boolean, List<Integer>> result =
            new HashMap<Boolean, List<Integer>>();

      for (List<Integer> inner : lists) {                    // flatMap → nested loop
         for (Integer n : inner) {

            int transformed = n * 10;                        // map
            Boolean key = transformed > 20;

            if (!result.containsKey(key)) {
               result.put(key, new ArrayList<Integer>());
            }
            result.get(key).add(transformed);
         }
      }
      System.out.println(result);
   }
}
```

10 flatMap() return Stream + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapStreamTransformGroup {
   public static void main(String[] args) {
      List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      );

      Map<Boolean, List<Integer>> result =
            lists.stream()
                  .flatMap(List::stream)
                  .map(n -> n * 10)                          // transform
                  .collect(Collectors.groupingBy(n -> n > 20));
```

```
        System.out.println(result);
    }
}
```

```
import java.util.*;

public class FlatMapSplitTransformGroup_Java15 {
    public static void main(String[] args) {

        List<String> sentences =
                Arrays.asList("Java Stream", "Flat Map");

        Map<Integer, List<String>> result =
                new HashMap<Integer, List<String>>();

        for (String sentence : sentences) {

            String[] words = sentence.split(" ");                    // split

            for (String word : words) {                              // flatMap
                String lower = word.toLowerCase();
                Integer key = lower.length();

                if (!result.containsKey(key)) {
                    result.put(key, new ArrayList<String>());
                }
                result.get(key).add(lower);
            }
        }

        System.out.println(result);
    }
}
```

11 flatMap() split data + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapSplitTransformGroup {
    public static void main(String[] args) {
        List<String> sentences = List.of("Java Stream", "Flat Map");

        Map<Integer, List<String>> result =
                sentences.stream()
                        .flatMap(s -> Arrays.stream(s.split(" ")))
                        .map(String::toLowerCase)                    // transform
                        .collect(Collectors.groupingBy(String::length));
```

```
        System.out.println(result);
    }
}
```

```
import java.util.*;

public class FlatMapExpandTransformGroup_Java15 {
    public static void main(String[] args) {

        Map<String, List<String>> data =
                new HashMap<String, List<String>>();

        data.put("A", Arrays.asList("x", "y"));
        data.put("B", Arrays.asList("z"));

        Map<Integer, List<String>> result =
                new HashMap<Integer, List<String>>();

        for (List<String> values : data.values()) {

            for (String value : values) {                          // flatMap
                String upper = value.toUpperCase();
                Integer key = upper.length();

                if (!result.containsKey(key)) {
                    result.put(key, new ArrayList<String>());
                }
                result.get(key).add(upper);
            }
        }

        System.out.println(result);
    }
}
```

12 flatMap() expand collections + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapExpandTransformGroup {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("x", "y"),
            "B", List.of("z")
      );

      Map<Integer, List<String>> result =
            data.values().stream()
                  .flatMap(Collection::stream)
                  .map(String::toUpperCase)                    // transform
                  .collect(Collectors.groupingBy(String::length));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class SortedNaturalTransformGroup_Java15 {
   public static void main(String[] args) {

      List<Integer> numbers =
            new ArrayList<Integer>(Arrays.asList(5, 1, 3));

      Collections.sort(numbers);                               // natural sort

      Map<Boolean, List<Integer>> result =
            new HashMap<Boolean, List<Integer>>();

      for (Integer n : numbers) {

         int transformed = n * 2;
         Boolean key = transformed > 5;

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<Integer>());
         }
         result.get(key).add(transformed);
```

```
        }

        System.out.println(result);
    }
}
```

13 sorted() natural order + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class SortedNaturalTransformGroup {
    public static void main(String[] args) {
        List<Integer> numbers = List.of(5, 1, 3);

        Map<Boolean, List<Integer>> result =
            numbers.stream()
                .sorted()
                .map(n -> n * 2)                    // transform
                .collect(Collectors.groupingBy(n -> n > 5));

        System.out.println(result);
    }
}
```

```
import java.util.*;

public class SortedCustomTransformGroup_Java15 {
   public static void main(String[] args) {

      List<String> names =
            new ArrayList<String>(Arrays.asList("Anurag", "Ravi", "Amit"));

      Collections.sort(names, new Comparator<String>() {
         public int compare(String a, String b) {
            return b.compareTo(a);                                    // reverse order
         }
      });

      Map<Character, List<String>> result =
            new HashMap<Character, List<String>>();

      for (String name : names) {

         String upper = name.toUpperCase();
         Character key = upper.charAt(0);

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<String>());
         }
         result.get(key).add(upper);
      }
      System.out.println(result);
   }
}
```

14 sorted() custom comparator + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class SortedCustomTransformGroup {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      Map<Character, List<String>> result =
            names.stream()
                  .sorted(Comparator.reverseOrder())
                  .map(String::toUpperCase)                           // transform
                  .collect(Collectors.groupingBy(s -> s.charAt(0)));
```

```
        System.out.println(result);
    }
}
```

```
import java.util.*;

public class SortedMultiFieldTransformGroup_Java15 {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {

        List<Employee> employees =
                new ArrayList<Employee>(Arrays.asList(
                        new Employee("Anurag", 30),
                        new Employee("Anurag", 25),
                        new Employee("Ravi", 35)
                ));

        Collections.sort(employees, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0)
                    return nameCompare;
                return e1.age - e2.age;
            }
        });

        Map<String, List<Integer>> result =
                new HashMap<String, List<Integer>>();

        for (Employee e : employees) {

            Integer age = e.age;
            String key = age >= 30 ? "SENIOR" : "JUNIOR";

            if (!result.containsKey(key)) {
                result.put(key, new ArrayList<Integer>());
            }
            result.get(key).add(age);
```

```
        }
        System.out.println(result);
    }
}
```

15 sorted() multi-field + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class SortedMultiFieldTransformGroup {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {
        List<Employee> employees = List.of(
                new Employee("Anurag", 30),
                new Employee("Anurag", 25),
                new Employee("Ravi", 35)
        );

        Map<String, List<Integer>> result =
                employees.stream()
                        .sorted(Comparator
                                .comparing((Employee e) -> e.name)
                                .thenComparingInt(e -> e.age))
                        .map(e -> e.age)                                // transform
                        .collect(Collectors.groupingBy(
                                age -> age >= 30 ? "SENIOR" : "JUNIOR"
                        ));

        System.out.println(result);
    }
}
```

```
import java.util.*;

public class PeekLoggingTransformGroup_Java15 {
   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(1, 2, 3);
      Map<Boolean, List<Integer>> result =
            new HashMap<Boolean, List<Integer>>();

      for (Integer n : numbers) {

         System.out.println("Log: " + n);                    // peek equivalent

         int transformed = n * 10;
         Boolean key = transformed > 20;

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<Integer>());
         }
         result.get(key).add(transformed);
      }

      System.out.println(result);
   }
}
```

16 peek() logging + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class PeekLoggingTransformGroup {
   public static void main(String[] args) {
      Map<Boolean, List<Integer>> result =
            List.of(1, 2, 3).stream()
                  .peek(n -> System.out.println("Log: " + n))
                  .map(n -> n * 10)                          // transform
                  .collect(Collectors.groupingBy(n -> n > 20));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class PeekDebugTransformGroup_Java15 {
   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(10, 20, 30);
      Map<Boolean, List<Integer>> result =
            new HashMap<Boolean, List<Integer>>();

      for (Integer n : numbers) {

         if (n == 20) {
            System.out.println("Debug hit: " + n);
         }

         int transformed = n + 1;
         Boolean key = transformed > 20;

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<Integer>());
         }
         result.get(key).add(transformed);
      }

      System.out.println(result);
   }
}
```

17 peek() debugging + collect() (Transform & Group)

```
import java.util.*;
import java.util.stream.*;

public class PeekDebugTransformGroup {
   public static void main(String[] args) {
      Map<Boolean, List<Integer>> result =
            List.of(10, 20, 30).stream()
                  .peek(n -> {
                     if (n == 20) System.out.println("Debug hit: " + n);
                  })
                  .map(n -> n + 1)                          // transform
                  .collect(Collectors.groupingBy(n -> n > 20));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class PeekMetricsTransformGroup_Java15 {
   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(5, 10, 15);
      Map<Boolean, List<Integer>> result =
            new HashMap<Boolean, List<Integer>>();

      int counter = 0;                          // AtomicInteger not needed in Java 1.5 single-thread

      for (Integer n : numbers) {

         counter++;                                                    // peek metric

         int transformed = n * 2;
         Boolean key = transformed > 20;

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<Integer>());
         }
         result.get(key).add(transformed);
      }
      System.out.println("Processed: " + counter);
      System.out.println(result);
   }
}
```

18 peek() metrics + collect() (Transform & Group)

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.*;

public class PeekMetricsTransformGroup {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      Map<Boolean, List<Integer>> result =
            List.of(5, 10, 15).stream()
                  .peek(n -> counter.incrementAndGet())
                  .map(n -> n * 2)                                     // transform
                  .collect(Collectors.groupingBy(n -> n > 20));

      System.out.println("Processed: " + counter.get());
```

```
        System.out.println(result);
    }
}
```

---

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + collect() map → reduce → store

filter() null checks + collect() map → reduce → store

filter() comparisons + collect() map → reduce → store

filter() method calls returning boolean + collect() map → reduce → store

distinct() rely on equals() / hashcode() + collect() map → reduce → store

distinct() immutable objects + collect() map → reduce → store

map() transform objects + collect() map → reduce → store

map() create new objects + collect() map → reduce → store

map() call pure functions + collect() map → reduce → store

flatmap() return stream + collect() map → reduce → store

flatmap() split data + collect() map → reduce → store

flatmap() expand collections + collect() map → reduce → store

sorted() natural ordering + collect() map → reduce → store

sorted() custom comparator + collect() map → reduce → store

sorted() multi-field compare + collect() map → reduce → store

peek() logging + collect() map → reduce → store

peek() debugging + collect() map → reduce → store

peek() metrics + collect() map → reduce → store

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** where **each pipeline explicitly follows**:

Map → Reduce → Store

- **Map** = transform data
- **Reduce** = aggregate (sum / count / max / min / join / etc.)
- **Store** = collect() into a Map or List

All examples:

- Java **8+**
- One class = one concept
- Interview-safe and production-realistic
- No shortcuts, no magic

```
import java.util.*;

public class FilterBooleanMapReduceStore {
   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(10, 15, 20, 25);
      Map<String, Integer> result = new HashMap<String, Integer>();

      for (Integer n : numbers) {

         if (n % 2 == 0) {                                    // FILTER

            int mapped = n * 2;                               // MAP

            String key = (mapped > 30) ? "HIGH" : "LOW";

            Integer sum = result.get(key);
            if (sum == null) sum = 0;

            result.put(key, sum + mapped);                    // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

1 filter() boolean checks + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FilterBooleanMapReduceStore {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      Map<String, Integer> result =
            numbers.stream()
                  .filter(n -> n % 2 == 0)                          // filter (boolean)
                  .map(n -> n * 2)                                  // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 30 ? "HIGH" : "LOW",
                        Collectors.summingInt(Integer::intValue) // REDUCE
                  ));                                               // STORE

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class FilterNullMapReduceStore {
   public static void main(String[] args) {

      List<String> names = Arrays.asList("Anurag", null, "Amit", null);
      Map<Integer, Long> result = new HashMap<Integer, Long>();

      for (String name : names) {

         if (name != null) {                                        // FILTER

            String mapped = name.toUpperCase();                     // MAP
            Integer key = mapped.length();

            Long count = result.get(key);
            if (count == null) count = 0L;

            result.put(key, count + 1);                             // REDUCE + STORE
         }
```

```
        }

        System.out.println(result);
    }
}
```

2 filter() null checks + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FilterNullMapReduceStore {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Anurag", null, "Amit", null);

        Map<Integer, Long> result =
            names.stream()
                .filter(Objects::nonNull)                    // null check
                .map(String::toUpperCase)                    // MAP
                .collect(Collectors.groupingBy(
                    String::length,
                    Collectors.counting()                    // REDUCE
                ));                                          // STORE

        System.out.println(result);
    }
}
```

```
import java.util.*;

public class FilterComparisonMapReduceStore {
    public static void main(String[] args) {

        List<Integer> salaries = Arrays.asList(20000, 40000, 60000);
        Map<String, Integer> result = new HashMap<String, Integer>();

        for (Integer s : salaries) {

            if (s >= 30000) {                                // FILTER

                int mapped = s + 5000;                       // MAP
                String key = (mapped >= 50000) ? "SENIOR" : "MID";

                Integer sum = result.get(key);
                if (sum == null) sum = 0;

                result.put(key, sum + mapped);               // REDUCE + STORE
            }
        }
```

```
        System.out.println(result);
    }
}
```

3 filter() comparisons + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FilterComparisonMapReduceStore {
    public static void main(String[] args) {
        List<Integer> salaries = List.of(20000, 40000, 60000);

        Map<String, Integer> result =
            salaries.stream()
                .filter(s -> s >= 30000)                    // comparison
                .map(s -> s + 5000)                         // MAP
                .collect(Collectors.groupingBy(
                    s -> s >= 50000 ? "SENIOR" : "MID",
                    Collectors.summingInt(Integer::intValue) // REDUCE
                ));

        System.out.println(result);
    }
}
```

```
import java.util.*;

public class FilterMethodMapReduceStore {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {

      List<User> users = Arrays.asList(
            new User("Anurag", true),
            new User("Amit", true),
            new User("Ravi", false)
      );

      Map<Character, Long> result = new HashMap<Character, Long>();

      for (User u : users) {

         if (u.isActive()) {                                    // FILTER

            String mapped = u.name;                             // MAP
            Character key = mapped.charAt(0);

            Long count = result.get(key);
            if (count == null) count = 0L;

            result.put(key, count + 1);                         // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
```

```
}
```

4 filter() method call boolean + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FilterMethodMapReduceStore {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {
        List<User> users = List.of(
                new User("Anurag", true),
                new User("Amit", true),
                new User("Ravi", false)
        );

        Map<Character, Long> result =
                users.stream()
                        .filter(User::isActive)                    // method call
                        .map(u -> u.name)                          // MAP
                        .collect(Collectors.groupingBy(
                                name -> name.charAt(0),
                                Collectors.counting()              // REDUCE
                        ));

        System.out.println(result);
    }
}
```

```
import java.util.*;

public class DistinctEqualsMapReduceStore {

   static class Person {
      String name;

      Person(String name) { this.name = name; }

      public boolean equals(Object o) {
         if (o instanceof Person) {
            Person p = (Person) o;
            return name.equals(p.name);
         }
         return false;
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> people = Arrays.asList(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
      );

      Set<Person> distinct = new HashSet<Person>(people);

      Map<Integer, Long> result = new HashMap<Integer, Long>();

      for (Person p : distinct) {

         String mapped = p.name;                          // MAP
         Integer key = mapped.length();

         Long count = result.get(key);
         if (count == null) count = 0L;

         result.put(key, count + 1);                      // REDUCE + STORE
      }
```

```
        System.out.println(result);
    }
}
```

5 distinct() equals/hashCode + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class DistinctEqualsMapReduceStore {

    static class Person {
        String name;

        Person(String name) { this.name = name; }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        Map<Integer, Long> result =
                people.stream()
                        .distinct()
                        .map(p -> p.name)                    // MAP
                        .collect(Collectors.groupingBy(
                                String::length,
                                Collectors.counting()        // REDUCE
                        ));

        System.out.println(result);
    }
}
```

6 distinct() immutable objects + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class DistinctImmutableMapReduceStore {
   public static void main(String[] args) {
      List<String> values = List.of("Java", "Java", "Go", "Python");

      Map<Character, Long> result =
            values.stream()
                  .distinct()
                  .map(String::toUpperCase)                    // MAP
                  .collect(Collectors.groupingBy(
                        s -> s.charAt(0),
                        Collectors.counting()                  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class MapTransformMapReduceStore {
   public static void main(String[] args) {

      List<String> names = Arrays.asList("anurag", "amit", "ravi");
      Map<Integer, Long> result = new HashMap<Integer, Long>();

      for (String name : names) {

         String mapped = name.toUpperCase();                   // MAP
         Integer key = mapped.length();

         Long count = result.get(key);
         if (count == null) count = 0L;

         result.put(key, count + 1);                           // REDUCE + STORE
      }

      System.out.println(result);
   }
```

```
}
```

7 map() transform objects + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class MapTransformMapReduceStore {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit", "ravi");

      Map<Integer, Long> result =
            names.stream()
                  .map(String::toUpperCase)                    // MAP
                  .collect(Collectors.groupingBy(
                        String::length,
                        Collectors.counting()                  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class MapNewObjectMapReduceStore {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {

      List<String> names = Arrays.asList("Anurag", "Amit", "Ravi");
      Map<Character, Long> result = new HashMap<Character, Long>();

      for (String name : names) {

         UserDTO dto = new UserDTO(name);                        // MAP
         Character key = dto.name.charAt(0);

         Long count = result.get(key);
         if (count == null) count = 0L;
```

```
        result.put(key, count + 1);                    // REDUCE + STORE
    }

    System.out.println(result);
  }
}
```

8 map() create new objects + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class MapNewObjectMapReduceStore {

  static class UserDTO {
    String name;
    UserDTO(String name) { this.name = name; }
  }

  public static void main(String[] args) {
    List<String> names = List.of("Anurag", "Amit", "Ravi");

    Map<Character, Long> result =
        names.stream()
            .map(UserDTO::new)                         // MAP
            .collect(Collectors.groupingBy(
                dto -> dto.name.charAt(0),
                Collectors.counting()                  // REDUCE
            ));

    System.out.println(result);
  }
}
```

```
import java.util.*;

public class MapPureFunctionMapReduceStore {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(2, 3, 4);
      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (Integer n : numbers) {

         int mapped = square(n);                          // MAP
         Boolean key = (mapped > 10);

         Integer sum = result.get(key);
         if (sum == null) sum = 0;

         result.put(key, sum + mapped);                   // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

9 map() pure functions + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class MapPureFunctionMapReduceStore {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = List.of(2, 3, 4);

      Map<Boolean, Integer> result =
            numbers.stream()
                  .map(MapPureFunctionMapReduceStore::square)          // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 10,
                        Collectors.summingInt(Integer::intValue)       // REDUCE
                  ));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class FlatMapStreamMapReduceStore {
   public static void main(String[] args) {

      List<List<Integer>> lists = new ArrayList<List<Integer>>();
      lists.add(Arrays.asList(1, 2));
      lists.add(Arrays.asList(3, 4));

      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (List<Integer> inner : lists) {                    // FLATTEN
         for (Integer n : inner) {

            int mapped = n * 10;                             // MAP
            Boolean key = (mapped > 20);

            Integer sum = result.get(key);
            if (sum == null) sum = 0;

            result.put(key, sum + mapped);                   // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

10 flatMap() return Stream + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapStreamMapReduceStore {
   public static void main(String[] args) {
      List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      );

      Map<Boolean, Integer> result =
            lists.stream()
                  .flatMap(List::stream)
                  .map(n -> n * 10)                          // MAP
                  .collect(Collectors.groupingBy(
```

```
                    n -> n > 20,
                    Collectors.summingInt(Integer::intValue) // REDUCE
                ));
        System.out.println(result);
    }
}
```

```
import java.util.*;

public class FlatMapSplitMapReduceStore {
    public static void main(String[] args) {

        List<String> sentences = Arrays.asList("Java Stream", "Flat Map");
        Map<Integer, Long> result = new HashMap<Integer, Long>();

        for (String sentence : sentences) {

            String[] words = sentence.split(" ");                    // SPLIT

            for (int i = 0; i < words.length; i++) {

                String mapped = words[i].toLowerCase();              // MAP
                Integer key = mapped.length();

                Long count = result.get(key);
                if (count == null) count = 0L;

                result.put(key, count + 1);                          // REDUCE + STORE
            }
        }

        System.out.println(result);
    }
}
```

11 flatMap() split data + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapSplitMapReduceStore {
    public static void main(String[] args) {
        List<String> sentences = List.of("Java Stream", "Flat Map");

        Map<Integer, Long> result =
            sentences.stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
                .map(String::toLowerCase)                            // MAP
                .collect(Collectors.groupingBy(
```

```
                    String::length,
                    Collectors.counting()                    // REDUCE
                ));
        System.out.println(result);
    }
}
```

```
import java.util.*;

public class FlatMapExpandMapReduceStore {
    public static void main(String[] args) {

        Map<String, List<String>> data = new HashMap<String, List<String>>();
        data.put("A", Arrays.asList("x", "y"));
        data.put("B", Arrays.asList("z"));

        Map<Integer, Long> result = new HashMap<Integer, Long>();

        for (List<String> list : data.values()) {                // FLATTEN
            for (String value : list) {

                String mapped = value.toUpperCase();              // MAP
                Integer key = mapped.length();

                Long count = result.get(key);
                if (count == null) count = 0L;

                result.put(key, count + 1);                       // REDUCE + STORE
            }
        }

        System.out.println(result);
    }
}
```

12 flatMap() expand collections + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class FlatMapExpandMapReduceStore {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("x", "y"),
            "B", List.of("z")
      );

      Map<Integer, Long> result =
            data.values().stream()
                  .flatMap(Collection::stream)
                  .map(String::toUpperCase)                    // MAP
                  .collect(Collectors.groupingBy(
                        String::length,
                        Collectors.counting()                  // REDUCE
                  ));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class SortedNaturalMapReduceStore {
   public static void main(String[] args) {

      List<Integer> numbers = new ArrayList<Integer>(
            Arrays.asList(5, 1, 3));

      Collections.sort(numbers);                             // SORT

      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (Integer n : numbers) {

         int mapped = n * 2;                                 // MAP
         Boolean key = (mapped > 5);

         Integer sum = result.get(key);
```

```
        if (sum == null) sum = 0;

        result.put(key, sum + mapped);                    // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

13 sorted() natural order + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class SortedNaturalMapReduceStore {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(5, 1, 3);

      Map<Boolean, Integer> result =
            numbers.stream()
                  .sorted()
                  .map(n -> n * 2)                        // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 5,
                        Collectors.summingInt(Integer::intValue) // REDUCE
                  ));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class SortedCustomMapReduceStore {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>(
            Arrays.asList("Anurag", "Ravi", "Amit"));

      Collections.sort(names, Collections.reverseOrder());    // CUSTOM SORT

      Map<Character, Long> result = new HashMap<Character, Long>();

      for (String name : names) {

         String mapped = name.toUpperCase();                  // MAP
         Character key = mapped.charAt(0);

         Long count = result.get(key);
         if (count == null) count = 0L;

         result.put(key, count + 1);                          // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

14 sorted() custom comparator + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class SortedCustomMapReduceStore {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      Map<Character, Long> result =
            names.stream()
                  .sorted(Comparator.reverseOrder())
                  .map(String::toUpperCase)                   // MAP
                  .collect(Collectors.groupingBy(
                        s -> s.charAt(0),
                        Collectors.counting()                 // REDUCE
                  ));
```

```
        System.out.println(result);
    }
}
```

```
import java.util.*;

public class SortedMultiFieldMapReduceStore {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<Employee>();
        employees.add(new Employee("Anurag", 30));
        employees.add(new Employee("Anurag", 25));
        employees.add(new Employee("Ravi", 35));

        Collections.sort(employees, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {

                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0) return nameCompare;

                return e1.age - e2.age;
            }
        });

        Map<String, Integer> result = new HashMap<String, Integer>();

        for (Employee e : employees) {

            int mapped = e.age;                                    // MAP
            String key = (mapped >= 30) ? "SENIOR" : "JUNIOR";

            Integer sum = result.get(key);
            if (sum == null) sum = 0;

            result.put(key, sum + mapped);                         // REDUCE + STORE
        }
```

```
        System.out.println(result);
    }
}
```

15 sorted() multi-field + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class SortedMultiFieldMapReduceStore {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {
        List<Employee> employees = List.of(
                new Employee("Anurag", 30),
                new Employee("Anurag", 25),
                new Employee("Ravi", 35)
        );

        Map<String, Integer> result =
                employees.stream()
                        .sorted(Comparator
                                .comparing((Employee e) -> e.name)
                                .thenComparingInt(e -> e.age))
                        .map(e -> e.age)                                    // MAP
                        .collect(Collectors.groupingBy(
                                age -> age >= 30 ? "SENIOR" : "JUNIOR",
                                Collectors.summingInt(Integer::intValue) // REDUCE
                        ));

        System.out.println(result);
    }
}
```

```
import java.util.*;

public class PeekLoggingMapReduceStore {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);
      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (Integer n : list) {

         System.out.println("Log: " + n);                          // PEEK

         int mapped = n * 10;                                      // MAP
         Boolean key = (mapped > 20);

         Integer sum = result.get(key);
         if (sum == null) sum = 0;

         result.put(key, sum + mapped);                            // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

16 peek() logging + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class PeekLoggingMapReduceStore {
   public static void main(String[] args) {
      Map<Boolean, Integer> result =
            List.of(1, 2, 3).stream()
                  .peek(n -> System.out.println("Log: " + n))
                  .map(n -> n * 10)                                 // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 20,
                        Collectors.summingInt(Integer::intValue) // REDUCE
                  ));

      System.out.println(result);
   }
}
```

```
import java.util.*;

public class PeekDebugMapReduceStore {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);
      Map<Boolean, Long> result = new HashMap<Boolean, Long>();

      for (Integer n : list) {

         if (n == 20) {
            System.out.println("Debug hit: " + n);                    // PEEK
         }

         int mapped = n + 1;                                          // MAP
         Boolean key = (mapped > 20);

         Long count = result.get(key);
         if (count == null) count = 0L;

         result.put(key, count + 1);                                  // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

17 peek() debugging + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.stream.*;

public class PeekDebugMapReduceStore {
   public static void main(String[] args) {
      Map<Boolean, Long> result =
            List.of(10, 20, 30).stream()
                  .peek(n -> {
                     if (n == 20) System.out.println("Debug hit: " + n);
                  })
                  .map(n -> n + 1)                                    // MAP
                  .collect(Collectors.groupingBy(
                        n -> n > 20,
                        Collectors.counting()                         // REDUCE
                  ));
```

```
        System.out.println(result);
    }
}
```

```
import java.util.*;

public class PeekMetricsMapReduceStore {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(5, 10, 15);
        int counter = 0;

        Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

        for (Integer n : list) {

            counter++;                                          // METRICS (peek equivalent)

            int mapped = n * 2;                                 // MAP
            Boolean key = (mapped > 20);

            Integer sum = result.get(key);
            if (sum == null) sum = 0;

            result.put(key, sum + mapped);                      // REDUCE + STORE
        }
        System.out.println("Processed: " + counter);
        System.out.println(result);
    }
}
```

18 peek() metrics + collect() (Map → Reduce → Store)

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.*;

public class PeekMetricsMapReduceStore {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        Map<Boolean, Integer> result =
            List.of(5, 10, 15).stream()
                .peek(n -> counter.incrementAndGet())
                .map(n -> n * 2)                                // MAP
                .collect(Collectors.groupingBy(
                    n -> n > 20,
                    Collectors.summingInt(Integer::intValue) // REDUCE
```

```
                ));
        System.out.println("Processed: " + counter.get());
        System.out.println(result);
    }
}
```

---

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + foreach() logging

filter() null checks + foreach() logging

filter() comparisons + foreach() logging

filter() method calls returning boolean + foreach() logging

distinct() rely on equals() / hashcode() + foreach() logging

distinct() immutable objects + foreach() logging

map() transform objects + foreach() logging

map() create new objects + foreach() logging

map() call pure functions + foreach() logging

flatmap() return stream + foreach() logging

flatmap() split data + foreach() logging

flatmap() expand collections + foreach() logging

sorted() natural ordering + foreach() logging

sorted() custom comparator + foreach() logging

sorted() multi-field compare + foreach() logging

peek() logging + foreach() logging

peek() debugging + foreach() logging

peek() metrics + foreach() logging

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** where **each pipeline ends with forEach() used purely for logging**.

Key rules followed:

- **One intermediate concept per class**
- **forEach() is the terminal operation**
- Logging is explicit (System.out.println)
- Java **8+**, interview-safe
- No collectors, no shortcuts

```
import java.util.*;

public class FilterBooleanForEachLogging {
   public static void main(String[] args) {
      List<Integer> numbers = Arrays.asList(10, 15, 20, 25);

      for (Integer n : numbers) {
         if (n % 2 == 0) {
            System.out.println("Even: " + n);
         }
      }
   }
}
```

1 filter() boolean checks + forEach() Logging

```
import java.util.*;

public class FilterBooleanForEachLogging {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      numbers.stream()
            .filter(n -> n % 2 == 0)                    // boolean check
            .forEach(n -> System.out.println("Even: " + n));
   }
}
```

```
import java.util.*;

public class FilterNullForEachLogging {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      for (String name : names) {
         if (name != null) {
            System.out.println("Name: " + name);
         }
      }
   }
}
```

2 filter() null checks + forEach() Logging

```
import java.util.*;

public class FilterNullForEachLogging {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      names.stream()
            .filter(Objects::nonNull)
            .forEach(name -> System.out.println("Name: " + name));
   }
}
```

```
import java.util.*;

public class FilterComparisonForEachLogging {
   public static void main(String[] args) {
      List<Integer> salaries = Arrays.asList(20000, 40000, 60000);

      for (Integer s : salaries) {
         if (s >= 30000) {
            System.out.println("Eligible salary: " + s);
         }
      }
   }
}
```

3 filter() comparisons + forEach() Logging

```
import java.util.*;

public class FilterComparisonForEachLogging {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      salaries.stream()
            .filter(s -> s >= 30000)
            .forEach(s -> System.out.println("Eligible salary: " + s));
   }
}
```

```
import java.util.*;

public class FilterMethodForEachLogging {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {
      List<User> users = Arrays.asList(
            new User("Anurag", true),
            new User("Ravi", false)
      );

      for (User u : users) {
         if (u.isActive()) {
            System.out.println("Active user: " + u.name);
```

```
        }
      }
    }
}
```

4 filter() method call boolean + forEach() Logging

```
import java.util.*;

public class FilterMethodForEachLogging {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {
        List<User> users = List.of(
                new User("Anurag", true),
                new User("Ravi", false)
        );

        users.stream()
                .filter(User::isActive)
                .forEach(u -> System.out.println("Active user: " + u.name));
    }
}
```

```
import java.util.*;

public class DistinctEqualsForEachLogging {

    static class Person {
        String name;

        Person(String name) {
            this.name = name;
        }

        public boolean equals(Object o) {
            if (o instanceof Person) {
                Person p = (Person) o;
                return name.equals(p.name);
            }
            return false;
        }

        public int hashCode() {
            return name.hashCode();
        }
    }

    public static void main(String[] args) {
        List<Person> people = Arrays.asList(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        Set<Person> unique = new HashSet<Person>(people);

        for (Person p : unique) {
            System.out.println("Unique person: " + p.name);
        }
    }
}
```

5 distinct() equals/hashCode + forEach() Logging

```
import java.util.*;

public class DistinctEqualsForEachLogging {

    static class Person {
        String name;

        Person(String name) {
            this.name = name;
        }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        people.stream()
                .distinct()
                .forEach(p -> System.out.println("Unique person: " + p.name));
    }
}
```

```
import java.util.*;

public class DistinctImmutableForEachLogging {
   public static void main(String[] args) {
      List<String> languages = Arrays.asList("Java", "Java", "Python");

      Set<String> unique = new HashSet<String>(languages);

      for (String lang : unique) {
         System.out.println("Language: " + lang);
      }
   }
}
```

6 distinct() immutable objects + forEach() Logging

```
import java.util.*;

public class DistinctImmutableForEachLogging {
   public static void main(String[] args) {
      List<String> languages = List.of("Java", "Java", "Python");

      languages.stream()
            .distinct()
            .forEach(lang -> System.out.println("Language: " + lang));
   }
}
```

```
import java.util.*;

public class MapTransformForEachLogging {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("anurag", "amit");

      for (String name : names) {
         String upper = name.toUpperCase();
         System.out.println("Upper: " + upper);
      }
   }
}
```

7 map() transform objects + forEach() Logging

```
import java.util.*;

public class MapTransformForEachLogging {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit");

      names.stream()
            .map(String::toUpperCase)
            .forEach(name -> System.out.println("Upper: " + name));
   }
}
```

```
import java.util.*;

public class MapNewObjectForEachLogging {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", "Amit");

      for (String name : names) {
         UserDTO dto = new UserDTO(name);
         System.out.println("DTO created: " + dto.name);
      }
   }
}
```

8 map() create new objects + forEach() Logging

```
import java.util.*;

public class MapNewObjectForEachLogging {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Amit");

      names.stream()
            .map(UserDTO::new)
            .forEach(dto -> System.out.println("DTO created: " + dto.name));
   }
}
```

```
import java.util.*;

public class MapPureFunctionForEachLogging {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = Arrays.asList(2, 3, 4);

      for (Integer n : numbers) {
         int result = square(n);
         System.out.println("Square: " + result);
      }
   }
}
```

9 map() pure functions + forEach() Logging

```
import java.util.*;

public class MapPureFunctionForEachLogging {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = List.of(2, 3, 4);

      numbers.stream()
            .map(MapPureFunctionForEachLogging::square)
            .forEach(n -> System.out.println("Square: " + n));
   }
}
```

```
import java.util.*;

public class FlatMapStreamForEachLogging {
   public static void main(String[] args) {
      List<List<Integer>> lists = new ArrayList<List<Integer>>();
      lists.add(Arrays.asList(1, 2));
      lists.add(Arrays.asList(3, 4));

      for (List<Integer> inner : lists) {
         for (Integer n : inner) {
            System.out.println("Value: " + n);
         }
      }
   }
}
```

10 flatMap() return Stream + forEach() Logging

```
import java.util.*;

public class FlatMapStreamForEachLogging {
   public static void main(String[] args) {
      List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      );

      lists.stream()
            .flatMap(List::stream)
            .forEach(n -> System.out.println("Value: " + n));
   }
}
```

```
import java.util.*;

public class FlatMapSplitForEachLogging {
   public static void main(String[] args) {
      List<String> sentences = Arrays.asList("Java Stream", "Flat Map");

      for (String s : sentences) {
         String[] words = s.split(" ");
         for (String word : words) {
            System.out.println("Word: " + word);
         }
      }
   }
}
```

11 flatMap() split data + forEach() Logging

```
import java.util.*;

public class FlatMapSplitForEachLogging {
   public static void main(String[] args) {
      List<String> sentences = List.of("Java Stream", "Flat Map");

      sentences.stream()
            .flatMap(s -> Arrays.stream(s.split(" ")))
```

```
            .forEach(word -> System.out.println("Word: " + word));
    }
}
```

```
import java.util.*;

public class FlatMapExpandForEachLogging {
    public static void main(String[] args) {
        Map<String, List<String>> data = new HashMap<String, List<String>>();
        data.put("A", Arrays.asList("x", "y"));
        data.put("B", Arrays.asList("z"));

        for (List<String> values : data.values()) {
            for (String v : values) {
                System.out.println("Value: " + v);
            }
        }
    }
}
```

12 flatMap() expand collections + forEach() Logging

```
import java.util.*;

public class FlatMapExpandForEachLogging {
    public static void main(String[] args) {
        Map<String, List<String>> data = Map.of(
                "A", List.of("x", "y"),
                "B", List.of("z")
        );

        data.values().stream()
                .flatMap(Collection::stream)
                .forEach(v -> System.out.println("Value: " + v));
    }
}
```

```
import java.util.*;

public class SortedNaturalForEachLogging {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<Integer>(
                Arrays.asList(5, 1, 3)
        );
```

```
        Collections.sort(numbers);

        for (Integer n : numbers) {
            System.out.println("Sorted: " + n);
        }
    }
}
```

13 sorted() natural ordering + forEach() Logging

```
import java.util.*;

public class SortedNaturalForEachLogging {
    public static void main(String[] args) {
        List<Integer> numbers = List.of(5, 1, 3);

        numbers.stream()
                .sorted()
                .forEach(n -> System.out.println("Sorted: " + n));
    }
}
```

```
import java.util.*;

public class SortedCustomForEachLogging {
    public static void main(String[] args) {
        List<String> names = new ArrayList<String>(
                Arrays.asList("Anurag", "Ravi", "Amit")
        );

        Collections.sort(names, new Comparator<String>() {
            public int compare(String a, String b) {
                return b.compareTo(a);                    // reverse
            }
        });

        for (String name : names) {
            System.out.println("Name: " + name);
        }
    }
}
```

14 sorted() custom comparator + forEach() Logging

```
import java.util.*;

public class SortedCustomForEachLogging {
    public static void main(String[] args) {
        List<String> names = List.of("Anurag", "Ravi", "Amit");
```

```
        names.stream()
            .sorted(Comparator.reverseOrder())
            .forEach(name -> System.out.println("Name: " + name));
    }
}
```

```
import java.util.*;

public class SortedMultiFieldForEachLogging {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<Employee>();
        employees.add(new Employee("Anurag", 30));
        employees.add(new Employee("Anurag", 25));
        employees.add(new Employee("Ravi", 35));

        Collections.sort(employees, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0) {
                    return nameCompare;
                }
                return e1.age - e2.age;
            }
        });

        for (Employee e : employees) {
            System.out.println(e.name + " - " + e.age);
        }
    }
}
```

15 sorted() multi-field + forEach() Logging

```
import java.util.*;

public class SortedMultiFieldForEachLogging {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {
        List<Employee> employees = List.of(
                new Employee("Anurag", 30),
                new Employee("Anurag", 25),
                new Employee("Ravi", 35)
        );

        employees.stream()
                .sorted(Comparator
                        .comparing((Employee e) -> e.name)
                        .thenComparingInt(e -> e.age))
                .forEach(e ->
                        System.out.println(e.name + " - " + e.age));
    }
}
```

```
import java.util.*;

public class PeekLoggingForEachLogging {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1, 2, 3);

        for (Integer n : list) {
            System.out.println("Peek log: " + n);
            System.out.println("Final: " + n);
```

```
    }
  }
}
```

16 peek() logging + forEach() Logging

```
import java.util.*;

public class PeekLoggingForEachLogging {
  public static void main(String[] args) {
    List.of(1, 2, 3).stream()
        .peek(n -> System.out.println("Peek log: " + n))
        .forEach(n -> System.out.println("Final: " + n));
  }
}
```

```
import java.util.*;

public class PeekDebugForEachLogging {
  public static void main(String[] args) {
    List<Integer> list = Arrays.asList(10, 20, 30);

    for (Integer n : list) {
      if (n == 20) {
        System.out.println("Debug breakpoint hit: " + n);
      }
      System.out.println("Processed: " + n);
    }
  }
}
```

17 peek() debugging + forEach() Logging

```
import java.util.*;

public class PeekDebugForEachLogging {
  public static void main(String[] args) {
    List.of(10, 20, 30).stream()
        .peek(n -> {
          if (n == 20) {
            System.out.println("Debug breakpoint hit: " + n);
          }
        })
        .forEach(n -> System.out.println("Processed: " + n));
  }
}
```

```
import java.util.*;

public class PeekMetricsForEachLogging {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(5, 10, 15);
      int counter = 0;

      for (Integer n : list) {
         counter++;
         System.out.println("Value: " + n);
      }

      System.out.println("Total processed: " + counter);
   }
}
```

18 peek() metrics + forEach() Logging

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsForEachLogging {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .forEach(n -> System.out.println("Value: " + n));

      System.out.println("Total processed: " + counter.get());
   }
}
```

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + foreach() printing

filter() null checks + foreach() printing

filter() comparisons + foreach() printing

filter() method calls returning boolean + foreach() printing

distinct() rely on equals() / hashcode() + foreach() printing

distinct() immutable objects + foreach() printing

map() transform objects + foreach() printing

map() create new objects + foreach() printing

map() call pure functions + foreach() printing

flatmap() return stream + foreach() printing

flatmap() split data + foreach() printing

flatmap() expand collections + foreach() printing

sorted() natural ordering + foreach() printing

sorted() custom comparator + foreach() printing

sorted() multi-field compare + foreach() printing

peek() logging + foreach() printing

peek() debugging + foreach() printing

peek() metrics + foreach() printing

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **One clear intermediate operation** is demonstrated
- **forEach() is the terminal operation**
- **Purpose = Printing output (not logging, not collecting)**
- Java **8+ compatible**
- Clean, interview-ready, minimal noise

```
import java.util.*;

public class FilterBooleanForEachPrint {
   public static void main(String[] args) {

      List<Integer> numbers = new ArrayList<Integer>();
      numbers.add(10);
      numbers.add(15);
      numbers.add(20);
      numbers.add(25);

      for (Integer n : numbers) {
         if (n % 2 == 0) {
            System.out.println(n);
         }
      }
   }
}
```

1 filter() boolean checks + forEach() Printing

```
import java.util.*;

public class FilterBooleanForEachPrint {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      numbers.stream()
            .filter(n -> n % 2 == 0)
```

```
            .forEach(System.out::println);
    }
}
```

```
import java.util.*;

public class FilterNullForEachPrint {
    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add(null);
        names.add("Amit");

        for (String name : names) {
            if (name != null) {
                System.out.println(name);
            }
        }
    }
}
```

2 filter() null checks + forEach() Printing

```
import java.util.*;

public class FilterNullForEachPrint {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Anurag", null, "Amit");

        names.stream()
            .filter(Objects::nonNull)
            .forEach(System.out::println);
    }
}
```

```
import java.util.*;

public class FilterComparisonForEachPrint {
    public static void main(String[] args) {

        List<Integer> salaries = new ArrayList<Integer>();
```

```
        salaries.add(20000);
        salaries.add(40000);
        salaries.add(60000);

        for (Integer s : salaries) {
            if (s >= 30000) {
                System.out.println(s);
            }
        }
    }
}
```

3 filter() comparisons + forEach() Printing

```
import java.util.*;

public class FilterComparisonForEachPrint {
    public static void main(String[] args) {
        List<Integer> salaries = List.of(20000, 40000, 60000);

        salaries.stream()
                .filter(s -> s >= 30000)
                .forEach(System.out::println);
    }
}
```

```
import java.util.*;

public class FilterMethodForEachPrint {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {

        List<User> users = new ArrayList<User>();
        users.add(new User("Anurag", true));
```

```
        users.add(new User("Ravi", false));

        for (User u : users) {
            if (u.isActive()) {
                System.out.println(u.name);
            }
        }
    }
}
```

4 filter() method call returning boolean + forEach() Printing

```
import java.util.*;

public class FilterMethodForEachPrint {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {
        List<User> users = List.of(
                new User("Anurag", true),
                new User("Ravi", false)
        );

        users.stream()
                .filter(User::isActive)
                .forEach(u -> System.out.println(u.name));
    }
}
```

```
import java.util.*;

public class DistinctEqualsForEachPrint {

   static class Person {
      String name;

      Person(String name) {
         this.name = name;
      }

      public boolean equals(Object o) {
         if (o instanceof Person) {
            Person p = (Person) o;
            return name.equals(p.name);
         }
         return false;
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> people = new ArrayList<Person>();
      people.add(new Person("Anurag"));
      people.add(new Person("Anurag"));
      people.add(new Person("Ravi"));

      Set<Person> unique = new HashSet<Person>(people);

      for (Person p : unique) {
         System.out.println(p.name);
      }
   }
}
```

5 distinct() rely on equals() / hashCode() + forEach() Printing

```
import java.util.*;

public class DistinctEqualsForEachPrint {

    static class Person {
        String name;

        Person(String name) { this.name = name; }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        people.stream()
                .distinct()
                .forEach(p -> System.out.println(p.name));
    }
}
```

```
import java.util.*;

public class DistinctImmutableForEachPrint {
   public static void main(String[] args) {

      List<String> languages = new ArrayList<String>();
      languages.add("Java");
      languages.add("Java");
      languages.add("Python");

      Set<String> unique = new HashSet<String>(languages);

      for (String lang : unique) {
         System.out.println(lang);
      }
   }
}
```

6 distinct() immutable objects + forEach() Printing

```
import java.util.*;

public class DistinctImmutableForEachPrint {
   public static void main(String[] args) {
      List<String> languages = List.of("Java", "Java", "Python");

      languages.stream()
            .distinct()
            .forEach(System.out::println);
   }
}
```

```
import java.util.*;

public class MapTransformForEachPrint {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
```

```
        names.add("anurag");
        names.add("amit");

        for (String name : names) {
            System.out.println(name.toUpperCase());
        }
    }
}
```

7 map() transform objects + forEach() Printing

```
import java.util.*;

public class MapTransformForEachPrint {
    public static void main(String[] args) {
        List<String> names = List.of("anurag", "amit");

        names.stream()
                .map(String::toUpperCase)
                .forEach(System.out::println);
    }
}
```

```
import java.util.*;

public class MapNewObjectForEachPrint {

    static class UserDTO {
        String name;

        UserDTO(String name) {
            this.name = name;
        }
    }

    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add("Amit");

        for (String name : names) {
            UserDTO dto = new UserDTO(name);
            System.out.println(dto.name);
        }
    }
}
```

8 map() create new objects + forEach() Printing

```
import java.util.*;

public class MapNewObjectForEachPrint {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Amit");

      names.stream()
            .map(UserDTO::new)
            .forEach(dto -> System.out.println(dto.name));
   }
}
```

```
import java.util.*;

public class MapPureFunctionForEachPrint {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List<Integer> numbers = new ArrayList<Integer>();
      numbers.add(2);
      numbers.add(3);
      numbers.add(4);

      for (Integer n : numbers) {
         System.out.println(square(n));
      }
```

```
    }
}
```

9 map() call pure functions + forEach() Printing

```
import java.util.*;

public class MapPureFunctionForEachPrint {

    static int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {
        List<Integer> numbers = List.of(2, 3, 4);

        numbers.stream()
                .map(MapPureFunctionForEachPrint::square)
                .forEach(System.out::println);
    }
}
```

```
import java.util.*;

public class FlatMapStreamForEachPrint {
    public static void main(String[] args) {

        List<List<Integer>> lists = new ArrayList<List<Integer>>();

        lists.add(Arrays.asList(1, 2));
        lists.add(Arrays.asList(3, 4));

        for (List<Integer> inner : lists) {
            for (Integer n : inner) {
                System.out.println(n);
            }
        }
    }
}
```

10 flatMap() return Stream + forEach() Printing

```
import java.util.*;

public class FlatMapStreamForEachPrint {
   public static void main(String[] args) {
      List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      );

      lists.stream()
            .flatMap(List::stream)
            .forEach(System.out::println);
   }
}
```

```
import java.util.*;

public class FlatMapSplitForEachPrint {
   public static void main(String[] args) {

      List<String> sentences = new ArrayList<String>();
      sentences.add("Java Stream");
      sentences.add("Flat Map");

      for (String sentence : sentences) {
         String[] words = sentence.split(" ");
         for (String word : words) {
            System.out.println(word);
         }
      }
   }
}
```

11 flatMap() split data + forEach() Printing

```
import java.util.*;
```

```
public class FlatMapSplitForEachPrint {
   public static void main(String[] args) {
      List<String> sentences = List.of("Java Stream", "Flat Map");
      sentences.stream()
            .flatMap(s -> Arrays.stream(s.split(" ")))
            .forEach(System.out::println);
   }
}
```

```
import java.util.*;

public class FlatMapExpandForEachPrint {
   public static void main(String[] args) {

      Map<String, List<String>> data = new HashMap<String, List<String>>();
      data.put("A", Arrays.asList("x", "y"));
      data.put("B", Arrays.asList("z"));

      for (List<String> list : data.values()) {
         for (String value : list) {
            System.out.println(value);
         }
      }
   }
}
```

12 flatMap() expand collections + forEach() Printing

```
import java.util.*;

public class FlatMapExpandForEachPrint {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("x", "y"),
            "B", List.of("z")
      );

      data.values().stream()
            .flatMap(Collection::stream)
            .forEach(System.out::println);
   }
}
```

```
import java.util.*;

public class SortedNaturalForEachPrint {
   public static void main(String[] args) {

      List<Integer> numbers = new ArrayList<Integer>();
      numbers.add(5);
      numbers.add(1);
      numbers.add(3);

      Collections.sort(numbers);

      for (Integer n : numbers) {
         System.out.println(n);
      }
   }
}
```

13 sorted() natural ordering + forEach() Printing

```
import java.util.*;

public class SortedNaturalForEachPrint {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(5, 1, 3);

      numbers.stream()
            .sorted()
            .forEach(System.out::println);
   }
}
```

```
import java.util.*;

public class SortedCustomForEachPrint {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("Anurag");
      names.add("Ravi");
      names.add("Amit");

      Collections.sort(names, new Comparator<String>() {
         public int compare(String a, String b) {
            return b.compareTo(a);                                    // reverse
         }
      });

      for (String name : names) {
         System.out.println(name);
      }
   }
}
```

14 sorted() custom comparator + forEach() Printing

```
import java.util.*;

public class SortedCustomForEachPrint {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      names.stream()
            .sorted(Comparator.reverseOrder())
            .forEach(System.out::println);
   }
}
```

```
import java.util.*;

public class SortedMultiFieldForEachPrint {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> employees = new ArrayList<Employee>();
      employees.add(new Employee("Anurag", 30));
      employees.add(new Employee("Anurag", 25));
      employees.add(new Employee("Ravi", 35));

      Collections.sort(employees, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {

            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0) {
               return nameCompare;
            }

            return e1.age - e2.age;
         }
      });

      for (Employee e : employees) {
         System.out.println(e.name + " " + e.age);
      }
   }
}
```

15 sorted() multi-field compare + forEach() Printing

```
import java.util.*;

public class SortedMultiFieldForEachPrint {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {
      List<Employee> employees = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      );

      employees.stream()
            .sorted(Comparator
                  .comparing((Employee e) -> e.name)
                  .thenComparingInt(e -> e.age))
            .forEach(e -> System.out.println(e.name + " " + e.age));
   }
}
```

```
import java.util.*;

public class PeekLoggingForEachPrint {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);
```

```
        for (Integer n : list) {
            System.out.println("Peek: " + n);
            System.out.println(n);
        }
    }
}
```

16 peek() logging + forEach() Printing

```
import java.util.*;

public class PeekLoggingForEachPrint {
    public static void main(String[] args) {
        List.of(1, 2, 3).stream()
                .peek(n -> System.out.println("Peek: " + n))
                .forEach(System.out::println);
    }
}
```

```
import java.util.*;

public class PeekDebugForEachPrint {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(10, 20, 30);

        for (Integer n : list) {

            if (n == 20) {
                System.out.println("Debug hit");
            }

            System.out.println(n);
        }
    }
}
```

17 peek() debugging + forEach() Printing

```
import java.util.*;

public class PeekDebugForEachPrint {
    public static void main(String[] args) {
        List.of(10, 20, 30).stream()
                .peek(n -> {
                    if (n == 20) System.out.println("Debug hit");
```

```
            })
            .forEach(System.out::println);
    }
}
```

```
import java.util.*;

public class PeekMetricsForEachPrint {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(5, 10, 15);
        int counter = 0;

        for (Integer n : list) {
            counter++;
            System.out.println(n);
        }

        System.out.println("Total processed: " + counter);
    }
}
```

18 peek() metrics + forEach() Printing

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsForEachPrint {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .forEach(System.out::println);

        System.out.println("Total processed: " + counter.get());
    }
}
```

---

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + foreach() updating external systems

filter() null checks + foreach() updating external systems

filter() comparisons + foreach() updating external systems

filter() method calls returning boolean + foreach() updating external systems

distinct() rely on equals() / hashcode() + foreach() updating external systems

distinct() immutable objects + foreach() updating external systems

map() transform objects + foreach() updating external systems

map() create new objects + foreach() updating external systems

map() call pure functions + foreach() updating external systems

flatmap() return stream + foreach() updating external systems

flatmap() split data + foreach() updating external systems

flatmap() expand collections + foreach() updating external systems

sorted() natural ordering + foreach() updating external systems

sorted() custom comparator + foreach() updating external systems

sorted() multi-field compare + foreach() updating external systems

peek() logging + foreach() updating external systems

peek() debugging + foreach() updating external systems

peek() metrics + foreach() updating external systems

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** showing **side-effects with forEach()** where the stream **updates an external system** (simulated as DB / cache / message queue / API).

 Important concept reminder (intentional here):
Using forEach() to update external systems is a **side-effect** and must be done **only at the terminal operation**. These examples are **educational and realistic**, not advocating misuse.

All examples:

- Java **8+**
- One intermediate concept per class
- External system = simulated service (ExternalSystem.update(...))

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;
public class FilterBooleanUpdateExternal {
  public static void main(String[] args) {
    List<Integer> orders = Arrays.asList(100, 150, 200);

    for (Integer o : orders) {
      if (o >= 150) {
        ExternalSystem.update("Approved order: " + o);
      }
    }
  }
}
```

1 filter() boolean checks + forEach() Updating external systems

```
import java.util.*;

public class FilterBooleanUpdateExternal {
   public static void main(String[] args) {
      List<Integer> orders = List.of(100, 150, 200);
      orders.stream()
            .filter(o -> o >= 150)
            .forEach(o -> ExternalSystem.update("Approved order: " + o));
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
            public static void update(String message) {
                        System.out.println("External system updated: " + message);
            }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class FilterNullUpdateExternal {
   public static void main(String[] args) {
      List<String> users = Arrays.asList("Anurag", null, "Amit");

      for (String u : users) {
         if (u != null) {
            ExternalSystem.update("User synced: " + u);
         }
      }
   }
}
```

2 filter() null checks + forEach() Updating external systems

```
import java.util.*;

public class FilterNullUpdateExternal {
   public static void main(String[] args) {
      List<String> users = Arrays.asList("Anurag", null, "Amit");

      users.stream()
            .filter(Objects::nonNull)
            .forEach(u -> ExternalSystem.update("User synced: " + u));
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class FilterComparisonUpdateExternal {
   public static void main(String[] args) {
      List<Integer> balances = Arrays.asList(500, 2000, 7000);

      for (Integer b : balances) {
         if (b > 1000) {
            ExternalSystem.update("Premium balance: " + b);
         }
      }
   }
}
```

3 filter() comparisons + forEach() Updating external systems

```
import java.util.*;

public class FilterComparisonUpdateExternal {
   public static void main(String[] args) {
      List<Integer> balances = List.of(500, 2000, 7000);

      balances.stream()
            .filter(b -> b > 1000)
            .forEach(b -> ExternalSystem.update("Premium balance: " + b));
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class FilterMethodUpdateExternal {

    static class Account {
        String id;
        boolean active;

        Account(String id, boolean active) {
            this.id = id;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {
        List<Account> accounts = Arrays.asList(
                new Account("A1", true),
                new Account("A2", false)
        );

        for (Account a : accounts) {
            if (a.isActive()) {
                ExternalSystem.update("Activated account: " + a.id);
            }
        }
    }
```

```
}
```

4 filter() method call returning boolean + forEach() Updating external systems

```
import java.util.*;

public class FilterMethodUpdateExternal {

    static class Account {
        String id;
        boolean active;

        Account(String id, boolean active) {
            this.id = id;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {
        List<Account> accounts = List.of(
                new Account("A1", true),
                new Account("A2", false)
        );

        accounts.stream()
                .filter(Account::isActive)
                .forEach(a -> ExternalSystem.update("Activated account: " + a.id));
    }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class DistinctEqualsUpdateExternal {

    static class Customer {
        String email;

        Customer(String email) {
            this.email = email;
        }

        @Override
        public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof Customer)) return false;
            Customer c = (Customer) o;
            return email.equals(c.email);
        }

        @Override
        public int hashCode() {
            return email.hashCode();
        }
    }

    public static void main(String[] args) {
        List<Customer> customers = Arrays.asList(
                new Customer("a@mail.com"),
                new Customer("a@mail.com"),
                new Customer("b@mail.com")
```

```
        );

        Set<Customer> unique = new HashSet<Customer>(customers);

        for (Customer c : unique) {
            ExternalSystem.update("Customer saved: " + c.email);
        }
    }
}
```

5 distinct() equals/hashCode + forEach() Updating external systems

```
import java.util.*;

public class DistinctEqualsUpdateExternal {

    static class Customer {
        String email;

        Customer(String email) { this.email = email; }

        @Override
        public boolean equals(Object o) {
            return o instanceof Customer c && email.equals(c.email);
        }

        @Override
        public int hashCode() {
            return Objects.hash(email);
        }
    }

    public static void main(String[] args) {
        List<Customer> customers = List.of(
                new Customer("a@mail.com"),
                new Customer("a@mail.com"),
                new Customer("b@mail.com")
        );

        customers.stream()
                .distinct()
                .forEach(c -> ExternalSystem.update("Customer saved: " + c.email));
    }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
```

```
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class DistinctImmutableUpdateExternal {
   public static void main(String[] args) {
      List<String> tags = Arrays.asList("java", "java", "spring");

      Set<String> unique = new HashSet<String>(tags);

      for (String t : unique) {
         ExternalSystem.update("Tag indexed: " + t);
      }
   }
}
```

6 distinct() immutable objects + forEach() Updating external systems

```
import java.util.*;

public class DistinctImmutableUpdateExternal {
   public static void main(String[] args) {
      List<String> tags = List.of("java", "java", "spring");

      tags.stream()
            .distinct()
            .forEach(t -> ExternalSystem.update("Tag indexed: " + t));
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;
```

```
public class MapTransformUpdateExternal {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("anurag", "amit");

      for (String name : names) {
         String upper = name.toUpperCase();
         ExternalSystem.update("Normalized name: " + upper);
      }
   }
}
```

7 map() transform objects + forEach() Updating external systems

```
import java.util.*;

public class MapTransformUpdateExternal {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit");

      names.stream()
            .map(String::toUpperCase)
            .forEach(n -> ExternalSystem.update("Normalized name: " + n));
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
            public static void update(String message) {
                        System.out.println("External system updated: " + message);
            }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class MapNewObjectUpdateExternal {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", "Amit");

      for (String name : names) {
         UserDTO dto = new UserDTO(name);
         ExternalSystem.update("DTO persisted: " + dto.name);
```

```
    }
  }
}
```

8 map() create new objects + forEach() Updating external systems

```
import java.util.*;

public class MapNewObjectUpdateExternal {

  static class UserDTO {
    String name;
    UserDTO(String name) { this.name = name; }
  }

  public static void main(String[] args) {
    List<String> names = List.of("Anurag", "Amit");

    names.stream()
        .map(UserDTO::new)
        .forEach(dto -> ExternalSystem.update("DTO persisted: " + dto.name));
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class MapPureFunctionUpdateExternal {

  static int calculateScore(int n) {
    return n * 10;
  }

  public static void main(String[] args) {
```

```
    List<Integer> inputs = Arrays.asList(1, 2, 3);

    for (Integer n : inputs) {
      int score = calculateScore(n);
      ExternalSystem.update("Score sent: " + score);
    }
  }
}
```

9 map() call pure functions + forEach() Updating external systems

```
import java.util.*;

public class MapPureFunctionUpdateExternal {

  static int calculateScore(int n) {
    return n * 10;
  }

  public static void main(String[] args) {
    List<Integer> inputs = List.of(1, 2, 3);

    inputs.stream()
        .map(MapPureFunctionUpdateExternal::calculateScore)
        .forEach(score -> ExternalSystem.update("Score sent: " + score));
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class FlatMapStreamUpdateExternal {
  public static void main(String[] args) {
    List<List<String>> batches = new ArrayList<List<String>>();
    batches.add(Arrays.asList("A", "B"));
    batches.add(Arrays.asList("C"));

    for (List<String> batch : batches) {
      for (String v : batch) {
```

```
            ExternalSystem.update("Batch item: " + v);
        }
    }
  }
}
```

10 flatMap() return Stream + forEach() Updating external systems

```
import java.util.*;

public class FlatMapStreamUpdateExternal {
  public static void main(String[] args) {
    List<List<String>> batches = List.of(
          List.of("A", "B"),
          List.of("C")
    );

    batches.stream()
          .flatMap(List::stream)
          .forEach(v -> ExternalSystem.update("Batch item: " + v));
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
            public static void update(String message) {
                        System.out.println("External system updated: " + message);
            }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class FlatMapSplitUpdateExternal {
  public static void main(String[] args) {
    List<String> lines = Arrays.asList("Java Stream", "Flat Map");

    for (String line : lines) {
      String[] words = line.split(" ");
      for (String word : words) {
        ExternalSystem.update("Word indexed: " + word);
      }
```

```
    }
  }
}
```

11 flatMap() split data + forEach() Updating external systems

```
import java.util.*;

public class FlatMapSplitUpdateExternal {
  public static void main(String[] args) {
    List<String> lines = List.of("Java Stream", "Flat Map");

    lines.stream()
        .flatMap(l -> Arrays.stream(l.split(" ")))
        .forEach(word -> ExternalSystem.update("Word indexed: " + word));
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class FlatMapExpandUpdateExternal {
  public static void main(String[] args) {

    Map<String, List<String>> data = new HashMap<String, List<String>>();
    data.put("A", Arrays.asList("1", "2"));
    data.put("B", Arrays.asList("3"));

    for (List<String> values : data.values()) {
      for (String v : values) {
        ExternalSystem.update("Value synced: " + v);
      }
    }
```

```
    }
}
```

12 flatMap() expand collections + forEach() Updating external systems

```
import java.util.*;

public class FlatMapExpandUpdateExternal {
    public static void main(String[] args) {
        Map<String, List<String>> data = Map.of(
            "A", List.of("1", "2"),
            "B", List.of("3")
        );

        data.values().stream()
            .flatMap(Collection::stream)
            .forEach(v -> ExternalSystem.update("Value synced: " + v));
    }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class SortedNaturalUpdateExternal {
    public static void main(String[] args) {
        List<Integer> priorities = Arrays.asList(3, 1, 2);

        Collections.sort(priorities);

        for (Integer p : priorities) {
            ExternalSystem.update("Processed priority: " + p);
        }
    }
```

```
}
```

13 sorted() natural ordering + forEach() Updating external systems

```
import java.util.*;

public class SortedNaturalUpdateExternal {
   public static void main(String[] args) {
      List<Integer> priorities = List.of(3, 1, 2);

      priorities.stream()
            .sorted()
            .forEach(p -> ExternalSystem.update("Processed priority: " + p));
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
            public static void update(String message) {
                        System.out.println("External system updated: " + message);
            }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class SortedCustomUpdateExternal {
   public static void main(String[] args) {
      List<String> jobs = Arrays.asList("LOW", "HIGH", "MEDIUM");

      Collections.sort(jobs, Collections.reverseOrder());

      for (String j : jobs) {
         ExternalSystem.update("Job executed: " + j);
      }
   }
}
```

14 sorted() custom comparator + forEach() Updating external systems

```
import java.util.*;

public class SortedCustomUpdateExternal {
   public static void main(String[] args) {
      List<String> jobs = List.of("LOW", "HIGH", "MEDIUM");

      jobs.stream()
            .sorted(Comparator.reverseOrder())
            .forEach(j -> ExternalSystem.update("Job executed: " + j));
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
            public static void update(String message) {
                        System.out.println("External system updated: " + message);
            }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class SortedMultiFieldUpdateExternal {

   static class Task {
      String name;
      int priority;

      Task(String name, int priority) {
         this.name = name;
         this.priority = priority;
      }
   }

   public static void main(String[] args) {
      List<Task> tasks = Arrays.asList(
            new Task("Deploy", 2),
            new Task("Build", 1),
            new Task("Test", 2)
      );

      Collections.sort(tasks, new Comparator<Task>() {
         public int compare(Task t1, Task t2) {
            if (t1.priority != t2.priority) {
               return t1.priority - t2.priority;
            }
            return t1.name.compareTo(t2.name);
         }
      });
```

```
        for (Task t : tasks) {
            ExternalSystem.update("Task executed: " + t.name);
        }
    }
}
```

15 sorted() multi-field compare + forEach() Updating external systems

```
import java.util.*;

public class SortedMultiFieldUpdateExternal {

    static class Task {
        String name;
        int priority;

        Task(String name, int priority) {
            this.name = name;
            this.priority = priority;
        }
    }

    public static void main(String[] args) {
        List<Task> tasks = List.of(
                new Task("Deploy", 2),
                new Task("Build", 1),
                new Task("Test", 2)
        );

        tasks.stream()
                .sorted(Comparator
                        .comparing((Task t) -> t.priority)
                        .thenComparing(t -> t.name))
                .forEach(t ->
                        ExternalSystem.update("Task executed: " + t.name));
    }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class PeekLoggingUpdateExternal {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(1, 2, 3);

      for (Integer n : list) {
         System.out.println("Peek log: " + n);
         ExternalSystem.update("Final update: " + n);
      }
   }
}
```

16 peek() logging + forEach() Updating external systems

```
import java.util.*;

public class PeekLoggingUpdateExternal {
   public static void main(String[] args) {
      List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Peek log: " + n))
            .forEach(n -> ExternalSystem.update("Final update: " + n));
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
```

```
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class PeekDebugUpdateExternal {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(10, 20, 30);

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug breakpoint hit");
         }
         ExternalSystem.update("Processed value: " + n);
      }
   }
}
```

17 peek() debugging + forEach() Updating external systems

```
import java.util.*;

public class PeekDebugUpdateExternal {
   public static void main(String[] args) {
      List.of(10, 20, 30).stream()
            .peek(n -> {
               if (n == 20) System.out.println("Debug breakpoint hit");
            })
            .forEach(n -> ExternalSystem.update("Processed value: " + n));
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

```
import java.util.*;

public class PeekMetricsUpdateExternal {
   public static void main(String[] args) {

      int counter = 0;
      List<Integer> list = Arrays.asList(5, 10, 15);

      for (Integer n : list) {
         counter++;
         ExternalSystem.update("Metric sent: " + n);
      }

      ExternalSystem.update("Total processed: " + counter);
   }
}
```

18 peek() metrics + forEach() Updating external systems

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsUpdateExternal {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .forEach(n -> ExternalSystem.update("Metric sent: " + n));

      ExternalSystem.update("Total processed: " + counter.get());
```

```
    }
}
```

---

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + reduce() combine values

filter() null checks + reduce() combine values

filter() comparisons + reduce() combine values

filter() method calls returning boolean + reduce() combine values

distinct() rely on equals() / hashcode() + reduce() combine values

distinct() immutable objects + reduce() combine values

map() transform objects + reduce() combine values

map() create new objects + reduce() combine values

map() call pure functions + reduce() combine values

flatmap() return stream + reduce() combine values

flatmap() split data + reduce() combine values

flatmap() expand collections + reduce() combine values

sorted() natural ordering + reduce() combine values

sorted() custom comparator + reduce() combine values

sorted() multi-field compare + reduce() combine values

peek() logging + reduce() combine values

peek() debugging + reduce() combine values

peek() metrics + reduce() combine values

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **every requested Stream combination**, where:

- **One primary intermediate operation** is demonstrated
- **reduce() is the terminal operation**
- **Goal = combine values** (sum, concat, merge)
- Java **8+ compatible**
- Clean, deterministic, interview-ready
- No collectors, no forEach

 Mental model
reduce(identity, accumulator) ⇒ **Combine stream elements into ONE result**

```
import java.util.*;

public class FilterBooleanReduce {
   public static void main(String[] args) {

      List<Integer> numbers = new ArrayList<Integer>();
      numbers.add(10);
      numbers.add(15);
      numbers.add(20);
      numbers.add(25);

      int sum = 0;

      for (Integer n : numbers) {
         if (n % 2 == 0) {                                    // filter
            sum += n;                                         // reduce
         }
      }

      System.out.println(sum);
```

```
    }
}
```

1 filter() boolean checks + reduce() Combine values

```
import java.util.*;

public class FilterBooleanReduce {
    public static void main(String[] args) {
        List<Integer> numbers = List.of(10, 15, 20, 25);

        int sum =
                numbers.stream()
                        .filter(n -> n % 2 == 0)
                        .reduce(0, Integer::sum);

        System.out.println(sum);
    }
}
```

```
import java.util.*;

public class FilterNullReduce {
    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add(null);
        names.add("Amit");

        String combined = "";

        for (String name : names) {
            if (name != null) {                    // filter
                combined += name;                  // reduce
            }
        }
```

```
        System.out.println(combined);
    }
}
```

2 filter() null checks + reduce() Combine values

```
import java.util.*;

public class FilterNullReduce {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Anurag", null, "Amit");

        String combined =
                names.stream()
                        .filter(Objects::nonNull)
                        .reduce("", (a, b) -> a + b);

        System.out.println(combined);
    }
}
```

```
import java.util.*;

public class FilterComparisonReduce {
    public static void main(String[] args) {

        List<Integer> salaries = new ArrayList<Integer>();
        salaries.add(20000);
        salaries.add(40000);
        salaries.add(60000);

        int total = 0;

        for (Integer s : salaries) {
            if (s >= 30000) {
                total += s;
            }
        }
```

```
        System.out.println(total);
    }
}
```

3 filter() comparisons + reduce() Combine values

```
import java.util.*;

public class FilterComparisonReduce {
    public static void main(String[] args) {
        List<Integer> salaries = List.of(20000, 40000, 60000);

        int total =
                salaries.stream()
                        .filter(s -> s >= 30000)
                        .reduce(0, Integer::sum);

        System.out.println(total);
    }
}
```

```
import java.util.*;

public class FilterMethodReduce {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }
```

```
    public static void main(String[] args) {

        List<User> users = new ArrayList<User>();
        users.add(new User("Anurag", true));
        users.add(new User("Amit", true));
        users.add(new User("Ravi", false));

        int count = 0;

        for (User u : users) {
            if (u.isActive()) {
                count += 1;
            }
        }

        System.out.println(count);
    }
}
```

4 filter() method call boolean + reduce() Combine values

```
import java.util.*;

public class FilterMethodReduce {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {
        List<User> users = List.of(
                new User("Anurag", true),
                new User("Amit", true),
                new User("Ravi", false)
        );

        int count =
                users.stream()
                        .filter(User::isActive)
                        .map(u -> 1)
                        .reduce(0, Integer::sum);
```

```
        System.out.println(count);
    }
}
```

```
import java.util.*;

public class DistinctEqualsReduce {

    static class Person {
        String name;

        Person(String name) {
            this.name = name;
        }

        public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof Person)) return false;
            Person p = (Person) o;
            return name.equals(p.name);
        }

        public int hashCode() {
            return name.hashCode();
        }
    }

    public static void main(String[] args) {

        List<Person> people = new ArrayList<Person>();
        people.add(new Person("Anurag"));
        people.add(new Person("Anurag"));
        people.add(new Person("Ravi"));

        Set<Person> unique = new HashSet<Person>(people);  // distinct

        int totalLength = 0;

        for (Person p : unique) {
            totalLength += p.name.length();
```

```
        }

        System.out.println(totalLength);
    }
}
```

5 distinct() equals/hashCode + reduce() Combine values

```
import java.util.*;

public class DistinctEqualsReduce {

    static class Person {
        String name;

        Person(String name) { this.name = name; }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        int totalLength =
                people.stream()
                        .distinct()
                        .map(p -> p.name.length())
                        .reduce(0, Integer::sum);

        System.out.println(totalLength);
    }
```

```
}
```

```
import java.util.*;

public class DistinctImmutableReduce {
   public static void main(String[] args) {

      List<String> values = new ArrayList<String>();
      values.add("Java");
      values.add("Java");
      values.add("Go");

      Set<String> unique = new HashSet<String>(values);

      int lengthSum = 0;

      for (String s : unique) {
         lengthSum += s.length();
      }

      System.out.println(lengthSum);
   }
}
```

6 distinct() immutable objects + reduce() Combine values

```
import java.util.*;

public class DistinctImmutableReduce {
   public static void main(String[] args) {
      List<String> values = List.of("Java", "Java", "Go");

      int lengthSum =
            values.stream()
                  .distinct()
                  .map(String::length)
                  .reduce(0, Integer::sum);

      System.out.println(lengthSum);
```

```
    }
}
```

```
import java.util.*;

public class MapTransformReduce {
    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("anurag");
        names.add("amit");

        int totalChars = 0;

        for (String name : names) {
            String upper = name.toUpperCase();                    // map
            totalChars += upper.length();                         // reduce
        }

        System.out.println(totalChars);
    }
}
```

7 map() transform objects + reduce() Combine values

```
import java.util.*;

public class MapTransformReduce {
    public static void main(String[] args) {
        List<String> names = List.of("anurag", "amit");

        int totalChars =
                names.stream()
                        .map(String::toUpperCase)
                        .map(String::length)
                        .reduce(0, Integer::sum);

        System.out.println(totalChars);
    }
}
```

```
import java.util.*;

public class MapNewObjectReduce {

    static class UserDTO {
        String name;
        UserDTO(String name) {
            this.name = name;
        }
    }

    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add("Amit");

        int count = 0;

        for (String name : names) {
            UserDTO dto = new UserDTO(name);                    // map
            count += 1;                                         // reduce
        }

        System.out.println(count);
    }
}
```

8 map() create new objects + reduce() Combine values

```
import java.util.*;

public class MapNewObjectReduce {

    static class UserDTO {
        String name;
        UserDTO(String name) { this.name = name; }
```

```
    }

    public static void main(String[] args) {
        List<String> names = List.of("Anurag", "Amit");

        int count =
                names.stream()
                        .map(UserDTO::new)
                        .map(dto -> 1)
                        .reduce(0, Integer::sum);

        System.out.println(count);
    }
}
```

```
import java.util.*;

public class MapPureFunctionReduce {

    static int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {

        List<Integer> numbers = new ArrayList<Integer>();
        numbers.add(2);
        numbers.add(3);
        numbers.add(4);

        int sumOfSquares = 0;

        for (Integer n : numbers) {
            sumOfSquares += square(n);
        }

        System.out.println(sumOfSquares);
    }
}
```

9 map() pure functions + reduce() Combine values

```
import java.util.*;

public class MapPureFunctionReduce {

    static int square(int n) {
        return n * n;
    }
```

```
    public static void main(String[] args) {
        List<Integer> numbers = List.of(2, 3, 4);

        int sumOfSquares =
                numbers.stream()
                        .map(MapPureFunctionReduce::square)
                        .reduce(0, Integer::sum);

        System.out.println(sumOfSquares);
    }
}
```

```
import java.util.*;

public class FlatMapStreamReduce {
    public static void main(String[] args) {

        List<List<Integer>> lists = new ArrayList<List<Integer>>();

        lists.add(Arrays.asList(1, 2));
        lists.add(Arrays.asList(3, 4));

        int total = 0;

        for (List<Integer> inner : lists) {
            for (Integer n : inner) {
                total += n;
            }
        }

        System.out.println(total);
    }
}
```

10 flatMap() return Stream + reduce() Combine values

```
import java.util.*;

public class FlatMapStreamReduce {
    public static void main(String[] args) {
        List<List<Integer>> lists = List.of(
                List.of(1, 2),
                List.of(3, 4)
        );

        int total =
                lists.stream()
                        .flatMap(List::stream)
```

```
                .reduce(0, Integer::sum);

        System.out.println(total);
    }
}
```

```
import java.util.*;

public class FlatMapSplitReduce {
    public static void main(String[] args) {

        List<String> sentences = new ArrayList<String>();
        sentences.add("Java Stream");
        sentences.add("Flat Map");

        int wordCount = 0;

        for (String s : sentences) {
            String[] words = s.split(" ");
            wordCount += words.length;
        }

        System.out.println(wordCount);
    }
}
```

11 flatMap() split data + reduce() Combine values

```
import java.util.*;

public class FlatMapSplitReduce {
    public static void main(String[] args) {
        List<String> sentences = List.of("Java Stream", "Flat Map");

        int wordCount =
            sentences.stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
                .map(w -> 1)
                .reduce(0, Integer::sum);

        System.out.println(wordCount);
    }
}
```

```
import java.util.*;

public class FlatMapExpandReduce {
   public static void main(String[] args) {

      Map<String, List<Integer>> data = new HashMap<String, List<Integer>>();
      data.put("A", Arrays.asList(1, 2));
      data.put("B", Arrays.asList(3));

      int sum = 0;

      for (List<Integer> list : data.values()) {
         for (Integer n : list) {
            sum += n;
         }
      }

      System.out.println(sum);
   }
}
```

12 flatMap() expand collections + reduce() Combine values

```
import java.util.*;

public class FlatMapExpandReduce {
   public static void main(String[] args) {
      Map<String, List<Integer>> data = Map.of(
            "A", List.of(1, 2),
            "B", List.of(3)
      );

      int sum =
            data.values().stream()
                  .flatMap(Collection::stream)
                  .reduce(0, Integer::sum);
```

```
        System.out.println(sum);
    }
}
```

```
import java.util.*;

public class SortedNaturalReduce {
    public static void main(String[] args) {

        List<Integer> numbers = new ArrayList<Integer>();
        numbers.add(5);
        numbers.add(1);
        numbers.add(3);

        Collections.sort(numbers);                          // sorted

        int total = 0;

        for (Integer n : numbers) {
            total += n;
        }

        System.out.println(total);
    }
}
```

13 sorted() natural ordering + reduce() Combine values

```
import java.util.*;

public class SortedNaturalReduce {
    public static void main(String[] args) {
        List<Integer> numbers = List.of(5, 1, 3);

        int total =
                numbers.stream()
                        .sorted()
                        .reduce(0, Integer::sum);

        System.out.println(total);
    }
```

```
}
```

```
import java.util.*;

public class SortedCustomReduce {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("Anurag");
      names.add("Ravi");
      names.add("Amit");

      Collections.sort(names, Collections.reverseOrder());

      String combined = "";

      for (String name : names) {
         combined += name;
      }

      System.out.println(combined);
   }
}
```

14 sorted() custom comparator + reduce() Combine values

```
import java.util.*;

public class SortedCustomReduce {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      String combined =
            names.stream()
                  .sorted(Comparator.reverseOrder())
                  .reduce("", (a, b) -> a + b);

      System.out.println(combined);
   }
```

}

```
import java.util.*;

public class SortedMultiFieldReduce {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<Employee>();
        employees.add(new Employee("Anurag", 30));
        employees.add(new Employee("Anurag", 25));
        employees.add(new Employee("Ravi", 35));

        Collections.sort(employees, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0) return nameCompare;
                return e1.age - e2.age;
            }
        });

        int ageSum = 0;

        for (Employee e : employees) {
            ageSum += e.age;
        }

        System.out.println(ageSum);
```

```
    }
}
```

15 sorted() multi-field compare + reduce() Combine values

```
import java.util.*;

public class SortedMultiFieldReduce {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {
        List<Employee> employees = List.of(
                new Employee("Anurag", 30),
                new Employee("Anurag", 25),
                new Employee("Ravi", 35)
        );

        int ageSum =
                employees.stream()
                        .sorted(Comparator
                                .comparing((Employee e) -> e.name)
                                .thenComparingInt(e -> e.age))
                        .map(e -> e.age)
                        .reduce(0, Integer::sum);

        System.out.println(ageSum);
    }
}
```

```
import java.util.*;

public class PeekLoggingReduce {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      int sum = 0;

      for (Integer n : list) {
         System.out.println("Peek: " + n);
         sum += n;
      }

      System.out.println(sum);
   }
}
```

16 peek() logging + reduce() Combine values

```
import java.util.*;

public class PeekLoggingReduce {
   public static void main(String[] args) {
      int sum =
            List.of(1, 2, 3).stream()
                  .peek(n -> System.out.println("Peek: " + n))
                  .reduce(0, Integer::sum);

      System.out.println(sum);
   }
}
```

```
import java.util.*;

public class PeekDebugReduce {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);

      int sum = 0;

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug hit");
         }
         sum += n;
      }

      System.out.println(sum);
   }
}
```

17 peek() debugging + reduce() Combine values

```
import java.util.*;

public class PeekDebugReduce {
   public static void main(String[] args) {
      int sum =
            List.of(10, 20, 30).stream()
                  .peek(n -> {
                     if (n == 20) System.out.println("Debug hit");
                  })
                  .reduce(0, Integer::sum);

      System.out.println(sum);
   }
}
```

```
import java.util.*;

public class PeekMetricsReduce {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(5, 10, 15);

      int sum = 0;
      int counter = 0;

      for (Integer n : list) {
         counter++;
         sum += n;
      }

      System.out.println("Sum: " + sum);
      System.out.println("Processed: " + counter);
   }
}
```

18 peek() metrics + reduce() Combine values

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsReduce {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      int sum =
            List.of(5, 10, 15).stream()
                  .peek(n -> counter.incrementAndGet())
                  .reduce(0, Integer::sum);

      System.out.println("Sum: " + sum);
      System.out.println("Processed: " + counter.get());
   }
```

}

---

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + reduce() aggregate (sum, max, concat)

filter() null checks + reduce() aggregate (sum, max, concat)

filter() comparisons + reduce() aggregate (sum, max, concat)

filter() method calls returning boolean + reduce() aggregate (sum, max, concat)

distinct() rely on equals() / hashcode() + reduce() aggregate (sum, max, concat)

distinct() immutable objects + reduce() aggregate (sum, max, concat)

map() transform objects + reduce() aggregate (sum, max, concat)

map() create new objects + reduce() aggregate (sum, max, concat)

map() call pure functions + reduce() aggregate (sum, max, concat)

flatmap() return stream + reduce() aggregate (sum, max, concat)

flatmap() split data + reduce() aggregate (sum, max, concat)

flatmap() expand collections + reduce() aggregate (sum, max, concat)

sorted() natural ordering + reduce() aggregate (sum, max, concat)

sorted() custom comparator + reduce() aggregate (sum, max, concat)

sorted() multi-field compare + reduce() aggregate (sum, max, concat)

peek() logging + reduce() aggregate (sum, max, concat)

peek() debugging + reduce() aggregate (sum, max, concat)

peek() metrics + reduce() aggregate (sum, max, concat)

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where the **terminal operation is reduce()** and it demonstrates **aggregation**:

- **SUM** (numeric aggregation)
- **MAX** (numeric aggregation)
- **CONCAT** (string aggregation)

```
import java.util.*;

public class FilterBooleanReduceAggregate {
   public static void main(String[] args) {

      List<Integer> nums = new ArrayList<Integer>();
      nums.add(10);
      nums.add(15);
      nums.add(20);

      int sum = 0;
      int max = 0;
      String concat = "";

      for (Integer n : nums) {
         if (n % 2 == 0) {
            sum += n;
            if (n > max) {
               max = n;
            }
            concat += String.valueOf(n);
         }
      }

      System.out.println(sum + " | " + max + " | " + concat);
   }
```

```
}
```

1 filter() boolean checks + reduce() Aggregate

```
import java.util.*;

public class FilterBooleanReduceAggregate {
   public static void main(String[] args) {
      List<Integer> nums = List.of(10, 15, 20);

      int sum = nums.stream()
            .filter(n -> n % 2 == 0)
            .reduce(0, Integer::sum);

      int max = nums.stream()
            .filter(n -> n % 2 == 0)
            .reduce(Integer::max)
            .orElse(0);

      String concat = nums.stream()
            .filter(n -> n % 2 == 0)
            .map(String::valueOf)
            .reduce("", String::concat);

      System.out.println(sum + " | " + max + " | " + concat);
   }
}
```

```
import java.util.*;

public class FilterNullReduceAggregate {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("Anurag");
      names.add(null);
      names.add("Amit");

      String concat = "";
      int maxLength = 0;

      for (String name : names) {
         if (name != null) {
            concat += name;
            if (name.length() > maxLength) {
               maxLength = name.length();
            }
         }
      }

      System.out.println(concat + " | " + maxLength);
   }
}
```

2 filter() null checks + reduce() Aggregate

```
import java.util.*;

public class FilterNullReduceAggregate {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      String concat = names.stream()
```

```
            .filter(Objects::nonNull)
            .reduce("", String::concat);

        int maxLength = names.stream()
            .filter(Objects::nonNull)
            .map(String::length)
            .reduce(Integer::max)
            .orElse(0);

        System.out.println(concat + " | " + maxLength);
    }
}
```

```
import java.util.*;

public class FilterComparisonReduceAggregate {
    public static void main(String[] args) {

        List<Integer> salaries = new ArrayList<Integer>();
        salaries.add(20000);
        salaries.add(40000);
        salaries.add(60000);

        int sum = 0;
        int max = 0;

        for (Integer s : salaries) {
            if (s >= 30000) {
                sum += s;
                if (s > max) {
                    max = s;
                }
            }
        }

        System.out.println(sum + " | " + max);
    }
}
```

3 filter() comparisons + reduce() Aggregate

```
import java.util.*;

public class FilterComparisonReduceAggregate {
    public static void main(String[] args) {
        List<Integer> salaries = List.of(20000, 40000, 60000);

        int sum = salaries.stream()
            .filter(s -> s >= 30000)
```

```
            .reduce(0, Integer::sum);

        int max = salaries.stream()
            .filter(s -> s >= 30000)
            .reduce(Integer::max)
            .orElse(0);

        System.out.println(sum + " | " + max);
    }
}
```

```
import java.util.*;

public class FilterMethodReduceAggregate {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {

        List<User> users = new ArrayList<User>();
        users.add(new User("Anurag", true));
        users.add(new User("Amit", true));
        users.add(new User("Ravi", false));

        int count = 0;
        String concat = "";

        for (User u : users) {
            if (u.isActive()) {
                count++;
                concat += u.name;
            }
        }
```

```
        System.out.println(count + " | " + concat);
    }
}
```

4 filter() method call boolean + reduce() Aggregate

```
import java.util.*;

public class FilterMethodReduceAggregate {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() { return active; }
    }

    public static void main(String[] args) {
        List<User> users = List.of(
                new User("Anurag", true),
                new User("Amit", true),
                new User("Ravi", false)
        );

        int count = users.stream()
                .filter(User::isActive)
                .map(u -> 1)
                .reduce(0, Integer::sum);

        String concat = users.stream()
                .filter(User::isActive)
                .map(u -> u.name)
                .reduce("", String::concat);

        System.out.println(count + " | " + concat);
```

```
    }
}
```

```
import java.util.*;

public class DistinctEqualsReduceAggregate {

    static class Person {
        String name;

        Person(String name) {
            this.name = name;
        }

        public boolean equals(Object o) {
            if (o instanceof Person) {
                Person p = (Person) o;
                return name.equals(p.name);
            }
            return false;
        }

        public int hashCode() {
            return name.hashCode();
        }
    }

    public static void main(String[] args) {

        List<Person> people = new ArrayList<Person>();
        people.add(new Person("Anurag"));
        people.add(new Person("Anurag"));
        people.add(new Person("Ravi"));

        Set<Person> unique = new HashSet<Person>(people);

        String concat = "";
        for (Person p : unique) {
```

```
        concat += p.name;
    }

    System.out.println(concat);
  }
}
```

5 distinct() equals/hashCode + reduce() Aggregate

```
import java.util.*;

public class DistinctEqualsReduceAggregate {

  static class Person {
    String name;
    Person(String name) { this.name = name; }

    @Override
    public boolean equals(Object o) {
      return o instanceof Person p && name.equals(p.name);
    }

    @Override
    public int hashCode() {
      return Objects.hash(name);
    }
  }

  public static void main(String[] args) {
    List<Person> people = List.of(
        new Person("Anurag"),
        new Person("Anurag"),
        new Person("Ravi")
    );

    String concat = people.stream()
        .distinct()
        .map(p -> p.name)
        .reduce("", String::concat);

    System.out.println(concat);
  }
}
```

```
import java.util.*;

public class DistinctImmutableReduceAggregate {
   public static void main(String[] args) {

      List<String> langs = new ArrayList<String>();
      langs.add("Java");
      langs.add("Java");
      langs.add("Go");

      Set<String> unique = new HashSet<String>(langs);

      String concat = "";
      int maxLen = 0;

      for (String s : unique) {
         concat += s;
         if (s.length() > maxLen) {
            maxLen = s.length();
         }
      }

      System.out.println(concat + " | " + maxLen);
   }
}
```

6 distinct() immutable objects + reduce() Aggregate

```
import java.util.*;

public class DistinctImmutableReduceAggregate {
   public static void main(String[] args) {
      List<String> langs = List.of("Java", "Java", "Go");

      String concat = langs.stream()
            .distinct()
```

```
            .reduce("", String::concat);

        int maxLen = langs.stream()
            .distinct()
            .map(String::length)
            .reduce(Integer::max)
            .orElse(0);

        System.out.println(concat + " | " + maxLen);
    }
}
```

```
import java.util.*;

public class MapTransformReduceAggregate {
    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("anurag");
        names.add("amit");

        String concat = "";
        int maxLen = 0;

        for (String name : names) {
            String upper = name.toUpperCase();
            concat += upper;

            if (upper.length() > maxLen) {
                maxLen = upper.length();
            }
        }

        System.out.println(concat + " | " + maxLen);
    }
}
```

7 map() transform objects + reduce() Aggregate

```
import java.util.*;

public class MapTransformReduceAggregate {
    public static void main(String[] args) {
        List<String> names = List.of("anurag", "amit");

        String concat = names.stream()
            .map(String::toUpperCase)
            .reduce("", String::concat);
```

```
        int maxLen = names.stream()
                .map(String::toUpperCase)
                .map(String::length)
                .reduce(Integer::max)
                .orElse(0);

        System.out.println(concat + " | " + maxLen);
    }
}
```

```
import java.util.*;

public class MapNewObjectReduceAggregate {

    static class UserDTO {
        String name;
        UserDTO(String name) {
            this.name = name;
        }
    }

    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add("Amit");

        String concat = "";

        for (String name : names) {
            UserDTO dto = new UserDTO(name);
            concat += dto.name;
        }

        System.out.println(concat);
    }
}
```

8 map() create new objects + reduce() Aggregate

```
import java.util.*;

public class MapNewObjectReduceAggregate {

    static class UserDTO {
        String name;
```

```
    UserDTO(String name) { this.name = name; }
  }

  public static void main(String[] args) {
    List<String> names = List.of("Anurag", "Amit");

    String concat = names.stream()
        .map(UserDTO::new)
        .map(dto -> dto.name)
        .reduce("", String::concat);

    System.out.println(concat);
  }
}
```

```
import java.util.*;

public class MapPureFunctionReduceAggregate {

  static int square(int n) {
    return n * n;
  }

  public static void main(String[] args) {

    List<Integer> nums = new ArrayList<Integer>();
    nums.add(2);
    nums.add(3);
    nums.add(4);

    int sum = 0;
    int max = 0;

    for (Integer n : nums) {
      int sq = square(n);
      sum += sq;

      if (sq > max) {
        max = sq;
      }
    }

    System.out.println(sum + " | " + max);
  }
}
```

9 map() pure functions + reduce() Aggregate

```
import java.util.*;

public class MapPureFunctionReduceAggregate {

   static int square(int n) { return n * n; }

   public static void main(String[] args) {
      List<Integer> nums = List.of(2, 3, 4);

      int sum = nums.stream()
            .map(MapPureFunctionReduceAggregate::square)
            .reduce(0, Integer::sum);

      int max = nums.stream()
            .map(MapPureFunctionReduceAggregate::square)
            .reduce(Integer::max)
            .orElse(0);

      System.out.println(sum + " | " + max);
   }
}
```

```
import java.util.*;

public class FlatMapStreamReduceAggregate {
   public static void main(String[] args) {

      List<List<Integer>> lists = new ArrayList<List<Integer>>();

      List<Integer> l1 = new ArrayList<Integer>();
      l1.add(1);
      l1.add(2);

      List<Integer> l2 = new ArrayList<Integer>();
      l2.add(3);
      l2.add(4);

      lists.add(l1);
      lists.add(l2);

      int sum = 0;

      for (List<Integer> list : lists) {
         for (Integer n : list) {
            sum += n;
         }
      }

      System.out.println(sum);
   }
}
```

10 flatMap() return Stream + reduce() Aggregate

```
import java.util.*;

public class FlatMapStreamReduceAggregate {
   public static void main(String[] args) {
```

```
    List<List<Integer>> lists = List.of(
        List.of(1, 2),
        List.of(3, 4)
    );

    int sum = lists.stream()
        .flatMap(List::stream)
        .reduce(0, Integer::sum);

    System.out.println(sum);
  }
}
```

```
import java.util.*;

public class FlatMapSplitReduceAggregate {
  public static void main(String[] args) {

    List<String> lines = new ArrayList<String>();
    lines.add("Java Stream");
    lines.add("Flat Map");

    String concat = "";

    for (String line : lines) {
      String[] words = line.split(" ");
      for (String word : words) {
        concat += word;
      }
    }

    System.out.println(concat);
  }
}
```

11 flatMap() split data + reduce() Aggregate

```
import java.util.*;

public class FlatMapSplitReduceAggregate {
  public static void main(String[] args) {
    List<String> lines = List.of("Java Stream", "Flat Map");

    String concat = lines.stream()
        .flatMap(l -> Arrays.stream(l.split(" ")))
        .reduce("", String::concat);

    System.out.println(concat);
  }
```

```
}
```

```
import java.util.*;

public class FlatMapExpandReduceAggregate {
   public static void main(String[] args) {

      Map<String, List<Integer>> map = new HashMap<String, List<Integer>>();

      List<Integer> listA = new ArrayList<Integer>();
      listA.add(1);
      listA.add(2);

      List<Integer> listB = new ArrayList<Integer>();
      listB.add(3);

      map.put("A", listA);
      map.put("B", listB);

      int sum = 0;

      for (List<Integer> values : map.values()) {
         for (Integer n : values) {
            sum += n;
         }
      }

      System.out.println(sum);
   }
}
```

12 flatMap() expand collections + reduce() Aggregate

```
import java.util.*;

public class FlatMapExpandReduceAggregate {
   public static void main(String[] args) {
      Map<String, List<Integer>> map = Map.of(
```

```
            "A", List.of(1, 2),
            "B", List.of(3)
        );

        int sum = map.values().stream()
            .flatMap(Collection::stream)
            .reduce(0, Integer::sum);

        System.out.println(sum);
    }
}
```

```
import java.util.*;

public class SortedNaturalReduceAggregate {
    public static void main(String[] args) {

        List<Integer> nums = new ArrayList<Integer>();
        nums.add(5);
        nums.add(1);
        nums.add(3);

        Collections.sort(nums);

        int max = 0;

        for (Integer n : nums) {
            if (n > max) {
                max = n;
            }
        }

        System.out.println(max);
    }
}
```

13 sorted() natural ordering + reduce() Aggregate

```
import java.util.*;

public class SortedNaturalReduceAggregate {
    public static void main(String[] args) {
        List<Integer> nums = List.of(5, 1, 3);

        int max = nums.stream()
            .sorted()
            .reduce(Integer::max)
            .orElse(0);
```

```
        System.out.println(max);
    }
}
```

```
import java.util.*;

public class SortedCustomReduceAggregate {
    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add("Ravi");
        names.add("Amit");

        Collections.sort(names, new Comparator<String>() {
            public int compare(String s1, String s2) {
                return s2.compareTo(s1);                    // reverse order
            }
        });

        String concat = "";

        for (String name : names) {
            concat += name;
        }

        System.out.println(concat);
    }
}
```

14 sorted() custom comparator + reduce() Aggregate

```
import java.util.*;

public class SortedCustomReduceAggregate {
    public static void main(String[] args) {
        List<String> names = List.of("Anurag", "Ravi", "Amit");

        String concat = names.stream()
```

```
            .sorted(Comparator.reverseOrder())
            .reduce("", String::concat);

        System.out.println(concat);
    }
}
```

```
import java.util.*;

public class SortedMultiFieldReduceAggregate {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {

        List<Employee> emps = new ArrayList<Employee>();
        emps.add(new Employee("Anurag", 30));
        emps.add(new Employee("Anurag", 25));
        emps.add(new Employee("Ravi", 35));

        Collections.sort(emps, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0) {
                    return nameCompare;
                }
                return e1.age - e2.age;
            }
        });

        int maxAge = 0;
        for (Employee e : emps) {
            if (e.age > maxAge) {
                maxAge = e.age;
```

```
        }
    }

    System.out.println(maxAge);
  }
}
```

15 sorted() multi-field compare + reduce() Aggregate

```
import java.util.*;

public class SortedMultiFieldReduceAggregate {

  static class Employee {
    String name;
    int age;
    Employee(String name, int age) {
      this.name = name; this.age = age;
    }
  }

  public static void main(String[] args) {
    List<Employee> emps = List.of(
        new Employee("Anurag", 30),
        new Employee("Anurag", 25),
        new Employee("Ravi", 35)
    );

    int maxAge = emps.stream()
        .sorted(Comparator
            .comparing((Employee e) -> e.name)
            .thenComparingInt(e -> e.age))
        .map(e -> e.age)
        .reduce(Integer::max)
        .orElse(0);

    System.out.println(maxAge);
  }
}
```

```
import java.util.*;

public class PeekLoggingReduceAggregate {
   public static void main(String[] args) {

      List<Integer> nums = new ArrayList<Integer>();
      nums.add(1);
      nums.add(2);
      nums.add(3);

      int sum = 0;

      for (Integer n : nums) {
         System.out.println("Peek: " + n);
         sum += n;
      }

      System.out.println(sum);
   }
}
```

16 peek() logging + reduce() Aggregate

```
import java.util.*;

public class PeekLoggingReduceAggregate {
   public static void main(String[] args) {
      int sum = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Peek: " + n))
            .reduce(0, Integer::sum);

      System.out.println(sum);
   }
}
```

```
import java.util.*;

public class PeekDebugReduceAggregate {
   public static void main(String[] args) {

      List<Integer> nums = new ArrayList<Integer>();
      nums.add(10);
      nums.add(20);
      nums.add(30);

      int max = 0;

      for (Integer n : nums) {
         if (n == 20) {
            System.out.println("Debug hit");
         }

         if (n > max) {
            max = n;
         }
      }

      System.out.println(max);
   }
}
```

17 peek() debugging + reduce() Aggregate

```
import java.util.*;

public class PeekDebugReduceAggregate {
   public static void main(String[] args) {
      int max = List.of(10, 20, 30).stream()
            .peek(n -> {
               if (n == 20) System.out.println("Debug hit");
            })
```

```
            .reduce(Integer::max)
            .orElse(0);

        System.out.println(max);
    }
}
```

```
import java.util.*;

public class PeekMetricsReduceAggregate {
    public static void main(String[] args) {

        List<Integer> nums = new ArrayList<Integer>();
        nums.add(5);
        nums.add(10);
        nums.add(15);

        int counter = 0;
        int sum = 0;

        for (Integer n : nums) {
            counter++;                                          // metric tracking
            sum += n;
        }

        System.out.println("Sum: " + sum);
        System.out.println("Processed: " + counter);
    }
}
```

18 peek() metrics + reduce() Aggregate

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsReduceAggregate {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        int sum = List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .reduce(0, Integer::sum);
```

```
        System.out.println("Sum: " + sum);
        System.out.println("Processed: " + counter.get());
    }
}
```

---

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + count() no custom logic allowed

filter() null checks + count() no custom logic allowed

filter() comparisons + count() no custom logic allowed

filter() method calls returning boolean + count() no custom logic allowed

distinct() rely on equals() / hashcode() + count() no custom logic allowed

distinct() immutable objects + count() no custom logic allowed

map() transform objects + count() no custom logic allowed

map() create new objects + count() no custom logic allowed

map() call pure functions + count() no custom logic allowed

flatmap() return stream + count() no custom logic allowed

flatmap() split data + count() no custom logic allowed

flatmap() expand collections + count() no custom logic allowed

sorted() natural ordering + count() no custom logic allowed

sorted() custom comparator + count() no custom logic allowed

sorted() multi-field compare + count() no custom logic allowed

peek() logging + count() no custom logic allowed

peek() debugging + count() no custom logic allowed

peek() metrics + count() no custom logic allowed

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = count()**
- **No custom aggregation / reduce / collect logic**
- Only **built-in Stream operations**
- Java **8+ compatible**
- One clear intent per class

✅ "No custom logic allowed" respected:
count() is the **only terminal**, no reducers, no collectors, no mutation.

```
import java.util.*;

public class FilterBooleanCount {
   public static void main(String[] args) {
      List<Integer> numbers = new ArrayList<Integer>();
      numbers.add(10);
      numbers.add(15);
      numbers.add(20);
      numbers.add(25);

      long count = 0;

      for (Integer n : numbers) {
         if (n % 2 == 0) {
            count++;
         }
      }
      System.out.println(count);
   }
```

```
}
```

1 filter() boolean checks + count()

```
import java.util.*;

public class FilterBooleanCount {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      long count = numbers.stream()
            .filter(n -> n % 2 == 0)
            .count();

      System.out.println(count);
   }
}
```

```
import java.util.*;

public class FilterNullCount {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      long count = 0;

      for (String name : names) {
         if (name != null) {
            count++;
         }
      }

      System.out.println(count);
   }
}
```

2 filter() null checks + count()

```
import java.util.*;

public class FilterNullCount {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      long count = names.stream()
            .filter(Objects::nonNull)
            .count();

      System.out.println(count);
   }
}
```

```
import java.util.*;

public class FilterComparisonCount {
   public static void main(String[] args) {
      List<Integer> salaries = new ArrayList<Integer>();
      salaries.add(20000);
      salaries.add(40000);
      salaries.add(60000);

      long count = 0;

      for (Integer s : salaries) {
         if (s >= 30000) {
            count++;
         }
      }

      System.out.println(count);
   }
}
```

3 filter() comparisons + count()

```
import java.util.*;

public class FilterComparisonCount {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      long count = salaries.stream()
            .filter(s -> s >= 30000)
            .count();

      System.out.println(count);
   }
```

}

```
import java.util.*;

public class FilterMethodCount {

    static class User {
        boolean active;
        User(boolean active) { this.active = active; }
        boolean isActive() { return active; }
    }

    public static void main(String[] args) {
        List<User> users = new ArrayList<User>();
        users.add(new User(true));
        users.add(new User(false));
        users.add(new User(true));

        long count = 0;

        for (User user : users) {
            if (user.isActive()) {
                count++;
            }
        }

        System.out.println(count);
    }
}
```

4 filter() method call returning boolean + count()

```
import java.util.*;

public class FilterMethodCount {

    static class User {
        boolean active;
        User(boolean active) { this.active = active; }
        boolean isActive() { return active; }
    }

    public static void main(String[] args) {
        List<User> users = List.of(
                new User(true),
                new User(false),
                new User(true)
        );

        long count = users.stream()
                .filter(User::isActive)
                .count();

        System.out.println(count);
    }
}
```

```
import java.util.*;

public class DistinctEqualsCount {

   static class Person {
      String name;

      Person(String name) { this.name = name; }

      public boolean equals(Object o) {
         if (this == o) return true;
         if (!(o instanceof Person)) return false;

         Person p = (Person) o;
         return name.equals(p.name);
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {
      List<Person> people = new ArrayList<Person>();
      people.add(new Person("Anurag"));
      people.add(new Person("Anurag"));
      people.add(new Person("Ravi"));

      Set<Person> unique = new HashSet<Person>(people);

      System.out.println(unique.size());
   }
}
```

5 distinct() rely on equals() / hashCode() + count()

```
import java.util.*;

public class DistinctEqualsCount {

    static class Person {
        String name;
        Person(String name) { this.name = name; }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        long count = people.stream()
                .distinct()
                .count();

        System.out.println(count);
    }
}
```

```
import java.util.*;

public class DistinctImmutableCount {
   public static void main(String[] args) {
      List<String> values = new ArrayList<String>();
      values.add("Java");
      values.add("Java");
      values.add("Python");

      Set<String> unique = new HashSet<String>(values);
      System.out.println(unique.size());
   }
}
```

6 distinct() immutable objects + count()

```
import java.util.*;

public class DistinctImmutableCount {
   public static void main(String[] args) {
      List<String> values = List.of("Java", "Java", "Python");

      long count = values.stream()
            .distinct()
            .count();

      System.out.println(count);
   }
}
```

```
import java.util.*;

public class MapTransformCount {
   public static void main(String[] args) {
      List<String> names = new ArrayList<String>();
      names.add("anurag");
      names.add("amit");

      long count = 0;

      for (String name : names) {
         String upper = name.toUpperCase();
         count++;
      }

      System.out.println(count);
```

```
    }
}
```

7 map() transform objects + count()

```
import java.util.*;

public class MapTransformCount {
    public static void main(String[] args) {
        List<String> names = List.of("anurag", "amit");

        long count = names.stream()
                .map(String::toUpperCase)
                .count();

        System.out.println(count);
    }
}
```

```
import java.util.*;

public class MapCreateObjectCount {

    static class UserDTO {
        String name;
        UserDTO(String name) { this.name = name; }
    }

    public static void main(String[] args) {
        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add("Amit");

        long count = 0;

        for (String name : names) {
```

```
        UserDTO dto = new UserDTO(name);
        count++;
    }

    System.out.println(count);
  }
}
```

8 map() create new objects + count()

```
import java.util.*;

public class MapCreateObjectCount {

  static class UserDTO {
    String name;
    UserDTO(String name) { this.name = name; }
  }

  public static void main(String[] args) {
    List<String> names = List.of("Anurag", "Amit");

    long count = names.stream()
        .map(UserDTO::new)
        .count();

    System.out.println(count);
  }
}
```

```
import java.util.*;

public class MapPureFunctionCount {

  static int square(int n) { return n * n; }

  public static void main(String[] args) {
    List<Integer> numbers = new ArrayList<Integer>();
    numbers.add(2);
    numbers.add(3);
    numbers.add(4);
```

```
        long count = 0;

        for (Integer n : numbers) {
            int result = square(n);
            count++;
        }

        System.out.println(count);
    }
}
```

9 map() call pure functions + count()

```
import java.util.*;

public class MapPureFunctionCount {

    static int square(int n) { return n * n; }

    public static void main(String[] args) {
        List<Integer> numbers = List.of(2, 3, 4);

        long count = numbers.stream()
                .map(MapPureFunctionCount::square)
                .count();

        System.out.println(count);
    }
}
```

```
import java.util.*;

public class FlatMapStreamCount {
    public static void main(String[] args) {

        List<List<Integer>> lists = new ArrayList<List<Integer>>();

        lists.add(Arrays.asList(1, 2));
        lists.add(Arrays.asList(3, 4));

        long count = 0;

        for (List<Integer> list : lists) {
            for (Integer value : list) {
```

```
            count++;
        }
    }

    System.out.println(count);
  }
}
```

10 flatMap() return Stream + count()

```
import java.util.*;

public class FlatMapStreamCount {
  public static void main(String[] args) {
    List<List<Integer>> lists = List.of(
          List.of(1, 2),
          List.of(3, 4)
    );

    long count = lists.stream()
          .flatMap(List::stream)
          .count();

    System.out.println(count);
  }
}
```

```
import java.util.*;

public class FlatMapSplitCount {
  public static void main(String[] args) {

    List<String> lines = new ArrayList<String>();
    lines.add("Java Stream");
    lines.add("Flat Map");

    long count = 0;

    for (String line : lines) {
      String[] words = line.split(" ");
      for (int i = 0; i < words.length; i++) {
```

```
            count++;
        }
    }

    System.out.println(count);
  }
}
```

11 flatMap() split data + count()

```
import java.util.*;

public class FlatMapSplitCount {
  public static void main(String[] args) {
    List<String> lines = List.of("Java Stream", "Flat Map");

    long count = lines.stream()
          .flatMap(s -> Arrays.stream(s.split(" ")))
          .count();

    System.out.println(count);
  }
}
```

```
import java.util.*;

public class FlatMapExpandCount {
  public static void main(String[] args) {

    Map<String, List<String>> data = new HashMap<String, List<String>>();
    data.put("A", Arrays.asList("1", "2"));
    data.put("B", Arrays.asList("3"));

    long count = 0;

    for (List<String> list : data.values()) {
      for (String value : list) {
          count++;
      }
    }
```

```
        System.out.println(count);
    }
}
```

12 flatMap() expand collections + count()

```
import java.util.*;

public class FlatMapExpandCount {
    public static void main(String[] args) {
        Map<String, List<String>> data = Map.of(
            "A", List.of("1", "2"),
            "B", List.of("3")
        );

        long count = data.values().stream()
            .flatMap(Collection::stream)
            .count();

        System.out.println(count);
    }
}
```

```
import java.util.*;

public class SortedNaturalCount {
    public static void main(String[] args) {

        List<Integer> numbers = new ArrayList<Integer>();
        numbers.add(5);
        numbers.add(1);
        numbers.add(3);

        Collections.sort(numbers);

        System.out.println(numbers.size());
    }
```

```
}
```

13 sorted() natural ordering + count()

```
import java.util.*;

public class SortedNaturalCount {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(5, 1, 3);

      long count = numbers.stream()
            .sorted()
            .count();
      System.out.println(count);
   }
}
```

```
import java.util.*;

public class SortedCustomCount {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("Anurag");
      names.add("Ravi");
      names.add("Amit");

      Collections.sort(names, Collections.reverseOrder());

      System.out.println(names.size());
   }
}
```

14 sorted() custom comparator + count()

```
import java.util.*;

public class SortedCustomCount {
   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Ravi", "Amit");

      long count = names.stream()
            .sorted(Comparator.reverseOrder())
            .count();

      System.out.println(count);
   }
}
```

```
import java.util.*;

public class SortedMultiFieldCount {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {

        List<Employee> emps = new ArrayList<Employee>();
        emps.add(new Employee("Anurag", 30));
        emps.add(new Employee("Anurag", 25));
        emps.add(new Employee("Ravi", 35));

        Collections.sort(emps, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0) {
                    return nameCompare;
                }
                return e1.age - e2.age;
            }
        });

        System.out.println(emps.size());
    }
}
```

15 sorted() multi-field compare + count()

```
import java.util.*;

public class SortedMultiFieldCount {

    static class Employee {
        String name;
        int age;
        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {
        List<Employee> emps = List.of(
                new Employee("Anurag", 30),
                new Employee("Anurag", 25),
                new Employee("Ravi", 35)
        );

        long count = emps.stream()
                .sorted(Comparator
                        .comparing((Employee e) -> e.name)
                        .thenComparingInt(e -> e.age))
                .count();

        System.out.println(count);
    }
}
```

```
import java.util.*;
```

```
public class PeekLoggingCount {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      long count = 0;

      for (Integer n : list) {
         System.out.println("Peek: " + n);
         count++;
      }

      System.out.println(count);
   }
}
```

16 peek() logging + count()

```
import java.util.*;

public class PeekLoggingCount {
   public static void main(String[] args) {
      long count = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Peek: " + n))
            .count();

      System.out.println(count);
   }
}
```

```
import java.util.*;

public class PeekDebugCount {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);

      long count = 0;

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug hit");
         }
         count++;
      }

      System.out.println(count);
   }
```

```
}
```

17 peek() debugging + count()

```
import java.util.*;

public class PeekDebugCount {
   public static void main(String[] args) {
      long count = List.of(10, 20, 30).stream()
            .peek(n -> {
               if (n == 20) {
                  System.out.println("Debug hit");
               }
            })
            .count();

      System.out.println(count);
   }
}
```

```
import java.util.*;

public class PeekMetricsCount {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(5, 10, 15);

      int metricCounter = 0;
      long count = 0;

      for (Integer n : list) {
         metricCounter++;
         count++;
      }
```

```
        System.out.println("Stream count: " + count);
        System.out.println("Metric count: " + metricCounter);
    }
}
```

18 peek() metrics + count()

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsCount {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        long count = List.of(5, 10, 15).stream()
                .peek(n -> counter.incrementAndGet())
                .count();

        System.out.println("Stream count: " + count);
        System.out.println("Metric count: " + counter.get());
    }
}
```

---

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + findfirst() conditional check before first match

filter() null checks + findfirst() conditional check before first match

filter() comparisons + findfirst() conditional check before first match

filter() method calls returning boolean + findfirst() conditional check before first match

distinct() rely on equals() / hashcode() + findfirst() conditional check before first match

distinct() immutable objects + findfirst() conditional check before first match

map() transform objects + findfirst() conditional check before first match

map() create new objects + findfirst() conditional check before first match

map() call pure functions + findfirst() conditional check before first match

flatmap() return stream + findfirst() conditional check before first match

flatmap() split data + findfirst() conditional check before first match

flatmap() expand collections + findfirst() conditional check before first match

sorted() natural ordering + findfirst() conditional check before first match

sorted() custom comparator + findfirst() conditional check before first match

sorted() multi-field compare + findfirst() conditional check before first match

peek() logging + findfirst() conditional check before first match

peek() debugging + findfirst() conditional check before first match

peek() metrics + findfirst() conditional check before first match

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = findFirst()**
- Purpose = **conditional check before the first matching element**
- Demonstrates **short-circuiting** (stream stops after first match)
- Java **8+ compatible**
- No collectors, no reduce, no extra logic

 findFirst() returns Optional<T> and respects **encounter order**.

```
import java.util.*;

public class FilterBooleanFindFirst {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(11, 14, 18);
      Integer firstEven = null;

      for (Integer n : list) {
         if (n % 2 == 0) {
            firstEven = n;
            break;
         }
      }

      System.out.println(firstEven != null ? firstEven : -1);
   }
}
```

1 filter() boolean checks + findFirst()

```
import java.util.*;

public class FilterBooleanFindFirst {
   public static void main(String[] args) {
      Optional<Integer> firstEven =
            List.of(11, 14, 18).stream()
                  .filter(n -> n % 2 == 0)
                  .findFirst();

      System.out.println(firstEven.orElse(-1));
   }
}
```

```
import java.util.*;

public class FilterNullFindFirst {
   public static void main(String[] args) {

      List<String> list = Arrays.asList(null, "Anurag", "Amit");
      String firstNonNull = null;

      for (String s : list) {
         if (s != null) {
            firstNonNull = s;
            break;
         }
      }

      System.out.println(firstNonNull != null ? firstNonNull : "NONE");
   }
}
```

2 filter() null checks + findFirst()

```
import java.util.*;

public class FilterNullFindFirst {
   public static void main(String[] args) {
      Optional<String> firstNonNull =
            Arrays.asList(null, "Anurag", "Amit").stream()
                  .filter(Objects::nonNull)
                  .findFirst();

      System.out.println(firstNonNull.orElse("NONE"));
   }
}
```

```
import java.util.*;

public class FilterComparisonFindFirst {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(20000, 45000, 60000);
      Integer firstHigh = null;

      for (Integer s : list) {
         if (s >= 40000) {
            firstHigh = s;
            break;
         }
      }

      System.out.println(firstHigh != null ? firstHigh : 0);
   }
}
```

3 filter() comparisons + findFirst()

```
import java.util.*;

public class FilterComparisonFindFirst {
   public static void main(String[] args) {
      Optional<Integer> firstHighSalary =
            List.of(20000, 45000, 60000).stream()
                  .filter(s -> s >= 40000)
                  .findFirst();

      System.out.println(firstHighSalary.orElse(0));
   }
}
```

```
import java.util.*;

public class FilterMethodFindFirst {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {

        List<User> list = Arrays.asList(
                new User("A", false),
                new User("B", true),
                new User("C", true)
        );

        User firstActive = null;

        for (User u : list) {
            if (u.isActive()) {
                firstActive = u;
                break;
            }
        }
```

```
        System.out.println(firstActive != null ? firstActive.name : "NONE");
    }
}
```

4 filter() method call returning boolean + findFirst()

```
import java.util.*;

public class FilterMethodFindFirst {

    static class User {
        String name;
        boolean active;
        User(String name, boolean active) {
            this.name = name; this.active = active;
        }
        boolean isActive() { return active; }
    }

    public static void main(String[] args) {
        Optional<User> firstActive =
                List.of(
                        new User("A", false),
                        new User("B", true),
                        new User("C", true)
                ).stream()
                 .filter(User::isActive)
                 .findFirst();

        System.out.println(firstActive.map(u -> u.name).orElse("NONE"));
    }
}
```

```
import java.util.*;

public class DistinctEqualsFindFirst {

    static class Person {
        String name;

        Person(String name) {
            this.name = name;
        }

        public boolean equals(Object o) {
            if (o instanceof Person) {
                Person p = (Person) o;
                return name.equals(p.name);
            }
            return false;
        }

        public int hashCode() {
            return name.hashCode();
        }
    }

    public static void main(String[] args) {

        List<Person> list = Arrays.asList(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        Set<Person> set = new LinkedHashSet<Person>(list);
```

```
        Person first = null;
        for (Person p : set) {
            first = p;
            break;
        }

        System.out.println(first != null ? first.name : "NONE");
    }
}
```

5 distinct() rely on equals() / hashCode() + findFirst()

```
import java.util.*;

public class DistinctEqualsFindFirst {

    static class Person {
        String name;
        Person(String name) { this.name = name; }
        @Override public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }
        @Override public int hashCode() { return Objects.hash(name); }
    }

    public static void main(String[] args) {
        Optional<Person> firstUnique =
                List.of(
                        new Person("Anurag"),
                        new Person("Anurag"),
                        new Person("Ravi")
                ).stream()
                 .distinct()
                 .findFirst();

        System.out.println(firstUnique.map(p -> p.name).orElse("NONE"));
    }
}
```

```
import java.util.*;

public class DistinctImmutableFindFirst {
    public static void main(String[] args) {
```

```
        List<String> list = Arrays.asList("Java", "Java", "Go");
        Set<String> set = new LinkedHashSet<String>(list);

        String first = null;
        for (String s : set) {
            first = s;
            break;
        }

        System.out.println(first != null ? first : "NONE");
    }
}
```

6 distinct() immutable objects + findFirst()

```
import java.util.*;

public class DistinctImmutableFindFirst {
    public static void main(String[] args) {
        Optional<String> firstUnique =
                List.of("Java", "Java", "Go").stream()
                        .distinct()
                        .findFirst();

        System.out.println(firstUnique.orElse("NONE"));
    }
}
```

```
import java.util.*;

public class MapTransformFindFirst {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("anurag", "amit");
        String firstUpper = null;

        for (String s : list) {
            firstUpper = s.toUpperCase();
            break;
        }

        System.out.println(firstUpper != null ? firstUpper : "NONE");
    }
}
```

7 map() transform objects + findFirst()

```
import java.util.*;

public class MapTransformFindFirst {
   public static void main(String[] args) {
      Optional<String> firstUpper =
            List.of("anurag", "amit").stream()
                  .map(String::toUpperCase)
                  .findFirst();

      System.out.println(firstUpper.orElse("NONE"));
   }
}
```

```
import java.util.*;

public class MapCreateObjectFindFirst {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");
      UserDTO first = null;

      for (String s : list) {
         first = new UserDTO(s);
         break;
      }

      System.out.println(first != null ? first.name : "NONE");
   }
}
```

8 map() create new objects + findFirst()

```
import java.util.*;

public class MapCreateObjectFindFirst {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
```

```
    }

    public static void main(String[] args) {
        Optional<UserDTO> firstDTO =
                List.of("Anurag", "Amit").stream()
                        .map(UserDTO::new)
                        .findFirst();

        System.out.println(firstDTO.map(d -> d.name).orElse("NONE"));
    }
}
```

```
import java.util.*;

public class MapPureFunctionFindFirst {

    static int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(2, 3, 4);
        Integer first = null;

        for (Integer n : list) {
            first = square(n);
            break;
        }

        System.out.println(first != null ? first : 0);
    }
}
```

9 map() call pure functions + findFirst()

```
import java.util.*;

public class MapPureFunctionFindFirst {

    static int square(int n) { return n * n; }

    public static void main(String[] args) {
        Optional<Integer> firstSquare =
                List.of(2, 3, 4).stream()
                        .map(MapPureFunctionFindFirst::square)
                        .findFirst();
```

```
        System.out.println(firstSquare.orElse(0));
    }
}
```

```
import java.util.*;

public class FlatMapStreamFindFirst {
    public static void main(String[] args) {

        List<List<Integer>> list = new ArrayList<List<Integer>>();
        list.add(Arrays.asList(10, 20));
        list.add(Arrays.asList(30));

        Integer first = null;

        for (List<Integer> inner : list) {
            for (Integer n : inner) {
                first = n;
                break;
            }
            if (first != null) break;
        }

        System.out.println(first != null ? first : 0);
    }
}
```

10 flatMap() return Stream + findFirst()

```
import java.util.*;

public class FlatMapStreamFindFirst {
    public static void main(String[] args) {
        Optional<Integer> first =
                List.of(
                        List.of(10, 20),
                        List.of(30)
                ).stream()
                 .flatMap(List::stream)
```

```
            .findFirst();

        System.out.println(first.orElse(0));
    }
}
```

```
import java.util.*;

public class FlatMapSplitFindFirst {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Java Stream", "Flat Map");
        String firstWord = null;

        for (String s : list) {
            String[] words = s.split(" ");
            for (int i = 0; i < words.length; i++) {
                firstWord = words[i];
                break;
            }
            if (firstWord != null) break;
        }

        System.out.println(firstWord != null ? firstWord : "NONE");
    }
}
```

11 flatMap() split data + findFirst()

```
import java.util.*;

public class FlatMapSplitFindFirst {
    public static void main(String[] args) {
        Optional<String> firstWord =
            List.of("Java Stream", "Flat Map").stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
                .findFirst();

        System.out.println(firstWord.orElse("NONE"));
    }
}
```

```
import java.util.*;

public class FlatMapExpandFindFirst {
   public static void main(String[] args) {

      Map<String, List<String>> map = new HashMap<String, List<String>>();
      map.put("A", Arrays.asList("1", "2"));
      map.put("B", Arrays.asList("3"));

      String first = null;

      for (List<String> values : map.values()) {
         for (String v : values) {
            first = v;
            break;
         }
         if (first != null) break;
      }

      System.out.println(first != null ? first : "NONE");
   }
}
```

12 flatMap() expand collections + findFirst()

```
import java.util.*;

public class FlatMapExpandFindFirst {
   public static void main(String[] args) {
      Optional<String> firstValue =
            Map.of(
                  "A", List.of("1", "2"),
                  "B", List.of("3")
            ).values().stream()
             .flatMap(Collection::stream)
```

```
            .findFirst();

        System.out.println(firstValue.orElse("NONE"));
    }
}
```

```
import java.util.*;

public class SortedNaturalFindFirst {
    public static void main(String[] args) {

        List<Integer> list = new ArrayList<Integer>(Arrays.asList(5, 1, 3));
        Collections.sort(list);

        Integer first = list.size() > 0 ? list.get(0) : null;

        System.out.println(first != null ? first : 0);
    }
}
```

13 sorted() natural ordering + findFirst()

```
import java.util.*;

public class SortedNaturalFindFirst {
    public static void main(String[] args) {
        Optional<Integer> smallest =
            List.of(5, 1, 3).stream()
                .sorted()
                .findFirst();

        System.out.println(smallest.orElse(0));
    }
}
```

```
import java.util.*;

public class SortedCustomFindFirst {
```

```
    public static void main(String[] args) {

        List<String> list = new ArrayList<String>(Arrays.asList("Anurag", "Ravi", "Amit"));

        Collections.sort(list, new Comparator<String>() {
            public int compare(String a, String b) {
                return b.compareTo(a);
            }
        });

        String first = list.size() > 0 ? list.get(0) : null;

        System.out.println(first != null ? first : "NONE");
    }
}
```

14 sorted() custom comparator + findFirst()

```
import java.util.*;

public class SortedCustomFindFirst {
    public static void main(String[] args) {
        Optional<String> firstDesc =
                List.of("Anurag", "Ravi", "Amit").stream()
                        .sorted(Comparator.reverseOrder())
                        .findFirst();

        System.out.println(firstDesc.orElse("NONE"));
    }
}
```

```
import java.util.*;

public class SortedMultiFieldFindFirst {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {

        List<Employee> list = new ArrayList<Employee>();
```

```
        list.add(new Employee("Anurag", 30));
        list.add(new Employee("Anurag", 25));
        list.add(new Employee("Ravi", 35));

        Collections.sort(list, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0) return nameCompare;
                return e1.age - e2.age;
            }
        });

        Employee first = list.size() > 0 ? list.get(0) : null;
        System.out.println(first != null ? first.name + " " + first.age : "NONE");
    }
}
```

15 sorted() multi-field compare + findFirst()

```
import java.util.*;

public class SortedMultiFieldFindFirst {

    static class Employee {
        String name;
        int age;
        Employee(String name, int age) {
            this.name = name; this.age = age;
        }
    }

    public static void main(String[] args) {
        Optional<Employee> first =
                List.of(
                        new Employee("Anurag", 30),
                        new Employee("Anurag", 25),
                        new Employee("Ravi", 35)
                ).stream()
                 .sorted(Comparator
                         .comparing((Employee e) -> e.name)
                         .thenComparingInt(e -> e.age))
                 .findFirst();

        System.out.println(first.map(e -> e.name + " " + e.age).orElse("NONE"));
    }
}
```

```
import java.util.*;
```

```
public class PeekLoggingFindFirst {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);
      Integer first = null;

      for (Integer n : list) {
         System.out.println("Peek: " + n);
         first = n;
         break;
      }

      System.out.println(first != null ? first : 0);
   }
}
```

16 peek() logging + findFirst()

```
import java.util.*;

public class PeekLoggingFindFirst {
   public static void main(String[] args) {
      Optional<Integer> first =
            List.of(1, 2, 3).stream()
                  .peek(n -> System.out.println("Peek: " + n))
                  .findFirst();

      System.out.println(first.orElse(0));
   }
}
```

```
import java.util.*;

public class PeekDebugFindFirst {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);
      Integer first = null;

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug hit");
         }
         first = n;
         break;
      }

      System.out.println(first != null ? first : 0);
```

```
    }
}
```

17 peek() debugging + findFirst()

```
import java.util.*;

public class PeekDebugFindFirst {
    public static void main(String[] args) {
        Optional<Integer> first =
            List.of(10, 20, 30).stream()
                .peek(n -> {
                    if (n == 20) System.out.println("Debug hit");
                })
                .findFirst();

        System.out.println(first.orElse(0));
    }
}
```

```
import java.util.*;

public class PeekMetricsFindFirst {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(5, 10, 15);
        int counter = 0;
        Integer first = null;

        for (Integer n : list) {
            counter++;
            first = n;
            break;
        }

        System.out.println("First: " + (first != null ? first : 0));
```

```
        System.out.println("Processed elements: " + counter);
    }
}
```

18 peek() metrics + findFirst()

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsFindFirst {
    public static void main(String[] args) {
        AtomicInteger counter = new AtomicInteger();

        Optional<Integer> first =
            List.of(5, 10, 15).stream()
                .peek(n -> counter.incrementAndGet())
                .findFirst();

        System.out.println("First: " + first.orElse(0));
        System.out.println("Processed elements: " + counter.get());
    }
}
```

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + allmatch() validation logic

filter() null checks + allmatch() validation logic

filter() comparisons + allmatch() validation logic

filter() method calls returning boolean + allmatch() validation logic

distinct() rely on equals() / hashcode() + allmatch() validation logic

distinct() immutable objects + allmatch() validation logic

map() transform objects + allmatch() validation logic

map() create new objects + allmatch() validation logic

map() call pure functions + allmatch() validation logic

flatmap() return stream + allmatch() validation logic

flatmap() split data + allmatch() validation logic

flatmap() expand collections + allmatch() validation logic

sorted() natural ordering + allmatch() validation logic

sorted() custom comparator + allmatch() validation logic

sorted() multi-field compare + allmatch() validation logic

peek() logging + allmatch() validation logic

peek() debugging + allmatch() validation logic

peek() metrics + allmatch() validation logic

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = allMatch()**
- Purpose = **validation logic** (returns true only if **all elements** satisfy the predicate)
- Demonstrates **short-circuiting** (stops on first failure)
- Java **8+** compatible
- No collectors / reduce / custom aggregation

```
import java.util.*;

public class FilterBooleanAllMatch {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(2, 4, 6, 8);

      boolean valid = true;

      for (Integer n : list) {
         if (n > 0) {                                    // filter
            if (n % 2 != 0) {                            // allMatch
               valid = false;
               break;
            }
         }
      }

      System.out.println(valid);
   }
}
```

1 filter() boolean checks + allMatch() (Validation)

```
import java.util.*;

public class FilterBooleanAllMatch {
   public static void main(String[] args) {
      boolean valid = List.of(2, 4, 6, 8).stream()
            .filter(n -> n > 0)
            .allMatch(n -> n % 2 == 0);

      System.out.println(valid);
   }
}
```

```
import java.util.*;

public class FilterNullAllMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");

      boolean valid = true;

      for (String s : list) {
         if (s != null) {                                    // filter non-null
            if (s.length() == 0) {                           // allMatch
               valid = false;
               break;
            }
         }
      }

      System.out.println(valid);
   }
}
```

2 filter() null checks + allMatch() (Validation)

```
import java.util.*;

public class FilterNullAllMatch {
   public static void main(String[] args) {
      boolean valid = Arrays.asList("Anurag", "Amit").stream()
            .filter(Objects::nonNull)
            .allMatch(s -> !s.isEmpty());
```

```
        System.out.println(valid);
    }
}
```

```
import java.util.*;

public class FilterComparisonAllMatch {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(30, 40, 50);

        boolean valid = true;

        for (Integer age : list) {
            if (age >= 18) {                                    // filter
                if (age > 60) {                                 // allMatch
                    valid = false;
                    break;
                }
            }
        }

        System.out.println(valid);
    }
}
```

3 filter() comparisons + allMatch() (Validation)

```
import java.util.*;

public class FilterComparisonAllMatch {
    public static void main(String[] args) {
        boolean valid = List.of(30, 40, 50).stream()
                .filter(age -> age >= 18)
                .allMatch(age -> age <= 60);
```

```
        System.out.println(valid);
    }
}
```

```
import java.util.*;

public class FilterMethodAllMatch {

    static class User {
        boolean active;
        User(boolean active) { this.active = active; }
        boolean isActive() { return active; }
    }

    public static void main(String[] args) {

        List<User> list = Arrays.asList(
                new User(true),
                new User(true)
        );

        boolean valid = true;

        for (User u : list) {
            if (u.isActive()) {                                    // filter
                if (!u.isActive()) {                               // allMatch
                    valid = false;
                    break;
                }
            }
        }

        System.out.println(valid);
    }
}
```

4 filter() method calls returning boolean + allMatch() (Validation)

```
import java.util.*;

public class FilterMethodAllMatch {

   static class User {
      boolean active;
      User(boolean active) { this.active = active; }
      boolean isActive() { return active; }
   }

   public static void main(String[] args) {
      boolean valid = List.of(
            new User(true),
            new User(true)
      ).stream()
       .filter(User::isActive)
       .allMatch(User::isActive);

      System.out.println(valid);
   }
}
```

```
import java.util.*;

public class DistinctEqualsAllMatch {

   static class Person {
      String name;
      Person(String name) { this.name = name; }

      public boolean equals(Object o) {
         if (!(o instanceof Person)) return false;
         Person p = (Person) o;
         return name.equals(p.name);
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> list = Arrays.asList(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
      );

      Set<Person> distinct = new HashSet<Person>(list);

      boolean valid = true;
```

```
        for (Person p : distinct) {
            if (p.name.length() < 4) {
                valid = false;
                break;
            }
        }

        System.out.println(valid);
    }
}
```

5 distinct() rely on equals() / hashCode() + allMatch() (Validation)

```
import java.util.*;

public class DistinctEqualsAllMatch {

    static class Person {
        String name;
        Person(String name) { this.name = name; }

        @Override public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }
        @Override public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        boolean valid = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        ).stream()
         .distinct()
         .allMatch(p -> p.name.length() >= 4);

        System.out.println(valid);
    }
}
```

```
import java.util.*;

public class DistinctImmutableAllMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java", "Java", "Go");

      Set<String> distinct = new HashSet<String>(list);

      boolean valid = true;

      for (String s : distinct) {
         if (s.length() < 2) {
            valid = false;
            break;
         }
      }

      System.out.println(valid);
   }
}
```

6 distinct() immutable objects + allMatch() (Validation)

```
import java.util.*;

public class DistinctImmutableAllMatch {
   public static void main(String[] args) {
      boolean valid = List.of("Java", "Java", "Go").stream()
            .distinct()
            .allMatch(s -> s.length() >= 2);
```

```
        System.out.println(valid);
    }
}
```

```
import java.util.*;

public class MapTransformAllMatch {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("anurag", "amit");

        boolean valid = true;

        for (String s : list) {
            String upper = s.toUpperCase();                    // map

            if (!upper.equals(upper.toUpperCase())) {          // allMatch
                valid = false;
                break;
            }
        }

        System.out.println(valid);
    }
}
```

7 map() transform objects + allMatch() (Validation)

```
import java.util.*;

public class MapTransformAllMatch {
    public static void main(String[] args) {
        boolean valid = List.of("anurag", "amit").stream()
                .map(String::toUpperCase)
                .allMatch(s -> s.equals(s.toUpperCase()));
```

```
        System.out.println(valid);
    }
}
```

```
import java.util.*;

public class MapCreateObjectAllMatch {

    static class UserDTO {
        String name;
        UserDTO(String name) { this.name = name; }
    }

    public static void main(String[] args) {

        List<String> list = Arrays.asList("Anurag", "Amit");

        boolean valid = true;

        for (String s : list) {
            UserDTO dto = new UserDTO(s);                    // map

            if (dto.name.length() < 4) {                     // allMatch
                valid = false;
                break;
            }
        }

        System.out.println(valid);
    }
}
```

8 map() create new objects + allMatch() (Validation)

```
import java.util.*;
```

```
public class MapCreateObjectAllMatch {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      boolean valid = List.of("Anurag", "Amit").stream()
            .map(UserDTO::new)
            .allMatch(dto -> dto.name.length() >= 4);

      System.out.println(valid);
   }
}
```

```
import java.util.*;

public class MapPureFunctionAllMatch {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(2, 3, 4);

      boolean valid = true;

      for (Integer n : list) {
         int squared = square(n);                    // map

         if (squared <= 0) {                         // allMatch
            valid = false;
            break;
         }
      }

      System.out.println(valid);
   }
}
```

9 map() call pure functions + allMatch() (Validation)

```
import java.util.*;

public class MapPureFunctionAllMatch {
```

```
    static int square(int n) { return n * n; }

    public static void main(String[] args) {
        boolean valid = List.of(2, 3, 4).stream()
                .map(MapPureFunctionAllMatch::square)
                .allMatch(n -> n > 0);

        System.out.println(valid);
    }
}
```

```
import java.util.*;

public class FlatMapStreamAllMatch {
    public static void main(String[] args) {

        List<List<Integer>> list = new ArrayList<List<Integer>>();
        list.add(Arrays.asList(1, 2));
        list.add(Arrays.asList(3, 4));

        boolean valid = true;

        for (List<Integer> inner : list) {
            for (Integer n : inner) {                    // flatMap
                if (n <= 0) {                            // allMatch
                    valid = false;
                    break;
                }
            }
        }

        System.out.println(valid);
    }
}
```

10 flatMap() return Stream + allMatch() (Validation)

```
import java.util.*;

public class FlatMapStreamAllMatch {
    public static void main(String[] args) {
        boolean valid = List.of(
                List.of(1, 2),
```

```
            List.of(3, 4)
        ).stream()
         .flatMap(List::stream)
         .allMatch(n -> n > 0);

        System.out.println(valid);
    }
}
```

```
import java.util.*;

public class FlatMapSplitAllMatch {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Java Stream", "Flat Map");

        boolean valid = true;

        for (String s : list) {
            String[] words = s.split(" ");                    // flatMap

            for (int i = 0; i < words.length; i++) {
                if (words[i].length() < 3) {
                    valid = false;
                    break;
                }
            }
        }

        System.out.println(valid);
    }
}
```

11 flatMap() split data + allMatch() (Validation)

```
import java.util.*;

public class FlatMapSplitAllMatch {
    public static void main(String[] args) {
        boolean valid = List.of("Java Stream", "Flat Map").stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
```

```
            .allMatch(word -> word.length() >= 3);

        System.out.println(valid);
    }
}
```

```
import java.util.*;

public class FlatMapExpandAllMatch {
    public static void main(String[] args) {

        Map<String, List<String>> map = new HashMap<String, List<String>>();
        map.put("A", Arrays.asList("1", "2"));
        map.put("B", Arrays.asList("3"));

        boolean valid = true;

        for (List<String> values : map.values()) {
            for (String v : values) {
                if (v.length() == 0) {
                    valid = false;
                    break;
                }
            }
        }

        System.out.println(valid);
    }
}
```

12 flatMap() expand collections + allMatch() (Validation)

```
import java.util.*;

public class FlatMapExpandAllMatch {
    public static void main(String[] args) {
        boolean valid = Map.of(
            "A", List.of("1", "2"),
```

```
            "B", List.of("3")
        ).values().stream()
         .flatMap(Collection::stream)
         .allMatch(v -> !v.isEmpty());

        System.out.println(valid);
    }
}
```

```
import java.util.*;

public class SortedNaturalAllMatch {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(1, 2, 3);

        Collections.sort(list);                                    // sorted

        boolean valid = true;

        for (Integer n : list) {
            if (n < 1) {
                valid = false;
                break;
            }
        }

        System.out.println(valid);
    }
}
```

13 sorted() natural ordering + allMatch() (Validation)

```
import java.util.*;

public class SortedNaturalAllMatch {
    public static void main(String[] args) {
        boolean valid = List.of(1, 2, 3).stream()
                .sorted()
                .allMatch(n -> n >= 1);
```

```
        System.out.println(valid);
    }
}
```

```
import java.util.*;

public class SortedCustomAllMatch {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Anurag", "Ravi", "Amit");

        Collections.sort(list, Collections.reverseOrder());

        boolean valid = true;

        for (String name : list) {
            if (name.length() < 4) {
                valid = false;
                break;
            }
        }

        System.out.println(valid);
    }
}
```

14 sorted() custom comparator + allMatch() (Validation)

```
import java.util.*;

public class SortedCustomAllMatch {
    public static void main(String[] args) {
        boolean valid = List.of("Anurag", "Ravi", "Amit").stream()
                .sorted(Comparator.reverseOrder())
                .allMatch(name -> name.length() >= 4);
```

```
        System.out.println(valid);
    }
}
```

```
import java.util.*;

public class SortedMultiFieldAllMatch {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {

        List<Employee> list = Arrays.asList(
                new Employee("Anurag", 30),
                new Employee("Anurag", 25),
                new Employee("Ravi", 35)
        );

        Collections.sort(list, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0)
                    return nameCompare;
                return e1.age - e2.age;
            }
        });
```

```
        boolean valid = true;

        for (Employee e : list) {
            if (e.age < 18) {
                valid = false;
                break;
            }
        }

        System.out.println(valid);
    }
}
```

15 sorted() multi-field compare + allMatch() (Validation)

```
import java.util.*;

public class SortedMultiFieldAllMatch {

    static class Employee {
        String name;
        int age;
        Employee(String name, int age) {
            this.name = name; this.age = age;
        }
    }

    public static void main(String[] args) {
        boolean valid = List.of(
                new Employee("Anurag", 30),
                new Employee("Anurag", 25),
                new Employee("Ravi", 35)
        ).stream()
         .sorted(Comparator
                 .comparing((Employee e) -> e.name)
                 .thenComparingInt(e -> e.age))
         .allMatch(e -> e.age >= 18);

        System.out.println(valid);
    }
}
```

```
import java.util.*;
```

```
public class PeekLoggingAllMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      boolean valid = true;

      for (Integer n : list) {
         System.out.println("Checking: " + n);                    // peek

         if (n <= 0) {
            valid = false;
            break;
         }
      }
      System.out.println(valid);
   }
}
```

16 peek() logging + allMatch() (Validation)

```
import java.util.*;

public class PeekLoggingAllMatch {
   public static void main(String[] args) {
      boolean valid = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Checking: " + n))
            .allMatch(n -> n > 0);

      System.out.println(valid);
   }
}
```

```
import java.util.*;

public class PeekDebugAllMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);

      boolean valid = true;

      for (Integer n : list) {
         if (n == 20)
            System.out.println("Debug hit");                    // peek

         if (n < 10) {
            valid = false;
```

```
            break;
        }
    }

    System.out.println(valid);
  }
}
```

17 peek() debugging + allMatch() (Validation)

```
import java.util.*;

public class PeekDebugAllMatch {
  public static void main(String[] args) {
    boolean valid = List.of(10, 20, 30).stream()
          .peek(n -> {
            if (n == 20) System.out.println("Debug hit");
          })
          .allMatch(n -> n >= 10);

    System.out.println(valid);
  }
}
```

```
import java.util.*;

public class PeekMetricsAllMatch {
  public static void main(String[] args) {

    List<Integer> list = Arrays.asList(5, 10, 15);

    int counter = 0;
    boolean valid = true;

    for (Integer n : list) {
      counter++;                                        // peek

      if (n <= 0) {
```

```
            valid = false;
            break;
        }
    }

    System.out.println("Valid: " + valid);
    System.out.println("Processed elements: " + counter);
  }
}
```

18 peek() metrics + allMatch() (Validation)

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsAllMatch {
  public static void main(String[] args) {
    AtomicInteger counter = new AtomicInteger();

    boolean valid = List.of(5, 10, 15).stream()
          .peek(n -> counter.incrementAndGet())
          .allMatch(n -> n > 0);

    System.out.println("Valid: " + valid);
    System.out.println("Processed elements: " + counter.get());
  }
}
```

---

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + allmatch() rule checks

filter() null checks + allmatch() rule checks

filter() comparisons + allmatch() rule checks

filter() method calls returning boolean + allmatch() rule checks

distinct() rely on equals() / hashcode() + allmatch() rule checks

distinct() immutable objects + allmatch() rule checks

map() transform objects + allmatch() rule checks

map() create new objects + allmatch() rule checks

map() call pure functions + allmatch() rule checks

flatmap() return stream + allmatch() rule checks

flatmap() split data + allmatch() rule checks

flatmap() expand collections + allmatch() rule checks

sorted() natural ordering + allmatch() rule checks

sorted() custom comparator + allmatch() rule checks

sorted() multi-field compare + allmatch() rule checks

peek() logging + allmatch() rule checks

peek() debugging + allmatch() rule checks

peek() metrics + allmatch() rule checks

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = allMatch()**
- Purpose = **rule checks** (business / validation rules)
- **Short-circuiting behavior** is intentional
- Java **8+**
- No collectors, no reduce, no side-effect mutation (except peek demos)

```
import java.util.*;

public class FilterBooleanAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(2, 4, 6, 8);

      boolean rulePassed = true;

      for (Integer n : list) {
         if (n > 0) {                                    // filter
            if (n % 2 != 0) {                            // allMatch condition
               rulePassed = false;
               break;
            }
         }
      }

      System.out.println(rulePassed);
   }
```

```
}
```

1 filter() boolean checks + allMatch() — Rule checks

```
import java.util.*;

public class FilterBooleanAllMatchRule {
   public static void main(String[] args) {
      boolean rulePassed = List.of(2, 4, 6, 8).stream()
            .filter(n -> n > 0)
            .allMatch(n -> n % 2 == 0);

      System.out.println(rulePassed);
   }
}
```

```
import java.util.*;

public class FilterNullAllMatchRule {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");

      boolean rulePassed = true;

      for (String name : list) {
         if (name != null) {
            if (name.length() < 4) {
               rulePassed = false;
               break;
            }
         }
      }

      System.out.println(rulePassed);
   }
}
```

2 filter() null checks + allMatch() — Rule checks

```
import java.util.*;

public class FilterNullAllMatchRule {
   public static void main(String[] args) {
      boolean rulePassed = Arrays.asList("Anurag", "Amit").stream()
            .filter(Objects::nonNull)
            .allMatch(name -> name.length() >= 4);
```

```
        System.out.println(rulePassed);
    }
}
```

```
import java.util.*;

public class FilterComparisonAllMatchRule {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(25, 30, 40);

        boolean rulePassed = true;

        for (Integer age : list) {
            if (age >= 18) {
                if (age > 60) {
                    rulePassed = false;
                    break;
                }
            }
        }

        System.out.println(rulePassed);
    }
}
```

3 filter() comparisons + allMatch() — Rule checks

```
import java.util.*;

public class FilterComparisonAllMatchRule {
    public static void main(String[] args) {
        boolean rulePassed = List.of(25, 30, 40).stream()
                .filter(age -> age >= 18)
                .allMatch(age -> age <= 60);
```

```
        System.out.println(rulePassed);
    }
}
```

```
import java.util.*;

public class FilterMethodAllMatchRule {

    static class User {
        boolean verified;
        User(boolean verified) { this.verified = verified; }
        boolean isVerified() { return verified; }
    }

    public static void main(String[] args) {

        List<User> list = Arrays.asList(
                new User(true),
                new User(true)
        );

        boolean rulePassed = true;

        for (User u : list) {
            if (u.isVerified()) {
                if (!u.isVerified()) {
                    rulePassed = false;
                    break;
                }
            }
        }

        System.out.println(rulePassed);
    }
}
```

4 filter() method calls returning boolean + allMatch() — Rule checks

```
import java.util.*;

public class FilterMethodAllMatchRule {

    static class User {
        boolean verified;
        User(boolean verified) { this.verified = verified; }
        boolean isVerified() { return verified; }
    }

    public static void main(String[] args) {
        boolean rulePassed = List.of(
                new User(true),
                new User(true)
        ).stream()
         .filter(User::isVerified)
         .allMatch(User::isVerified);

        System.out.println(rulePassed);
    }
}
```

```
import java.util.*;

public class DistinctEqualsAllMatchRule {

   static class Person {
      String email;

      Person(String email) { this.email = email; }

      public boolean equals(Object o) {
         if (this == o) return true;
         if (!(o instanceof Person)) return false;
         Person p = (Person) o;
         return email.equals(p.email);
      }

      public int hashCode() {
         return email.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> list = Arrays.asList(
            new Person("a@mail.com"),
            new Person("a@mail.com"),
            new Person("b@mail.com")
      );

      Set<Person> unique = new HashSet<Person>(list);
```

```
        boolean rulePassed = true;

        for (Person p : unique) {
            if (!p.email.contains("@")) {
                rulePassed = false;
                break;
            }
        }

        System.out.println(rulePassed);
    }
}
```

5 distinct() rely on equals() / hashCode() + allMatch() — Rule checks

```
import java.util.*;

public class DistinctEqualsAllMatchRule {

    static class Person {
        String email;
        Person(String email) { this.email = email; }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && email.equals(p.email);
        }

        @Override
        public int hashCode() {
            return Objects.hash(email);
        }
    }

    public static void main(String[] args) {
        boolean rulePassed = List.of(
                new Person("a@mail.com"),
                new Person("a@mail.com"),
                new Person("b@mail.com")
        ).stream()
         .distinct()
         .allMatch(p -> p.email.contains("@"));

        System.out.println(rulePassed);
    }
```

}

```
import java.util.*;

public class DistinctImmutableAllMatchRule {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java", "Java", "Python");
      Set<String> unique = new HashSet<String>(list);

      boolean rulePassed = true;

      for (String lang : unique) {
         if (lang.length() < 4) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

6 distinct() immutable objects + allMatch() — Rule checks

```
import java.util.*;

public class DistinctImmutableAllMatchRule {
   public static void main(String[] args) {
      boolean rulePassed = List.of("Java", "Java", "Python").stream()
            .distinct()
            .allMatch(lang -> lang.length() >= 4);

      System.out.println(rulePassed);
```

```
    }
}
```

```
import java.util.*;

public class MapTransformAllMatchRule {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("anurag", "amit");

        boolean rulePassed = true;

        for (String s : list) {
            String upper = s.toUpperCase();
            if (!upper.equals(upper.toUpperCase())) {
                rulePassed = false;
                break;
            }
        }

        System.out.println(rulePassed);
    }
}
```

7 map() transform objects + allMatch() — Rule checks

```
import java.util.*;

public class MapTransformAllMatchRule {
    public static void main(String[] args) {
        boolean rulePassed = List.of("anurag", "amit").stream()
                .map(String::toUpperCase)
                .allMatch(s -> s.equals(s.toUpperCase()));

        System.out.println(rulePassed);
```

```
    }
}
```

```
import java.util.*;

public class MapCreateObjectAllMatchRule {

    static class UserDTO {
        String name;
        UserDTO(String name) { this.name = name; }
    }

    public static void main(String[] args) {

        List<String> list = Arrays.asList("Anurag", "Amit");

        boolean rulePassed = true;

        for (String name : list) {
            UserDTO dto = new UserDTO(name);
            if (dto.name.length() < 4) {
                rulePassed = false;
                break;
            }
        }

        System.out.println(rulePassed);
    }
}
```

8 map() create new objects + allMatch() — Rule checks

```
import java.util.*;

public class MapCreateObjectAllMatchRule {
```

```
    static class UserDTO {
        String name;
        UserDTO(String name) { this.name = name; }
    }

    public static void main(String[] args) {
        boolean rulePassed = List.of("Anurag", "Amit").stream()
                .map(UserDTO::new)
                .allMatch(dto -> dto.name.length() >= 4);

        System.out.println(rulePassed);
    }
}
```

```
import java.util.*;

public class MapPureFunctionAllMatchRule {

    static int normalize(int n) { return n * 10; }

    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(1, 2, 3);

        boolean rulePassed = true;

        for (Integer n : list) {
            int value = normalize(n);
            if (value < 10) {
                rulePassed = false;
                break;
            }
        }

        System.out.println(rulePassed);
    }
}
```

9 map() call pure functions + allMatch() — Rule checks

```
import java.util.*;

public class MapPureFunctionAllMatchRule {

    static int normalize(int n) { return n * 10; }
```

```
    public static void main(String[] args) {
        boolean rulePassed = List.of(1, 2, 3).stream()
                .map(MapPureFunctionAllMatchRule::normalize)
                .allMatch(n -> n >= 10);

        System.out.println(rulePassed);
    }
}
```

```
import java.util.*;

public class FlatMapStreamAllMatchRule {
    public static void main(String[] args) {

        List<List<Integer>> list = new ArrayList<List<Integer>>();
        list.add(Arrays.asList(1,2));
        list.add(Arrays.asList(3,4));

        boolean rulePassed = true;

        for (List<Integer> inner : list) {
            for (Integer n : inner) {
                if (n <= 0) {
                    rulePassed = false;
                    break;
                }
            }
        }

        System.out.println(rulePassed);
    }
}
```

10 flatMap() return Stream + allMatch() — Rule checks

```
import java.util.*;

public class FlatMapStreamAllMatchRule {
    public static void main(String[] args) {
        boolean rulePassed = List.of(
                List.of(1, 2),
```

```
            List.of(3, 4)
        ).stream()
         .flatMap(List::stream)
         .allMatch(n -> n > 0);

        System.out.println(rulePassed);
    }
}
```

```
import java.util.*;

public class FlatMapSplitAllMatchRule {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Java Stream", "Flat Map");

        boolean rulePassed = true;

        for (String s : list) {
            String[] words = s.split(" ");
            for (String word : words) {
                if (word.length() < 3) {
                    rulePassed = false;
                    break;
                }
            }
        }

        System.out.println(rulePassed);
    }
}
```

11 flatMap() split data + allMatch() — Rule checks

```
import java.util.*;

public class FlatMapSplitAllMatchRule {
    public static void main(String[] args) {
        boolean rulePassed = List.of("Java Stream", "Flat Map").stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
                .allMatch(word -> word.length() >= 3);
```

```
        System.out.println(rulePassed);
    }
}
```

```
import java.util.*;

public class FlatMapExpandAllMatchRule {
    public static void main(String[] args) {

        Map<String, List<String>> map = new HashMap<String, List<String>>();
        map.put("A", Arrays.asList("1", "2"));
        map.put("B", Arrays.asList("3"));

        boolean rulePassed = true;

        for (List<String> values : map.values()) {
            for (String v : values) {
                if (v.length() == 0) {
                    rulePassed = false;
                    break;
                }
            }
        }

        System.out.println(rulePassed);
    }
}
```

12 flatMap() expand collections + allMatch() — Rule checks

```
import java.util.*;

public class FlatMapExpandAllMatchRule {
    public static void main(String[] args) {
        boolean rulePassed = Map.of(
                "A", List.of("1", "2"),
```

```
            "B", List.of("3")
        ).values().stream()
         .flatMap(Collection::stream)
         .allMatch(v -> !v.isEmpty());

        System.out.println(rulePassed);
    }
}
```

```
import java.util.*;

public class SortedNaturalAllMatchRule {
    public static void main(String[] args) {

        List<Integer> list = new ArrayList<Integer>(Arrays.asList(1,2,3));
        Collections.sort(list);

        boolean rulePassed = true;

        for (Integer n : list) {
            if (n < 1) {
                rulePassed = false;
                break;
            }
        }

        System.out.println(rulePassed);
    }
}
```

13 sorted() natural ordering + allMatch() — Rule checks

```
import java.util.*;

public class SortedNaturalAllMatchRule {
    public static void main(String[] args) {
        boolean rulePassed = List.of(1, 2, 3).stream()
                .sorted()
                .allMatch(n -> n >= 1);

        System.out.println(rulePassed);
```

```
    }
}
```

```
import java.util.*;

public class SortedCustomAllMatchRule {
    public static void main(String[] args) {

        List<String> list = new ArrayList<String>(
                Arrays.asList("Anurag", "Ravi", "Amit")
        );

        Collections.sort(list, new Comparator<String>() {
            public int compare(String a, String b) {
                return b.compareTo(a);
            }
        });

        boolean rulePassed = true;

        for (String name : list) {
            if (name.length() < 4) {
                rulePassed = false;
                break;
            }
        }

        System.out.println(rulePassed);
    }
}
```

14 sorted() custom comparator + allMatch() — Rule checks

```
import java.util.*;
```

```
public class SortedCustomAllMatchRule {
   public static void main(String[] args) {
      boolean rulePassed = List.of("Anurag", "Ravi", "Amit").stream()
            .sorted(Comparator.reverseOrder())
            .allMatch(name -> name.length() >= 4);

      System.out.println(rulePassed);
   }
}
```

```
import java.util.*;

public class SortedMultiFieldAllMatchRule {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> list = new ArrayList<Employee>();
      list.add(new Employee("Anurag", 30));
      list.add(new Employee("Anurag", 25));
      list.add(new Employee("Ravi", 35));

      Collections.sort(list, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {
            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0) return nameCompare;
            return e1.age - e2.age;
         }
      });

      boolean rulePassed = true;
```

```
        for (Employee e : list) {
            if (e.age < 18) {
                rulePassed = false;
                break;
            }
        }

        System.out.println(rulePassed);
    }
}
```

15 sorted() multi-field compare + allMatch() — Rule checks

```
import java.util.*;

public class SortedMultiFieldAllMatchRule {

    static class Employee {
        String name;
        int age;
        Employee(String name, int age) {
            this.name = name; this.age = age;
        }
    }

    public static void main(String[] args) {
        boolean rulePassed = List.of(
                new Employee("Anurag", 30),
                new Employee("Anurag", 25),
                new Employee("Ravi", 35)
        ).stream()
         .sorted(Comparator
                .comparing((Employee e) -> e.name)
                .thenComparingInt(e -> e.age))
         .allMatch(e -> e.age >= 18);

        System.out.println(rulePassed);
    }
}
```

```
import java.util.*;
```

```
public class PeekLoggingAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1,2,3);

      boolean rulePassed = true;

      for (Integer n : list) {
         System.out.println("Checking: " + n);
         if (n <= 0) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

16 peek() logging + allMatch() — Rule checks

```
import java.util.*;

public class PeekLoggingAllMatchRule {
   public static void main(String[] args) {
      boolean rulePassed = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Checking: " + n))
            .allMatch(n -> n > 0);

      System.out.println(rulePassed);
   }
}
```

```
import java.util.*;

public class PeekDebugAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10,20,30);

      boolean rulePassed = true;

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug breakpoint");
         }
         if (n < 10) {
            rulePassed = false;
```

```
            break;
        }
    }

    System.out.println(rulePassed);
  }
}
```

17 peek() debugging + allMatch() — Rule checks

```
import java.util.*;

public class PeekDebugAllMatchRule {
  public static void main(String[] args) {
    boolean rulePassed = List.of(10, 20, 30).stream()
          .peek(n -> {
            if (n == 20) System.out.println("Debug breakpoint");
          })
          .allMatch(n -> n >= 10);

    System.out.println(rulePassed);
  }
}
```

```
import java.util.*;

public class PeekMetricsAllMatchRule {
  public static void main(String[] args) {

    List<Integer> list = Arrays.asList(5,10,15);

    int processed = 0;
    boolean rulePassed = true;

    for (Integer n : list) {
      processed++;
      if (n <= 0) {
          rulePassed = false;
```

```
            break;
        }
    }

    System.out.println("Rule passed: " + rulePassed);
    System.out.println("Processed elements: " + processed);
  }
}
```

18 peek() metrics + allMatch() — Rule checks

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsAllMatchRule {
  public static void main(String[] args) {
    AtomicInteger processed = new AtomicInteger();

    boolean rulePassed = List.of(5, 10, 15).stream()
          .peek(n -> processed.incrementAndGet())
          .allMatch(n -> n > 0);

    System.out.println("Rule passed: " + rulePassed);
    System.out.println("Processed elements: " + processed.get());
  }
}
```

---

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + anymatch() existence checks

filter() null checks + anymatch() existence checks

filter() comparisons + anymatch() existence checks

filter() method calls returning boolean + anymatch() existence checks

distinct() rely on equals() / hashcode() + anymatch() existence checks

distinct() immutable objects + anymatch() existence checks

map() transform objects + anymatch() existence checks

map() create new objects + anymatch() existence checks

map() call pure functions + anymatch() existence checks

flatmap() return stream + anymatch() existence checks

flatmap() split data + anymatch() existence checks

flatmap() expand collections + anymatch() existence checks

sorted() natural ordering + anymatch() existence checks

sorted() custom comparator + anymatch() existence checks

sorted() multi-field compare + anymatch() existence checks

peek() logging + anymatch() existence checks

peek() debugging + anymatch() existence checks

peek() metrics + anymatch() existence checks

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = anyMatch()**
- Purpose = **existence checks** (returns true if **any element** satisfies the predicate)
- Demonstrates **short-circuiting** (stops on first match)
- Java **8+** compatible
- No collectors / reduce / custom aggregation

```
import java.util.*;

public class FilterBooleanAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 3, 4, 7);
      boolean exists = false;

      for (Integer n : list) {
         if (n > 0 && n % 2 == 0) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

1 filter() boolean checks + anyMatch() — Existence check

```
import java.util.*;

public class FilterBooleanAnyMatch {
   public static void main(String[] args) {
      boolean exists = List.of(1, 3, 4, 7).stream()
            .filter(n -> n > 0)
            .anyMatch(n -> n % 2 == 0);

      System.out.println(exists);
   }
}
```

```
import java.util.*;

public class FilterNullAnyMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList(null, "Anurag", "Amit");
      boolean exists = false;

      for (String s : list) {
         if (s != null && s.startsWith("A")) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

2 filter() null checks + anyMatch() — Existence check

```
import java.util.*;

public class FilterNullAnyMatch {
   public static void main(String[] args) {
      boolean exists = Arrays.asList(null, "Anurag", "Amit").stream()
            .filter(Objects::nonNull)
            .anyMatch(s -> s.startsWith("A"));

      System.out.println(exists);
```

```
    }
}
```

```
import java.util.*;

public class FilterComparisonAnyMatch {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(20000, 45000, 60000);
        boolean exists = false;

        for (Integer s : list) {
            if (s >= 30000 && s > 50000) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

3 filter() comparisons + anyMatch() — Existence check

```
import java.util.*;

public class FilterComparisonAnyMatch {
    public static void main(String[] args) {
        boolean exists = List.of(20000, 45000, 60000).stream()
                .filter(s -> s >= 30000)
                .anyMatch(s -> s > 50000);

        System.out.println(exists);
```

```
    }
}
```

```
import java.util.*;

public class FilterMethodAnyMatch {

    static class User {
        boolean active;
        User(boolean active) { this.active = active; }
        boolean isActive() { return active; }
    }

    public static void main(String[] args) {

        List<User> list = Arrays.asList(
                new User(false),
                new User(true)
        );

        boolean exists = false;

        for (User u : list) {
            if (u.isActive() && u.isActive()) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

4 filter() method calls returning boolean + anyMatch() — Existence check

```
import java.util.*;

public class FilterMethodAnyMatch {

   static class User {
      boolean active;
      User(boolean active) { this.active = active; }
      boolean isActive() { return active; }
   }

   public static void main(String[] args) {
      boolean exists = List.of(
            new User(false),
            new User(true)
      ).stream()
       .filter(User::isActive)
       .anyMatch(User::isActive);

      System.out.println(exists);
   }
}
```

```
import java.util.*;

public class DistinctEqualsAnyMatch {

   static class Person {
      String name;
      Person(String name) { this.name = name; }

      public boolean equals(Object o) {
         if (!(o instanceof Person)) return false;
         Person p = (Person) o;
         return name.equals(p.name);
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> list = Arrays.asList(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
      );

      Set<Person> distinct = new HashSet<Person>(list);
```

```
        boolean exists = false;

        for (Person p : distinct) {
            if (p.name.equals("Ravi")) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

5 distinct() rely on equals() / hashCode() + anyMatch() — Existence check

```
import java.util.*;

public class DistinctEqualsAnyMatch {

    static class Person {
        String name;
        Person(String name) { this.name = name; }

        @Override public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }
        @Override public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        boolean exists = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        ).stream()
         .distinct()
         .anyMatch(p -> p.name.equals("Ravi"));

        System.out.println(exists);
    }
}
```

```
import java.util.*;

public class DistinctImmutableAnyMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java", "Java", "Python");
      Set<String> distinct = new HashSet<String>(list);

      boolean exists = false;

      for (String lang : distinct) {
         if (lang.startsWith("P")) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

6 distinct() immutable objects + anyMatch() — Existence check

```
import java.util.*;

public class DistinctImmutableAnyMatch {
   public static void main(String[] args) {
      boolean exists = List.of("Java", "Java", "Python").stream()
            .distinct()
            .anyMatch(lang -> lang.startsWith("P"));
```

```
        System.out.println(exists);
    }
}
```

```
import java.util.*;

public class MapTransformAnyMatch {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("anurag", "amit");
        boolean exists = false;

        for (String s : list) {
            String upper = s.toUpperCase();
            if (upper.equals("AMIT")) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

7 map() transform objects + anyMatch() — Existence check

```
import java.util.*;

public class MapTransformAnyMatch {
    public static void main(String[] args) {
        boolean exists = List.of("anurag", "amit").stream()
                .map(String::toUpperCase)
                .anyMatch(s -> s.equals("AMIT"));
```

```
        System.out.println(exists);
    }
}
```

```
import java.util.*;

public class MapCreateObjectAnyMatch {

    static class UserDTO {
        String name;
        UserDTO(String name) { this.name = name; }
    }

    public static void main(String[] args) {

        List<String> list = Arrays.asList("Anurag", "Amit");
        boolean exists = false;

        for (String s : list) {
            UserDTO dto = new UserDTO(s);
            if (dto.name.equals("Amit")) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

8 map() create new objects + anyMatch() — Existence check

```
import java.util.*;
```

```
public class MapCreateObjectAnyMatch {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      boolean exists = List.of("Anurag", "Amit").stream()
            .map(UserDTO::new)
            .anyMatch(dto -> dto.name.equals("Amit"));

      System.out.println(exists);
   }
}
```

```
import java.util.*;

public class MapPureFunctionAnyMatch {

   static int square(int n) { return n * n; }

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(2, 3, 4);
      boolean exists = false;

      for (Integer n : list) {
         int sq = square(n);
         if (sq == 16) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

9 map() call pure functions + anyMatch() — Existence check

```
import java.util.*;

public class MapPureFunctionAnyMatch {

   static int square(int n) { return n * n; }
```

```
    public static void main(String[] args) {
        boolean exists = List.of(2, 3, 4).stream()
                .map(MapPureFunctionAnyMatch::square)
                .anyMatch(n -> n == 16);

        System.out.println(exists);
    }
}
```

```
import java.util.*;

public class FlatMapStreamAnyMatch {
    public static void main(String[] args) {

        List<List<Integer>> list = Arrays.asList(
                Arrays.asList(1, 2),
                Arrays.asList(3, 4)
        );

        boolean exists = false;

        for (List<Integer> inner : list) {
            for (Integer n : inner) {
                if (n == 3) {
                    exists = true;
                    break;
                }
            }
        }

        System.out.println(exists);
    }
}
```

10 flatMap() return Stream + anyMatch() — Existence check

```
import java.util.*;

public class FlatMapStreamAnyMatch {
```

```
    public static void main(String[] args) {
        boolean exists = List.of(
                List.of(1, 2),
                List.of(3, 4)
        ).stream()
         .flatMap(List::stream)
         .anyMatch(n -> n == 3);

        System.out.println(exists);
    }
}
```

```
import java.util.*;

public class FlatMapSplitAnyMatch {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Java Stream", "Flat Map");
        boolean exists = false;

        for (String s : list) {
            String[] parts = s.split(" ");
            for (int i = 0; i < parts.length; i++) {
                if (parts[i].equals("Stream")) {
                    exists = true;
                    break;
                }
            }
        }

        System.out.println(exists);
    }
}
```

11 flatMap() split data + anyMatch() — Existence check

```
import java.util.*;

public class FlatMapSplitAnyMatch {
    public static void main(String[] args) {
        boolean exists = List.of("Java Stream", "Flat Map").stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
```

```
            .anyMatch(word -> word.equals("Stream"));

        System.out.println(exists);
    }
}
```

```
import java.util.*;

public class FlatMapExpandAnyMatch {
    public static void main(String[] args) {

        Map<String, List<String>> map = new HashMap<String, List<String>>();
        map.put("A", Arrays.asList("1", "2"));
        map.put("B", Arrays.asList("3"));

        boolean exists = false;

        for (List<String> values : map.values()) {
            for (String v : values) {
                if (v.equals("2")) {
                    exists = true;
                    break;
                }
            }
        }

        System.out.println(exists);
    }
}
```

12 flatMap() expand collections + anyMatch() — Existence check

```
import java.util.*;

public class FlatMapExpandAnyMatch {
    public static void main(String[] args) {
```

```
        boolean exists = Map.of(
                "A", List.of("1", "2"),
                "B", List.of("3")
        ).values().stream()
         .flatMap(Collection::stream)
         .anyMatch(v -> v.equals("2"));

        System.out.println(exists);
    }
}
```

```
import java.util.*;

public class SortedNaturalAnyMatch {
    public static void main(String[] args) {

        List<Integer> list = new ArrayList<Integer>(Arrays.asList(5, 1, 3));
        Collections.sort(list);

        boolean exists = false;

        for (Integer n : list) {
            if (n == 1) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

13 sorted() natural ordering + anyMatch() — Existence check

```
import java.util.*;

public class SortedNaturalAnyMatch {
    public static void main(String[] args) {
        boolean exists = List.of(5, 1, 3).stream()
                .sorted()
                .anyMatch(n -> n == 1);
```

```
        System.out.println(exists);
    }
}
```

```
import java.util.*;

public class SortedCustomAnyMatch {
    public static void main(String[] args) {

        List<String> list = new ArrayList<String>(
                Arrays.asList("Anurag", "Ravi", "Amit"));

        Collections.sort(list, Collections.reverseOrder());

        boolean exists = false;

        for (String name : list) {
            if (name.equals("Amit")) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

14 sorted() custom comparator + anyMatch() — Existence check

```
import java.util.*;

public class SortedCustomAnyMatch {
    public static void main(String[] args) {
        boolean exists = List.of("Anurag", "Ravi", "Amit").stream()
```

```
            .sorted(Comparator.reverseOrder())
            .anyMatch(name -> name.equals("Amit"));

        System.out.println(exists);
    }
}
```

```
import java.util.*;

public class SortedMultiFieldAnyMatch {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {

        List<Employee> list = new ArrayList<Employee>();
        list.add(new Employee("Anurag", 30));
        list.add(new Employee("Anurag", 25));
        list.add(new Employee("Ravi", 35));

        Collections.sort(list, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0) return nameCompare;
                return e1.age - e2.age;
            }
        });
```

```
        boolean exists = false;

        for (Employee e : list) {
            if (e.age == 25) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

15 sorted() multi-field compare + anyMatch() — Existence check

```
import java.util.*;

public class SortedMultiFieldAnyMatch {

    static class Employee {
        String name;
        int age;
        Employee(String name, int age) {
            this.name = name; this.age = age;
        }
    }

    public static void main(String[] args) {
        boolean exists = List.of(
                new Employee("Anurag", 30),
                new Employee("Anurag", 25),
                new Employee("Ravi", 35)
        ).stream()
         .sorted(Comparator
                .comparing((Employee e) -> e.name)
                .thenComparingInt(e -> e.age))
         .anyMatch(e -> e.age == 25);

        System.out.println(exists);
    }
}
```

```
import java.util.*;

public class PeekLoggingAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);
      boolean exists = false;

      for (Integer n : list) {
         System.out.println("Checking: " + n);
         if (n == 2) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

16 peek() logging + anyMatch() — Existence check

```
import java.util.*;

public class PeekLoggingAnyMatch {
   public static void main(String[] args) {
      boolean exists = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Checking: " + n))
            .anyMatch(n -> n == 2);

      System.out.println(exists);
   }
}
```

```
import java.util.*;

public class PeekDebugAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);
      boolean exists = false;

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug breakpoint");
         }
         if (n == 20) {
            exists = true;
```

```
            break;
        }
    }

    System.out.println(exists);
  }
}
```

17 peek() debugging + anyMatch() — Existence check

```
import java.util.*;

public class PeekDebugAnyMatch {
  public static void main(String[] args) {
    boolean exists = List.of(10, 20, 30).stream()
          .peek(n -> {
             if (n == 20) System.out.println("Debug breakpoint");
          })
          .anyMatch(n -> n == 20);
    System.out.println(exists);
  }
}
```

```
import java.util.*;

public class PeekMetricsAnyMatch {
  public static void main(String[] args) {

    List<Integer> list = Arrays.asList(5, 10, 15);
    int processed = 0;
    boolean exists = false;

    for (Integer n : list) {
       processed++;
       if (n == 10) {
          exists = true;
          break;
       }
    }

    System.out.println("Exists: " + exists);
    System.out.println("Processed elements: " + processed);
  }
}
```

18 peek() metrics + anyMatch() — Existence check

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
```

```
public class PeekMetricsAnyMatch {
   public static void main(String[] args) {
      AtomicInteger processed = new AtomicInteger();

      boolean exists = List.of(5, 10, 15).stream()
            .peek(n -> processed.incrementAndGet())
            .anyMatch(n -> n == 10);

      System.out.println("Exists: " + exists);
      System.out.println("Processed elements: " + processed.get());
   }
}
```

---

complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + nonematch() negative validation

filter() null checks + nonematch() negative validation

filter() comparisons + nonematch() negative validation

filter() method calls returning boolean + nonematch() negative validation

distinct() rely on equals() / hashcode() + nonematch() negative validation

distinct() immutable objects + nonematch() negative validation

map() transform objects + nonematch() negative validation

map() create new objects + nonematch() negative validation

map() call pure functions + nonematch() negative validation

flatmap() return stream + nonematch() negative validation

flatmap() split data + nonematch() negative validation

flatmap() expand collections + nonematch() negative validation

sorted() natural ordering + nonematch() negative validation

sorted() custom comparator + nonematch() negative validation

sorted() multi-field compare + nonematch() negative validation

peek() logging + nonematch() negative validation

peek() debugging + nonematch() negative validation

peek() metrics + nonematch() negative validation

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = noneMatch()**
- Purpose = **negative validation** (assert *no element* violates a rule)
- **Short-circuiting**: stops on the first violation
- Java **8+**, no collectors/reduce

```
import java.util.*;

public class FilterBooleanNoneMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(2, 4, 6, 8);

      boolean ok = true;

      for (Integer n : list) {
         if (n > 0) {
            if (n % 2 != 0) {                                    // noneMatch condition
               ok = false;
               break;
            }
         }
      }
```

```
        System.out.println(ok);
    }
}
```

1 filter() boolean checks + noneMatch() — Negative validation

```
import java.util.*;

public class FilterBooleanNoneMatch {
    public static void main(String[] args) {
        boolean ok = List.of(2, 4, 6, 8).stream()
                .filter(n -> n > 0)
                .noneMatch(n -> n % 2 != 0);                    // no odd numbers

        System.out.println(ok);
    }
}
```

```
import java.util.*;

public class FilterNullNoneMatch {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Anurag", "Amit");

        boolean ok = true;

        for (String s : list) {
            if (s != null) {
                if (s.length() == 0) {                          // isEmpty()
                    ok = false;
                    break;
                }
            }
        }

        System.out.println(ok);
    }
}
```

2 filter() null checks + noneMatch() — Negative validation

```
import java.util.*;

public class FilterNullNoneMatch {
    public static void main(String[] args) {
```

```
        boolean ok = Arrays.asList("Anurag", "Amit").stream()
            .filter(Objects::nonNull)
            .noneMatch(String::isEmpty);                    // no empty strings

        System.out.println(ok);
    }
}
```

```
import java.util.*;

public class FilterComparisonNoneMatch {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(25, 30, 40);

        boolean ok = true;

        for (Integer age : list) {
            if (age >= 18) {
                if (age > 60) {
                    ok = false;
                    break;
                }
            }
        }

        System.out.println(ok);
    }
}
```

3 filter() comparisons + noneMatch() — Negative validation

```
import java.util.*;

public class FilterComparisonNoneMatch {
    public static void main(String[] args) {
```

```
        boolean ok = List.of(25, 30, 40).stream()
                .filter(age -> age >= 18)
                .noneMatch(age -> age > 60);                    // no seniors

        System.out.println(ok);
    }
}
```

```
import java.util.*;

public class FilterMethodNoneMatch {

    static class User {
        boolean banned;
        User(boolean banned) { this.banned = banned; }
        boolean isBanned() { return banned; }
    }

    public static void main(String[] args) {

        List<User> list = Arrays.asList(
                new User(false),
                new User(false)
        );

        boolean ok = true;

        for (User u : list) {
            if (u.isBanned()) {
                ok = false;
                break;
            }
        }

        System.out.println(ok);
```

```
    }
}
```

4 filter() method calls returning boolean + noneMatch() — Negative validation

```
import java.util.*;

public class FilterMethodNoneMatch {

    static class User {
        boolean banned;
        User(boolean banned) { this.banned = banned; }
        boolean isBanned() { return banned; }
    }

    public static void main(String[] args) {
        boolean ok = List.of(new User(false), new User(false)).stream()
                .filter(u -> true)
                .noneMatch(User::isBanned);                    // no banned users

        System.out.println(ok);
    }
}
```

```
import java.util.*;

public class DistinctEqualsNoneMatch {

    static class Person {
        String email;

        Person(String email) { this.email = email; }

        public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof Person)) return false;
            Person p = (Person) o;
            return email.equals(p.email);
        }

        public int hashCode() {
            return email.hashCode();
        }
    }

    public static void main(String[] args) {

        List<Person> list = Arrays.asList(
                new Person("a@mail.com"),
```

```
            new Person("a@mail.com"),
            new Person("b@mail.com")
        );

        Set<Person> unique = new HashSet<Person>(list);

        boolean ok = true;

        for (Person p : unique) {
           if (p.email.endsWith("@spam.com")) {
              ok = false;
              break;
           }
        }

        System.out.println(ok);
    }
}
```

5 distinct() rely on equals() / hashCode() + noneMatch() — Negative validation

```
import java.util.*;

public class DistinctEqualsNoneMatch {

    static class Person {
        String email;
        Person(String email) { this.email = email; }
        @Override public boolean equals(Object o) {
           return o instanceof Person p && email.equals(p.email);
        }
        @Override public int hashCode() { return Objects.hash(email); }
    }

    public static void main(String[] args) {
        boolean ok = List.of(
            new Person("a@mail.com"),
            new Person("a@mail.com"),
            new Person("b@mail.com")
        ).stream()
         .distinct()
         .noneMatch(p -> p.email.endsWith("@spam.com"));

        System.out.println(ok);
    }
}
```

```
import java.util.*;

public class DistinctImmutableNoneMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java", "Java", "Python");

      Set<String> unique = new HashSet<String>(list);

      boolean ok = true;

      for (String lang : unique) {
         if (lang.length() < 3) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

6 distinct() immutable objects + noneMatch() — Negative validation

```
import java.util.*;

public class DistinctImmutableNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of("Java", "Java", "Python").stream()
            .distinct()
            .noneMatch(lang -> lang.length() < 3);

      System.out.println(ok);
   }
}
```

```
import java.util.*;

public class MapTransformNoneMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("anurag", "amit");

      boolean ok = true;

      for (String s : list) {
         String upper = s.toUpperCase();
```

```
        if (upper.indexOf(" ") != -1) {
            ok = false;
            break;
        }
    }

    System.out.println(ok);
  }
}
```

7 map() transform objects + noneMatch() — Negative validation

```
import java.util.*;

public class MapTransformNoneMatch {
  public static void main(String[] args) {
    boolean ok = List.of("anurag", "amit").stream()
          .map(String::toUpperCase)
          .noneMatch(s -> s.contains(" "));                    // no spaces

    System.out.println(ok);
  }
}
```

```
import java.util.*;

public class MapCreateObjectNoneMatch {

  static class UserDTO {
    String name;
    UserDTO(String name) { this.name = name; }
  }

  public static void main(String[] args) {

    List<String> list = Arrays.asList("Anurag", "Amit");

    boolean ok = true;

    for (String name : list) {
      UserDTO dto = new UserDTO(name);
      if (dto.name.trim().length() == 0) {
          ok = false;
          break;
      }
    }

    System.out.println(ok);
  }
```

```
}
```

8 map() create new objects + noneMatch() — Negative validation

```
import java.util.*;

public class MapCreateObjectNoneMatch {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {
      boolean ok = List.of("Anurag", "Amit").stream()
            .map(UserDTO::new)
            .noneMatch(dto -> dto.name.isBlank());

      System.out.println(ok);
   }
}
```

```
import java.util.*;

public class MapPureFunctionNoneMatch {

   static int normalize(int n) {
      return n * 10;
   }

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      boolean ok = true;

      for (Integer n : list) {
         int value = normalize(n);
         if (value <= 0) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

9 map() call pure functions + noneMatch() — Negative validation

```
import java.util.*;

public class MapPureFunctionNoneMatch {

   static int normalize(int n) { return n * 10; }

   public static void main(String[] args) {
      boolean ok = List.of(1, 2, 3).stream()
            .map(MapPureFunctionNoneMatch::normalize)
            .noneMatch(n -> n <= 0);

      System.out.println(ok);
   }
}
```

```
import java.util.*;

public class FlatMapStreamNoneMatch {
   public static void main(String[] args) {

      List<List<Integer>> list = Arrays.asList(
            Arrays.asList(1, 2),
            Arrays.asList(3, 4)
      );

      boolean ok = true;

      for (List<Integer> inner : list) {
         for (Integer n : inner) {
            if (n < 0) {
               ok = false;
               break;
            }
         }
      }

      System.out.println(ok);
   }
}
```

10 flatMap() return Stream + noneMatch() — Negative validation

```
import java.util.*;
```

```
public class FlatMapStreamNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(
            List.of(1, 2),
            List.of(3, 4)
      ).stream()
       .flatMap(List::stream)
       .noneMatch(n -> n < 0);

      System.out.println(ok);
   }
}
```

```
import java.util.*;

public class FlatMapSplitNoneMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java Stream", "Flat Map");

      boolean ok = true;

      for (String s : list) {
         String[] words = s.split(" ");
         for (int i = 0; i < words.length; i++) {
            if (words[i].length() < 3) {
               ok = false;
               break;
            }
         }
      }

      System.out.println(ok);
   }
}
```

11 flatMap() split data + noneMatch() — Negative validation

```
import java.util.*;

public class FlatMapSplitNoneMatch {
```

```
    public static void main(String[] args) {
        boolean ok = List.of("Java Stream", "Flat Map").stream()
                .flatMap(s -> Arrays.stream(s.split(" ")))
                .noneMatch(word -> word.length() < 3);

        System.out.println(ok);
    }
}
```

.

```
import java.util.*;

public class FlatMapExpandNoneMatch {
    public static void main(String[] args) {

        Map<String, List<String>> map = new HashMap<String, List<String>>();
        map.put("A", Arrays.asList("1", "2"));
        map.put("B", Arrays.asList("3"));

        boolean ok = true;

        for (List<String> values : map.values()) {
            for (String s : values) {
                if (s.length() == 0) {
                    ok = false;
                    break;
                }
            }
        }

        System.out.println(ok);
    }
}
```

12 flatMap() expand collections + noneMatch() — Negative validation

```
import java.util.*;
```

```
public class FlatMapExpandNoneMatch {
   public static void main(String[] args) {
      boolean ok = Map.of(
            "A", List.of("1", "2"),
            "B", List.of("3")
      ).values().stream()
       .flatMap(Collection::stream)
       .noneMatch(String::isEmpty);

      System.out.println(ok);
   }
}
```

```
import java.util.*;

public class SortedNaturalNoneMatch {
   public static void main(String[] args) {

      List<Integer> list = new ArrayList<Integer>(
            Arrays.asList(1, 2, 3)
      );

      Collections.sort(list);

      boolean ok = true;

      for (Integer n : list) {
         if (n <= 0) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

13 sorted() natural ordering + noneMatch() — Negative validation

```
import java.util.*;
```

```
public class SortedNaturalNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(1, 2, 3).stream()
            .sorted()
            .noneMatch(n -> n <= 0);

      System.out.println(ok);
   }
}
```

```
import java.util.*;

public class SortedCustomNoneMatch {
   public static void main(String[] args) {

      List<String> list = new ArrayList<String>(
            Arrays.asList("Anurag", "Ravi", "Amit")
      );

      Collections.sort(list, new Comparator<String>() {
         public int compare(String a, String b) {
            return b.compareTo(a);                    // reverse
         }
      });

      boolean ok = true;

      for (String name : list) {
         if (name.length() < 4) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

14 sorted() custom comparator + noneMatch() — Negative validation

```
import java.util.*;

public class SortedCustomNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of("Anurag", "Ravi", "Amit").stream()
            .sorted(Comparator.reverseOrder())
            .noneMatch(name -> name.length() < 4);

      System.out.println(ok);
   }
}
```

```
import java.util.*;

public class SortedMultiFieldNoneMatch {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> list = new ArrayList<Employee>();
      list.add(new Employee("Anurag", 30));
      list.add(new Employee("Anurag", 25));
      list.add(new Employee("Ravi", 35));

      Collections.sort(list, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {
            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0)
               return nameCompare;
            return e1.age - e2.age;
```

```
        }
    });

    boolean ok = true;

    for (Employee e : list) {
        if (e.age < 18) {
            ok = false;
            break;
        }
    }

    System.out.println(ok);
  }
}
```

15 sorted() multi-field compare + noneMatch() — Negative validation

```
import java.util.*;

public class SortedMultiFieldNoneMatch {

  static class Employee {
    String name; int age;
    Employee(String name, int age) { this.name = name; this.age = age; }
  }

  public static void main(String[] args) {
    boolean ok = List.of(
          new Employee("Anurag", 30),
          new Employee("Anurag", 25),
          new Employee("Ravi", 35)
    ).stream()
     .sorted(Comparator.comparing((Employee e) -> e.name).thenComparingInt(e -> e.age))
     .noneMatch(e -> e.age < 18);

    System.out.println(ok);
  }
}
```

```
import java.util.*;
```

```
public class PeekLoggingNoneMatch {

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      boolean ok = true;

      for (Integer n : list) {

         // peek equivalent (logging)
         System.out.println("Checking: " + n);

         // noneMatch condition
         if (n < 0) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

16 peek() logging + noneMatch() — Negative validation

```
import java.util.*;

public class PeekLoggingNoneMatch {
   public static void main(String[] args) {
      boolean ok = List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Checking: " + n))
            .noneMatch(n -> n < 0);

      System.out.println(ok);
   }
}
```

```
import java.util.*;

public class PeekDebugNoneMatch {

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);

      boolean ok = true;

      for (Integer n : list) {
```

```
        // peek equivalent (debugging)
        if (n == 20) {
            System.out.println("Debug hit");
        }

        // noneMatch condition
        if (n < 10) {
            ok = false;
            break;
        }
    }

    System.out.println(ok);
  }
}
```

17 peek() debugging + noneMatch() — Negative validation

```
import java.util.*;

public class PeekDebugNoneMatch {
  public static void main(String[] args) {
    boolean ok = List.of(10, 20, 30).stream()
          .peek(n -> { if (n == 20) System.out.println("Debug hit"); })
          .noneMatch(n -> n < 10);

    System.out.println(ok);
  }
}
```

```
import java.util.*;

public class PeekMetricsNoneMatch {

  public static void main(String[] args) {

    List<Integer> list = Arrays.asList(5, 10, 15);

    int seen = 0;                                    // AtomicInteger replaced with simple counter
    boolean ok = true;

    for (Integer n : list) {
```

```
        // peek equivalent (metrics)
        seen++;

        // noneMatch condition
        if (n <= 0) {
            ok = false;
            break;
        }
    }

    System.out.println("OK: " + ok);
    System.out.println("Processed: " + seen);
  }
}
```

18 peek() metrics + noneMatch() — Negative validation

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsNoneMatch {
  public static void main(String[] args) {
    AtomicInteger seen = new AtomicInteger();

    boolean ok = List.of(5, 10, 15).stream()
          .peek(n -> seen.incrementAndGet())
          .noneMatch(n -> n <= 0);

    System.out.println("OK: " + ok);
    System.out.println("Processed: " + seen.get());
  }
}
```